

第 3 卷 DP 建模与模型复用

考试速查：主查 DP 怎么想：从暴力搜索找状态、加记忆化、再表推；覆盖背包、LIS/LCS/LCIS、区间、树形、DAG、状压、数位和高阶索引。

Contents

第 3 卷 DP 建模与模型复用	4
考试速查定位	4
DP-00: DP 总流程	4
考场总流程	5
五问检查卡	6
建模 8 问：从题面到代码前必须过一遍	6
初始化常用表	6
初始化与答案位置总表	6
转移顺序心法：pull、push 与依赖图	7
复杂度预算卡	7
最小表推骨架	7
暴力/部分分替代	7
DP 调试口令	8
DP-01: DP 路由表	8
一眼路由表	9
背包细分表	10
不是 DP 的常见误判	10
DP-02: 状态句式库	11
句式 1: 线性序列	12
句式 2: 选/不选	12
句式 3: 背包	12
句式 4: 两个序列	12
句式 5: 网格	12
句式 6: 区间	12
句式 7: 树形	12
句式 8: DAG	13
句式 9: 状压集合	13
句式 10: 数位	13
状态完整性检查	13
DP-03: DFS -> 记忆化 -> 表推升级图	13
升级图	14
什么时候可以直接加 memo	15
通用暴力 DFS 版本	15
通用记忆化版本	15
通用表推版本	15
从 DFS 到表推的对照表	15
map<tuple> 救场版本	16
什么时候不改表推	16
DP-03B: 状态增维/升维路由	16
核心原则	17
一眼增维表	18
维度 1: last	18
维度 2: mask	19
维度 3: cnt/rest	19
维度 4: tight	20

维度 5: leading	20
维度 6: color	20
维度 7: parity/mod	21
维度 8: parent/prev	21
维度 9: phase	22
map<tuple> 完整状态救场	22
状态完整性检查卡	23
DP-04: 线性 DP	23
DP-05: 选/不选 DP	25
DP-06: 0/1 背包	28
DP-07: 完全背包	30
DP-08: 分组背包	33
DP-09: LCS 最长公共子序列	36
DP-10: 编辑距离	38
DP-11: LIS 最长递增子序列	40
DP-12: 网格 DP	43
DP-13: 区间 DP	45
DP-14: 树形 DP	48
DP-15: DAG DP	51
DP-16: 状压 DP	54
DP-17: 数位 DP	57
DP-18: DP + 数据结构优化	60
先问: 这个 DP 真的能用数据结构优化吗?	61
DP-19: DP 模型拼接食谱	64
拼接总原则	65
食谱 1: 区间 DP + PrefixSum	65
食谱 2: LIS/线性 DP + 树状数组	66
食谱 3: 树形 DP + 背包合并	67
食谱 4: 状压 DP + Floyd/Dijkstra	68
食谱 5: DAG DP + Topo	69
食谱 6: 数位 DP + 取模/组合	70
拼接检查卡	71
DP-20: 计数 / 可行性 DP	72
计数和可行性的本质区别	73
从暴力到计数/可行性	73
取模套路	73
判定套路	74
套路 1: 0/1 背包方案数	74
套路 2: 完全背包方案数	75
套路 3: 可行性背包	75
套路 4: 分组计数 / 可行性	76
套路 5: 网格路径计数 / 可达性	76
套路 6: DAG 路径计数	77
套路 7: 数位 DP 计数	77
初始化总表	78
转移总表	78
答案位置总表	78

DP-21: P1874 快速求和建模例题	79
题意压缩	80
第一阶段: 先写暴力, 找决策点	80
第二阶段: 找重叠子问题	80
第三阶段: 定义状态并验可行性	80
第四阶段: 推导转移	81
第五阶段: 细节与陷阱	81
DP-22: 编辑距离建模例题	84
题意压缩	85
第一阶段: 从暴力尝试开始, 找决策点	85
第二阶段: 找到无后效性	85
第三阶段: 定义状态并验可行性	85
第四阶段: 倒推最后一步	86
情况一: 最后字符相等	86
情况二: 最后字符不相等	86
第五阶段: 初始化	86
从 P1874 到编辑距离: DP 五步法	87
DP-23: LIS/LCS 常见变体速查	89
1. 变体路由表	90
1A. 变体建模口令	90
2. LIS 严格/不下降短模板	90
3. 输出一条 LIS	91
4. 二维偏序: 排序后一维 LIS	91
5. 输出一条 LCS	92
6. 统计 LIS 个数	92
7. 带权 LIS	93
8. 山形/合唱队形	93
9. 最长公共子串	93
10. LCIS 最长公共上升子序列	94
DP-24: 背包常见变体总表	95
1. 背包变体路由表	96
1A. 背包变体建模口令	96
2. 多重背包: 二进制拆分成 0/1	97
3. 二维背包	97
4. 价值维度背包	97
5. 可行性背包 bitset	98
6. 组合数 vs 排列数	98
7. 至少装满的最小代价	98
8. 0/1 背包路径恢复: 二维 take	99
DP-25: 两道例题的暴力 DFS 到记忆化搜索	100
1. 考场口令	101
2. P1874 快速求和: 记忆化 DFS 完整代码	101
3. 编辑距离: 记忆化 DFS 完整代码	102
4. 什么时候记忆化能直接满分	104
5. 什么时候只能拿部分分	104
6. 数组 memo 与 map memo 的选择	104
7. 考场策略总结	104
DP-26: 发现有后效性怎么办: 升维吸收关键历史	105
1. 核心原则	106
2. 例题一: 网格最大收益, 不能连续向下走两步	106
3. 例题二: 爬楼梯, 不能连续跳同样步数	108
4. 例题三: 恰好选 K 个数, 且不能选相邻	109
5. 例题四: 网格最大收益, 最多一次收益翻倍	110
6. 例题五: 股票买卖, 冷冻期一天	111
7. 升维清单	112
8. 从 DFS 角度理解升维	112
DP-27: 高阶 DP 优化与少见模型索引	113
1. 高阶 DP 路由表	114

2. 可靠使用顺序	114
3. SOS DP 最短模板	114
4. 分治 DP 优化只做索引	115
5. Knuth 优化只做索引	115
6. 斜率优化只做索引	115
7. 轮廓线/插头 DP 只放最后	115
8. 自动机 DP	115

第 3 卷 DP 建模与模型复用

考试速查定位

第 3 卷 DP 建模与模型复用

- **这是第几卷：** 03。
- **主要查什么：** 主查 DP 怎么想：从暴力搜索找状态、加记忆化、再表推；覆盖背包、LIS/LCS/LCIS、区间、树形、DAG、状压、数位和高阶索引。
- **什么时候翻：** 疑似 DP、需要从暴力升级到记忆化/表推、不会定义状态时翻。
- **题面关键词：** DP 五问；线性；背包；LIS/LCS/LCIS；区间；树形；DAG；状压；数位；例题推导。

自动由 03_modules/DP-*.md 重建。定位是初学者友好的 DP 路由、建模、暴力到记忆化再到表推。

DP-00: DP 总流程

模块编号: DP-00

模块名称: DP 总流程

标签: DP、路由、状态设计、部分分、记忆化

一句话用途: 遇到疑似 DP 题时，按固定步骤从题面信号走到暴力 DFS、记忆化或表推 DP。

题面触发词:

- “最大/最小/方案数/是否存在”
- “前 i 个” “走到第 i 个” “容量/预算/次数限制”
- “两个序列” “区间合并” “树上选择” “集合访问” “数字范围 1..N”
- 数据范围不像暴力，但状态有重复

什么时候用:

- 题目要求全局最优、计数或可行性。
- 每一步有选择，且后续答案只依赖少量状态。
- 暴力搜索会反复到达相同局面。

不要什么时候用:

- 最少步数且每条边代价相同，优先 BFS。
- 明显是排序、贪心或数据结构维护，不需要记录历史状态。
- 状态会形成正权/负权环，不能直接套普通 DP。

复杂度:

- 估算公式: 状态数 * 每个状态转移数。
- 表推 DP 与记忆化 DFS 的理论状态数通常一致，常数不同。

数据范围参考:

数据范围	优先考虑
$n \leq 20$	DFS、记忆化、状压 DP
$n \leq 40$	折半枚举、状态压缩、小维 DP
$n \leq 300$	区间 DP、Floyd 类 $O(n^3)$
$n \leq 3000/5000$	$O(n^2)$ 线性 DP、LCS、朴素 LIS
$n \leq 2e5$	$O(n \log n)$ 、滚动数组、DP + 数据结构优化

依赖的标准容器:

- `vector<ll>`、`vector<vector<ll>>`
- `map<tuple<...>, ll>`: 状态复杂时先用它保命。
- `queue<int>`: DAG 拓扑或 BFS 状态搜索。
- Graph: 树形 DP、DAG DP。

输入如何整理:

- 序列: 整理成 `vector<ll> a(n + 1)`, 默认 1-index。
- 字符串: 默认 0-index; 做 LCS/编辑距离时常用 `s[i - 1]`。
- 图/树: 优先使用统一 Graph; 若为了速抄写局部 `g`, 必须注明它是从 `G.g` 抽出的简化邻接表。
- 区间代价: 优先预处理前缀和或 `cost(l, r)`。

接口:

```
using ll = long long;
const ll LINF = 4'000'000'000'000'000'000LL; // 与主骨架统一, 作为 Long Long 无穷大哨兵
```

```
// 最大值题: 不可达用 -LINF
// 最小值题: 不可达用 LINF
// 计数题: 初始 0, 成功边界 1, 必要时取 MOD
// 可行性题: bool / char
```

输出能力:

- 最大值/最小值
- 方案数
- 是否可行
- 可选: 记录路径或选择方案

下游可接:

- DP-01 路由表
- DP-02 状态句式库
- DP-03 DFS -> 记忆化 -> 表推升级图
- DP-03B/DP-26 状态增维/升维: 状态有后效性时使用。
- 各 DP 模型卡片

可拼接模块:

- BRUTE 记忆化搜索
- PrefixSum / 树状数组 / SegmentTree
- Graph / Topo / Tree
- Compressor
- Floyd / Dijkstra

考场总流程

1. 看数据范围
-> 判断能不能暴力、能不能 $O(n^2)$ 、是否需要优化。
2. 看处理对象
-> 序列 / 背包 / 两个序列 / 网格 / 区间 / 树 / DAG / 集合 / 数字位。
3. 查 DP 路由表
-> 先匹配现成模型, 不空想。
4. 套状态句式
-> “处理到哪里 + 还记得住什么 + dp 值代表什么”。
5. 写初始化
-> 边界状态是什么, 不合法状态用什么值。
6. 写转移
-> 当前状态从哪里来, 或当前状态能去哪里。
7. 确认答案位置
-> `dp[n]` / `max(dp)` / `dp[n][m]` / `dp[full][last]` / `dfs(0, ...)`。

8. 先交能写对的版本
-> 暴力 DFS -> 记忆化 -> 表推 -> 优化。

五问检查卡

状态表示什么?
初始状态是什么?
转移从哪里来?
循环顺序是否保证依赖已计算?
答案从哪里取?

建模 8 问：从题面到代码前必须过一遍

问题	要写下来的答案	常见例子
1. 处理对象是什么?	序列、两个字符串、网格、区间、树、集合、数字位	背包是物品序列，编辑距离是两个字符串
2. 进度维度是什么?	$i, i, j, l, r, u, \text{mask}, \text{pos}$	LCS 是 i, j , 区间 DP 是 l, r
3. 还要记住什么?	容量、数量、上一状态、是否用过机会、余数	有后效性就去 DP-03B/DP-26 升维
4. dp 值是什么类型?	最大值、最小值、方案数、可行性	决定初值是 $-\text{LINF}/\text{LINF}/0/\text{false}$
5. base case 是什么?	空前缀、容量 0、起点、叶子、空集合	背包 $\text{dp}[0]=0/1/\text{true}$
6. 状态从哪里来?	前驱状态 pull, 或当前状态 push 到后继	网格从上/左来, 背包从旧容量来
7. 计算顺序是什么?	小到大、长度递增、拓扑序、DFS memo	区间按长度, DAG 按拓扑
8. 答案在哪里?	$\text{dp}[n]$ 、 $\max(\text{dp})$ 、 $\text{dp}[n][m]$ 、 $\text{dp}[\text{full}][\text{last}]$ 、 $\text{sum}(\text{dp})$	LIS 是 $\max(\text{dp}[i])$, LCS 是 $\text{dp}[n][m]$

小例子：不相邻选数最大和。

处理对象：一个序列 $a[1..n]$

进度维度：处理到第 i 个数

关键历史：第 i 个数选没选，会影响 $i+1$ 能不能选

状态定义： $\text{dp}[i][0/1]$ = 处理完前 i 个数， i 不选/选时的最大和

base: $\text{dp}[0][0]=0, \text{dp}[0][1]=-\text{INF}$

转移: $\text{dp}[i][0]=\max(\text{dp}[i-1][0], \text{dp}[i-1][1]); \text{dp}[i][1]=\text{dp}[i-1][0]+a[i]$

答案: $\max(\text{dp}[n][0], \text{dp}[n][1])$

这个例子对应“有后效性就升维”：只写 $\text{dp}[i]$ 不够，因为不知道第 i 个是否被选；把选/不选放进状态后，未来就只看当前状态，不再回头问完整历史。

如果第 3 问答不出来，先写暴力 DFS，把所有影响未来的量放进函数参数，再决定哪些参数能成为 DP 下标。

初始化常用表

目标	初值	合法起点
最大值	$-\text{LINF}$	起点设 0 或题意值
最小值	LINF	起点设 0 或题意值
方案数	0	起点设 1
可行性	false	起点设 true

初始化与答案位置总表

模型	常见初始化	答案位置	易错点
线性 DP	$\text{dp}[0]=0$ 或 $\text{dp}[1]=a[1]$	$\text{dp}[n]$ 或 $\max(\text{dp})$	必须以 i 结尾时答案常是 $\max$
0/1 背包不要求装满	$\text{dp}[j]=0$	$\text{dp}[w]$	全 0 表示可以不选
恰好装满最大值	$\text{dp}[0]=0$, 其他 $-\text{LINF}$	$\text{dp}[w]$	不可达不能参与转移
完全背包最少件数	$\text{dp}[0]=0$, 其他 LINF	$\text{dp}[\text{target}]$	不能初始化全 0
LCS/编辑距离	第 0 行/列	$\text{dp}[n][m]$	字符访问用 $i-1/j-1$
LIS 朴素	$\text{dp}[i]=1$	$\max(\text{dp}[i])$	不是 $\text{dp}[n]$
网格 DP	起点 $\text{dp}[1][1]$	$\text{dp}[n][m]$	起点/终点障碍要特判
区间 DP	单点或空区间	$\text{dp}[1][n]$	先枚举长度

模型	常见初始化	答案位置	易错点
树形 DP	叶子/子树初值	<code>dp[root][state]</code> 或取 <code>max/min</code>	父子状态约束别漏
状压 DP	<code>dp[初始 mask][start]</code>	<code>dp[full][last]</code> 聚合	<code>mask</code> 和 <code>last</code> 常常都要

转移顺序心法：pull、push 与依赖图

pull: 当前状态从哪些前驱状态汇总而来。

push: 当前状态向哪些后继状态更新。

考场选择:

- 网格、LCS、编辑距离: pull 通常更自然。
- 背包、状压、DAG: push/pull 都可, 选不容易写错的。
- 记忆化 DFS: 不用手排循环顺序, 但必须保证状态无环或能处理环。

滚动数组判断:

如果 `dp[i]` 只依赖 `i-1`, 可以滚动。

如果一维压缩后会在同一轮重复使用当前物品, 循环方向必须调整。

0/1 背包倒序: 防止一个物品用多次。

完全背包正序: 允许同一种物品用多次。

复杂度预算卡

模型	状态数	每状态转移	常见复杂度
编辑距离/LCS	$n*m$	$O(1)$	$O(nm)$
P1874 快速求和	$len*target$	枚举下一段 $O(len)$	$O(len^2*target)$
0/1 背包	$n*w$	$O(1)$	$O(nw)$
区间 DP	n^2	枚举断点 $O(n)$	$O(n^3)$
状压 TSP	2^n*n	枚举下一点 $O(n)$	$O(2^n*n^2)$
树上背包	子树容量状态	合并容量	常见 $O(nK^2)$ 或优化到 $O(nK)$

内存估算:

int 数组: 状态数 * 4 字节

long long 数组: 状态数 * 8 字节

1e7 个 int 约 40MB

1e7 个 long long 约 80MB

如果估算爆了:

- 先看能否滚动数组。
- 再看能否换维度, 例如容量太大但价值和小。
- 再看是否只访问少量状态, 用 map/unordered_map 记忆化拿部分分。

最小表推骨架

```
vector<ll> dp(n + 1, -LINF);
```

```
dp[0] = 0;
```

```
for (int i = 1; i <= n; i++) {
    // 在这里按题意写 dp[i] 的转移
}
```

```
cout << dp[n] << '\n';
```

暴力/部分分替代

- $n \leq 20$: 直接 DFS 枚举选择。
- 状态参数明确: 加 memo, 通常能从小数据升到中档。
- 只会部分模型: 先写最接近的记忆化版本, 不强行表推。

升级方向:

暴力 DFS

- > 参数完整后加 memo
- > 状态范围清楚后改数组 memo
- > 依赖顺序清楚后改表推 DP
- > 转移太慢时加前缀和 / 树状数组 / 线段树 / 单调队列

常见坑：

- dp 初值与目标不匹配：最大值题不能默认 0，可能全负。
- 答案位置错：有些题是 $\max(dp[i])$ ，不是 $dp[n]$ 。
- 循环顺序错：0/1 背包容量要倒序，完全背包容量要正序。
- 记忆化状态漏参数：例如漏 last、mask、tight。
- 多测没有清空 dp/memo/vis。
- 同样下标未来不唯一，说明状态有后效性，要去 DP-03B/DP-26 升维。

DP 调试口令

样例不过，按顺序查：

1. 状态句子是否唯一？
2. 初值是否和最大/最小/计数/可行性匹配？
3. 不可达状态有没有参与转移？
4. 循环顺序是否保证依赖已算好？
5. 一维压缩方向是否正确？
6. 答案位置是 $dp[n]$ 、max 还是求和？
7. 下标是否 0/1-index 混乱？
8. 取模和 long long 是否处理？
9. 多组数据是否清空？

最小测试样例：

n=1
资源 =0
所有选择都非法
所有选择都合法
答案可能为负
存在重复状态

DP-01: DP 路由表

模块编号: DP-01

模块名称: DP 路由表

标签: DP、模型识别、题面触发词、复杂度

一句话用途: 把题面信号直接路由到可复用的 DP 模型卡片。

模板代码: 本模块是 DP 路由表，不放完整代码；具体可抄模板按路由结果去 DP-04 到 DP-27。

题面触发词：

- “每个物品选或不选”
- “容量/预算/重量/费用”
- “两个字符串/两个序列”
- “区间合并/删除/两端取”
- “树上选点/覆盖/染色”
- “集合访问/所有点/钥匙状态”
- “上界很大，问 $1..N$ 中满足条件的数”
- “字符串切分成若干数字段，段和/代价达到目标”

什么时候用：

- 读题 1 分钟内无法直接写转移时，先查表。
- 数据范围暗示不能纯暴力，但题面有典型结构。

不要什么时候用：

- 题目只是单次求和、排序、最短路、并查集连通性。

- 题面要求在线修改与查询，优先看数据结构卷。

复杂度：

- 本表只负责路由；真正复杂度见对应模型。

数据范围参考：

- $n \leq 20$ 且集合：状压 DP。
- $n \leq 300$ 且区间：区间 DP。
- $n, m \leq 5000$ 且两个序列：LCS/编辑距离。
- $W \leq 1e5$ 且容量：背包。
- $n \leq 2e5$ ：考虑优化 DP 或非 DP 模型。

依赖的标准容器：

- vector
- `map<tuple<...>, ll>`
- Graph

输入如何整理：

- 先标出对象：序列、物品、网格、区间、树、DAG、集合、数字。
- 再标出目标：最大、最小、方案数、可行性。
- 最后标出约束：容量、次数、相邻关系、顺序限制。

接口：

题面触发词 + 数据范围 -> 模型编号 -> 状态句式 -> 三版本路径

输出能力：

- 模型候选。
- 优先使用的状态句式。
- 可先交的部分版本。

下游可接：

- DP-02 状态句式库
- DP-03 升级图
- DP-04 到 DP-27 模型卡片

可拼接模块：

- BRUTE-07/08/09 记忆化搜索
- DS PrefixSum/树状数组/SegmentTree
- GRAPH Topo/Tree/Floyd

一眼路由表

看到题面信号	优先模型	状态句式入口	注意
前 i 个、最大收益、最小代价	DP-04 线性 DP	<code>dp[i]</code>	答案可能是 <code>max(dp)</code>
每个元素选或不选	DP-05 选/不选 DP	<code>dfs(i, state)</code> 或 <code>dp[i][j]</code>	小数据先 DFS
容量、重量、预算、价值	DP-06/07/08/24 背包	<code>dp[j]</code>	看能否重复选、是否分组/多重/多维
每个物品最多一次	DP-06 0/1 背包	<code>dp[j]</code>	容量倒序
每种物品无限个	DP-07 完全背包	<code>dp[j]</code>	容量正序
每组最多选一个	DP-08 分组背包	<code>dp[j] + ndp</code>	组内不要相互影响
每种物品有限个/多重/二维/价值	DP-24 背包变体	<code>dp[j] / dp[value]</code>	先判循环方向和初始化
两个序列，公共子序列	DP-09 LCS / DP-23 变体	<code>dp[i][j]</code>	子序列和子串别混
两个序列，公共，且要求上升/递增	DP-23 LCIS	<code>dp[j]</code>	不是先 LCS 再 LIS，扫描第二个序列维护 best
插入、删除、替换	DP-10 编辑距离 / DP-22 建模例题	<code>dp[i][j]</code>	初始化第一行/列
最长递增/不下降子序列	DP-11 LIS / DP-23 变体	<code>dp[i]</code> 或 <code>d[len]</code>	<code>lower_bound/upper_bound</code> 区分
网格只能右/下走	DP-12 网格 DP	<code>dp[i][j]</code>	障碍格初始化
同一位置但历史不同，后续合法选择不同	DP-03B/DP-26 状态升维	<code>dp[位置][关键历史]</code>	有后效性就把关键历史进状态

看到题面信号	优先模型	状态句式入口	注意
区间合并、括号、回文、两端取	DP-13 区间 DP	dp[l][r]	按长度枚举
树上选择、覆盖、染色	DP-14 树形 DP	dp[u][state]	后序 DFS
树上选 K 个、子树容量合并	DP-14 + DP-19 树上背包	dp[u][k]	子树合并时容量倒序
每个点都可能作为根/中心	TREE-02 换根 DP	down/up 或二次 DFS	第一遍子树，第二遍换根
依赖关系、有向无环、路径计数	DP-15 DAG DP	dp[u]	先拓扑排序
$n \leq 20$ ，集合、访问所有点	DP-16 状压 DP	dp[mask][last]	内存 $2^n * n$
1..N、数位限制、上界很大	DP-17 数位 DP	dfs(pos,tight,leading,state)	tight 状态通常不缓存
dp[i]=min/max dp[j]+cost	DP-18 DP+ 数据结构优化	dp[i] + 查询结构	先写 $O(n^2)$
且查询范围			
朴素 DP 会写但超时，出现 SOS/斜率/分治/Knuth/轮廓线	DP-27 高阶 DP 索引	先写朴素式	只作为索引，不要硬套
字符串切分成数字段，凑目标和/最小切分代价	DP-21 P1874 建模例题	dp[i][sum]	这不是数位 DP，是前缀切分 DP
DP 方程想不出，但 DFS 能写	DP-25 DFS+ 记忆化例题	dfs(可变参数)	函数参数就是状态

背包细分表

题面	模型	容量循环
每个物品只能选一次	0/1 背包	for j=W..w[i]
每个物品可以选无限次	完全背包	for j=w[i]..W
物品分成若干组，每组选 0 或 1 个	分组背包	每组用 ndp 或组内倒序小心写
每种物品最多 cnt[i] 个	多重背包	DP-24: 二进制拆分成 0/1
两个容量/费用限制	二维背包	DP-24: 两个容量都倒序
容量很大但价值和小	价值维度背包	DP-24: dp[value]= 最小重量
问能否凑出容量	可行性背包	bool dp[j]
问方案数	计数背包	dp[j] += dp[j-w]
问至少达到容量	至少装满背包	DP-24: 容量封顶或扩容
要输出选了哪些物品	背包路径恢复	初学优先二维 take

不是 DP 的常见误判

题面	更可能是
最少操作次数，每一步代价相同	BFS
边权非负最短路	Dijkstra
连通性、合并集合	DSU
区间修改查询	树状数组/SegmentTree
“每次选当前最小/最大” 且有明确局部规则	GREEDY-00/01 贪心 + 堆
只是最大/最小目标，但选择影响容量/历史	DP 或记忆化，先看 GREEDY-01 判别卡

暴力/部分替代：

- 路由不确定时，先写 DFS，把状态参数写全。
- 如果 DFS 有重复状态，加 map<tuple<...>, ll>。
- 能跑小数据后再翻对应模型改表推。

升级方向：

路由表候选 1 个 -> 直接套模型

路由表候选多个 -> 对照数据范围和“是否有容量/区间/集合”

完全匹配不上 -> DP-03 从 DFS 造模型

常见坑：

- 只看关键词不看数据范围：比如 $n \leq 20$ 的“选/不选”可能是状压或 DFS。
- “最长路径”在一般有环图不是普通 DP，DAG 才能拓扑 DP。
- “区间查询”不等于区间 DP，可能只是数据结构。

最小测试样例：

拿题面摘要，不写代码，只做：

1. 标出触发词
2. 标出数据范围
3. 选 1 个主模型
4. 写一句状态句式

DP-02：状态句式库

模块编号：DP-02

模块名称：状态句式库

标签：DP、状态设计、句式模板

一句话用途：不会设状态时，直接套“处理到哪里 + 记住什么 + dp 值含义”的固定句式。

模板代码：本模块是状态定义句式库，不放完整代码；定好状态后去对应模型模块抄循环或记忆化模板。

题面触发词：

- “前 i 个”
- “剩余容量/已用容量”
- “最后一个是谁”
- “区间 $[l, r]$ ”
- “子树”
- “集合 $mask$ ”
- “数位 pos ”

什么时候用：

- 已经选出 DP 模型，但不知道 dp 下标怎么解释。
- 写转移前，需要确认状态是否完整。

不要什么时候用：

- 状态含义无法排除路径历史影响时，不要硬套；先回到 DFS 参数。
- 有环依赖时，不要把句式当成拓扑顺序。

复杂度：

- 状态句式不决定复杂度；复杂度由状态数量和转移数量决定。

数据范围参考：

- 一维状态通常支持更大 n 。
- 二维 $n*m$ 要看 $n, m \leq 5000$ 左右是否能接受内存。
- 2^n 状态通常要求 $n \leq 20$ 。

依赖的标准容器：

- `vector`
- `vector<vector<ll>>`
- `map<tuple<...>, ll>`

输入如何整理：

- 把题面中的对象改名为统一变量： i 、 j 、 l/r 、 u 、 $mask$ 、 pos 。
- 把目标改成四类之一：最大值、最小值、方案数、可行性。

接口：

$dp[\text{下标}]$ 表示：处理范围 + 附加记忆 + 值的类型。

输出能力：

- 直接可写到题解或代码注释里的状态定义。
- 帮助检查是否漏掉必要参数。

下游可接：

- 全部 DP 模型。
- DP-03B/DP-26：如果状态句式写完后，同样下标下未来不唯一，需要状态增维/升维。

可拼接模块：

- BRUTE 记忆化状态卡片。

句式 1：线性序列

$dp[i]$ 表示：处理完前 i 个元素时，_____ 的最大值/最小值/方案数/是否可行。

常见填空：

- 以第 i 个结尾的最大收益。
- 前 i 个中的最优答案。
- 到达位置 i 的最小代价。

句式 2：选/不选

$dp[i][j]$ 表示：处理完前 i 个元素，并且资源/数量/状态为 j 时，_____。

如果表推顺序不清，先写：

$dfs(i, j)$ 表示：从第 i 个元素开始，当前资源/数量/状态为 j 时，后续能得到的答案。

句式 3：背包

$dp[j]$ 表示：当前已处理若干物品，容量/花费为 j 时的最大价值/方案数/是否可行。

初始化句：

$dp[0] = 0/1/true$ ：容量为 0 时什么都不选是合法起点。

其他为 $-LINF/0/false$ ：表示暂时不可达。

句式 4：两个序列

$dp[i][j]$ 表示：只考虑 A 的前 i 个和 B 的前 j 个时，_____。

用于：

- LCS：最长公共子序列长度。
- 编辑距离：把前 i 个变成前 j 个的最小操作数。

句式 5：网格

$dp[i][j]$ 表示：走到格子 (i, j) 时，_____。

常见填空：

- 路径数量。
- 最小路径和。
- 最大收益。

句式 6：区间

$dp[l][r]$ 表示：只考虑闭区间 $[l, r]$ 时，_____ 的最优值/方案数。

遍历句：

先枚举区间长度 len ，再枚举左端点 l ，右端点 $r = l + len - 1$ 。

句式 7：树形

$dp[u][state]$ 表示：只考虑 u 的子树，并且 u 自己处于 $state$ 状态时，_____。

常见 $state$ ：

- 0/1：不选/选 u 。
- 0/1/2：被父亲覆盖、被儿子覆盖、未覆盖。
- 颜色编号。

句式 8: DAG

$dp[u]$ 表示: 到达 u 的最优值/方案数。

或:

$dp[u]$ 表示: 从 u 出发能得到的最优值/方案数。

选哪一个取决于转移方向是否顺手。

句式 9: 状压集合

$dp[mask][last]$ 表示: 已经选择/访问的集合是 $mask$, 并且最后停在 $last$ 时, ____。

如果不需要最后位置:

$dp[mask]$ 表示: 已经选择集合 $mask$ 时, ____。

句式 10: 数位

$dfs(pos, tight, leading, state)$ 表示: 当前处理到第 pos 位, 是否贴上界 $tight$, 是否仍是前导零 $leading$, 并且附加状态为 $state$ 时, 后

常见附加状态:

- 各位和。
- 上一位数字。
- 是否出现某数字。
- 余数。

状态完整性检查

给定这些下标后, 未来答案是否唯一?

如果同样下标但因为“上一位/最后位置/已选集合/剩余次数”不同导致答案不同, 就必须把它加入状态。

这就是“无后效性”检查:

同样状态 $\rightarrow$ 后续答案唯一: 状态可以用。

同样状态 $\rightarrow$ 后续答案不唯一: 状态有后效性, 去 DP-03B/DP-26 升维。

暴力/部分分替代:

- 状态句式写不顺时, 先把句式改成 $dfs(\dots)$ 表示...
- DFS 参数一旦完整, 直接加 memo。

升级方向:

- $dfs(i, rest) \rightarrow memo[i][rest] \rightarrow dp[i][rest]$ 或滚动 $dp[rest]$ 。
- $dfs(l, r) \rightarrow dp[l][r]$, 按长度表推。
- $dfs(u, state) \rightarrow$ 树形后序 DP。
- $dfs(i, last/used/phase) \rightarrow$ DP-03B/DP-26 状态升维。

常见坑:

- $dp[i]$ 同时想表达“前 i 个最优”和“必须选 i ”, 转移会混乱。
- 方案数和最优值初值不同, 不能混用。
- 字符串 0-index 和 DP 的 i/j 前缀长度容易差 1。

最小测试样例:

任选一个模型, 先只写状态句式, 不写代码。

检查三个问题:

处理到哪里?

还记住什么?

dp 值代表最大、最小、方案数, 还是可行性?

DP-03: DFS $\rightarrow$ 记忆化 $\rightarrow$ 表推升级图

模块编号: DP-03

模块名称: DFS $\rightarrow$ 记忆化 $\rightarrow$ 表推升级图

标签：DP、暴力、记忆化、表推、升级路径

一句话用途：模型匹配不上时，先用 DFS 表达选择，再把 DFS 参数升级成 memo 和 DP 表。

题面触发词：

- “每一步有多个选择”
- “同一局面会重复出现”
- “表推顺序想不清”
- “想先拿部分分”

什么时候用：

- 题面不像标准背包/LCS/区间，但能写出递归枚举。
- 想从可运行的小数据暴力开始，逐步升级。

不要什么时候用：

- 状态图存在环且没有拓扑顺序或最短路性质。
- 递归深度可能到 $2e5$ 且无法改迭代。
- DFS 参数没有包含完整历史信息。

复杂度：

- 暴力：选择数的指数级。
- 记忆化：不同状态数 * 每状态选择数。
- 表推：状态表大小 * 转移数。

数据范围参考：

- $n \leq 20$ ：暴力或状压。
- 状态数 $\leq 1e7$ 且转移少：数组 memo/表推。
- 状态复杂且稀疏：map<tuple<...>, ll> 先保命。

依赖的标准容器：

- map<tuple<...>, ll>
- vector<vector<ll>>
- vector<vector<int>> vis

输入如何整理：

- 先把每一步的选择列出来。
- 再把“未来答案需要知道的东西”列为 DFS 参数。

接口：

```
ll dfs(State s);
```

输出能力：

- 暴力小数据。
- 记忆化中档分。
- 表推满分或接近满分。

下游可接：

- 全部 DP 模型。

可拼接模块：

- BRUTE-07 记忆化总论
- BRUTE-09 map<tuple, ...> 记忆化
- DP-18 优化 DP

升级图

题面选择

- > 写 dfs(当前进度, 资源, 必须记住的历史)
- > 检查参数是否完整
- > 加 memo: 同参数直接返回
- > 把 dfs 参数变成 dp 下标
- > 把 dfs 终止条件变成初始化
- > 把 dfs 枚举选择变成转移
- > 找到依赖顺序后表推

什么时候可以直接加 memo

记忆化的判断口令：

同样的 dfs 参数 -> 未来可选集合完全一样 -> 最优答案也完全一样。

如果同样的参数下，未来还会被“没放进参数里的历史”影响，就不能直接 memo，必须升维。常见漏掉的历史：

last：上一次选了谁、上一步方向、上一个颜色。

used/mask：哪些点/物品已经用过。

path property：当前连续次数、是否已经触发过某个限制。

反例：

dfs(i) 表示走到第 i 步的最好结果。

题目限制“不能连续向下走两步”。

如果不把 last_direction 放进参数，dfs(i) 不知道上一步是不是向下，后续选择集合不同，memo 会错。

正确状态应类似 dfs(i, last_direction)。

通用暴力 DFS 版本

```
11 dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    11 ans = -LINF;
    ans = max(ans, dfs(i + 1, rest)); // 不选
    ans = max(ans, val[i] + dfs(i + 1, rest - cost[i])); // 选
    return ans;
}
```

通用记忆化版本

vector<vector<ll>> memo;

vector<vector<int>> vis;

```
11 dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    if (vis[i][rest]) return memo[i][rest];
    vis[i][rest] = 1;

    11 ans = -LINF;
    ans = max(ans, dfs(i + 1, rest));
    ans = max(ans, val[i] + dfs(i + 1, rest - cost[i]));
    return memo[i][rest] = ans;
}
```

通用表推版本

```
vector<vector<ll>> dp(n + 2, vector<ll>(W + 1, -LINF));
for (int rest = 0; rest <= W; rest++) dp[n + 1][rest] = 0;

for (int i = n; i >= 1; i--) {
    for (int rest = 0; rest <= W; rest++) {
        dp[i][rest] = max(dp[i][rest], dp[i + 1][rest]);
        if (rest >= cost[i]) {
            dp[i][rest] = max(dp[i][rest], val[i] + dp[i + 1][rest - cost[i]]);
        }
    }
}

cout << dp[1][W] << '\n';
```

从 DFS 到表推的对照表

DFS 元素	表推 DP 元素
dfs 参数	dp 下标
非法状态	不转移或设 $LINF/-LINF$
结束状态	初始化
递归选择	状态转移
返回值	dp 值
初始调用	答案位置

map<tuple> 救场版本

```
map<tuple<int,int,int>, ll> memo;

ll dfs(int i, int rest, int last) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    auto key = make_tuple(i, rest, last);
    if (memo.count(key)) return memo[key];

    ll ans = -LINF;
    // 按题意枚举选择
    return memo[key] = ans;
}
```

什么时候不改表推

状态范围不清楚。
 依赖顺序会绕回来。
 只需要中档分，memo 已经过样例。
 表推写法比 memo 更容易错。

常见坑：

- 记忆化先查 memo 再判非法，可能数组越界。
- vis=1 代表“算完”，不能用于有环状态的“正在算”。
- 表推初始化忘记对应 DFS 的结束状态。
- 最大值题非法返回 0，导致选非法方案。
- 改成滚动数组后循环方向错。

暴力/部分替代：

- 用 map<tuple> 版本提交，哪怕慢，也比空着强。
- 大数据不会时，加简单特判：空集、全相同、容量为 0、n=1。

升级方向：

- 数组 memo 替换 map<tuple>。
- 二维表推替换递归。
- 滚动数组省空间。
- 用前缀和/树状数组/线段树/单调队列优化转移。

最小测试样例：

```
3 5
cost: 2 3 4
val : 3 4 5
答案: 7
```

DP-03B：状态增维/升维路由

模块编号：DP-03B

模块名称：状态增维/升维路由

标签：DP、状态设计、增维、升维、无后效性、有后效性、反例、记忆化

一句话用途：看到题面里有“未来会受某个历史信息影响”的信号，就把这个信息加进状态，让新状态重新满足无后效性。

题面触发词：

- “相邻不能相同” “上一个选择影响当前”
- “已经访问/已经拥有/已经使用过”
- “恰好 k 个” “还剩几次” “容量/预算”
- “不超过 N” “前导零” “数位限制”
- “颜色/奇偶/余数/父亲/阶段”

什么时候用：

- 已经会写 `dfs(...)`，但样例或手推发现同一个位置后续答案不唯一。
- 写 memo 前，需要检查 DFS 参数是否完整。
- 表推 DP 状态太少，转移里偷偷用了全局变量、路径数组或上一轮选择。

不要什么时候用：

- 这个信息只影响当前转移，不影响以后，可以不用放进状态。
- 这个信息能由现有状态唯一推出，不要重复增维。
- 状态数乘起来爆炸时，先判断能否换模型、压缩状态或只保留必要摘要。

复杂度：

状态数 = 每个维度取值数量相乘

总复杂度 = 状态数 * 每个状态的选择数

数据范围信号：

- 多加一维 last：通常乘 值域大小。
- 多加一维 cnt/rest/mod：通常乘 K/W/M。
- 多加一维 mask：通常乘 2^n ，要求 n 小。
- 多加 tight/leading：通常只乘 $2 * 2$ ，数位 DP 常用。

依赖的标准容器：

- vector
- array
- `map<tuple<...>, ll>`：状态维度复杂时先保命。

输入如何整理：

- 把题面限制逐条画圈。
- 每条限制问一句：未来选择还需要知道它吗？
- 需要就变成 DFS 参数或 DP 下标。

接口：

题面信号 -> 必须记住的信息 -> 状态维度 -> 漏掉会错的反例

输出能力：

- 帮你判断 `dp[i]` 是否该升级成 `dp[i][last]`、`dp[i][cnt]`、`dp[mask][last]` 等。
- 帮你发现 memo 漏参数导致的 WA。

下游可接：

- DP-02 状态句式库
- DP-03 DFS -> 记忆化 -> 表推升级图
- DP-16 状压 DP
- DP-17 数位 DP
- DP-20 计数/可行性 DP
- DP-26 有后效性例题：用“错误状态 -> 反例 -> 升维 -> 完整代码”练习。

可拼接模块：

- `map<tuple>` 记忆化。
- Graph/Tree: parent、color、phase 常见。
- Math: parity/mod 常见。

核心原则

看到什么题面信号，就给状态加什么维度。

更准确地说：

如果两个局面“处理到的位置相同”，但因为某个历史信息不同，导致后续合法选择或答案不同，那么这个历史信息必须进入状态。

换句话说：

状态有后效性 -> 找到影响未来的关键历史 -> 升维/增维 -> 新状态无后效性。

状态句式固定写成：

dp[处理到哪里][还必须记住什么] 表示：当前局面下的最优值/方案数/是否可行。

一眼增维表

题面信号	常加维度	状态句式	漏状态反例
相邻、递增、上一步影响当前	last	dfs(i, last)	二进制串相邻不能相同，只知道 i 不知道上一位，无法判断下一位能不能放 0
访问过、拥有钥匙、集合状态	mask	dp[mask][last]	同在一个房间，拿过钥匙和没拿钥匙能打开的门不同
恰好 k 个、还剩次数、容量	cnt/rest	dp[i][cnt] / dp[i][rest]	处理到第 i 个时，已选 0 个和已选 2 个，离“恰好 2 个”的答案不同
数字上界、不超过 N	tight	dfs(pos, tight, ...)	前缀等于上界时本位最多取 digits[pos]，前缀已经小于上界时本位可取 9
前导零是否还在	leading	dfs(pos, leading, ...)	统计不含数字 0 时，数字 7 前面的补位 0007 不能算出现了 0
染色、类别、模式	color	dp[u][color]	树染色时父亲是红色和蓝色，儿子可选颜色不同
奇偶、余数、整除	parity/mod	dp[i][rem]	只知道前 i 个，不知道和模 3 的余数，无法判断最后是否整除 3
树/图从哪里来	parent/prev	dfs(u, parent)	在无向树上到达 2，父亲是 1 和父亲是 3 时，能继续处理的子树不同
分阶段、冷却、持有状态	phase	dp[i][phase]	买卖股票只用 dp[i]，分不清今天结束后是手里有股票还是没股票

维度 1: last

题面信号：

相邻不能相同
 上一个数字/字符影响当前
 递增、递减、不下降
 路径最后停在哪里

状态句式：

dfs(i, last) 表示：已经处理到第 i 位/第 i 步，上一位或最后位置是 last 时，后续答案。

漏状态反例：

问长度为 n 的 0/1 串数量，要求相邻字符不同。

如果只写 dfs(i)，那么处理完第 1 位以后：

前缀 "0" 和前缀 "1" 都是 i=2，
 但前缀 "0" 下一位只能放 1，前缀 "1" 下一位只能放 0。
 后续合法选择不同，所以 last 必须进状态。

小模板：

```

11 dfs(int pos, int last) {
    if (pos == n) return 1;
    ll ans = 0;
    for (int x = 0; x <= 1; x++) {
        if (x == last) continue;
    }
}

```

```

        ans += dfs(pos + 1, x);
    }
    return ans;
}

```

维度 2: mask

题面信号:

$n \leq 20$

已经访问哪些点

已经拿到哪些钥匙

哪些任务已经完成

集合中的元素会影响后续选择

状态句式:

$dp[mask][last]$ 表示: 已经访问/选择集合 $mask$, 最后停在 $last$ 时的最优值/方案数。

$last$ 保持 $1..n$ 的业务编号; 第 $last$ 个元素对应 $mask$ 的第 $last-1$ 位。

漏状态反例:

迷宫里同一个格子 (x,y) , 如果已经拿到钥匙 A , 可以打开 A 门;
没拿到钥匙 A , 就不能打开。

只写 $dp[x][y]$ 会把两种局面合并, 导致把不能走的门当成能走, 或把能走的路剪掉。

小模板:

```

for (int mask = 0; mask < (1 << n); mask++) {
    for (int last = 1; last <= n; last++) {
        if (dp[mask][last] == LINF) continue;
        for (int nxt = 1; nxt <= n; nxt++) {
            if (mask >> (nxt - 1) & 1) continue;
            int nmask = mask | (1 << (nxt - 1));
            dp[nmask][nxt] = min(dp[nmask][nxt], dp[mask][last] + dist[last][nxt]);
        }
    }
}

```

维度 3: cnt/rest

题面信号:

恰好/至多/至少选 k 个

还剩几次机会

容量、预算、体力、时间

状态句式:

$dp[i][cnt]$ 表示: 处理完前 i 个元素, 已经选 cnt 个时的答案。

$dp[i][rest]$ 表示: 处理到第 i 个元素, 还剩 $rest$ 资源时的答案。

漏状态反例:

从 5 个数中恰好选 2 个, 使和最大。

处理到第 4 个数时, 如果只写 $dp[i]$,
不知道前面已经选了 0 个、1 个还是 2 个。
这些局面后续能不能继续选、答案是否已经合法都不同。

小模板:

```

vector<vector<ll>> dp(n + 1, vector<ll>(K + 1, -LINF));
dp[0][0] = 0;
for (int i = 1; i <= n; i++) {
    for (int cnt = 0; cnt <= K; cnt++) {
        dp[i][cnt] = max(dp[i][cnt], dp[i - 1][cnt]); // 不选
        if (cnt > 0) {
            if (dp[i - 1][cnt - 1] != -LINF) {

```

```

        dp[i][cnt] = max(dp[i][cnt], dp[i - 1][cnt - 1] + a[i]); // 选
    }
}
}
}

```

维度 4: tight

题面信号:

统计 $0..N$ 或 $1..N$

上界很大

数字逐位构造

状态句式:

$\text{dfs}(\text{pos}, \text{tight}, \text{state})$ 表示: 处理到第 pos 位, 当前前缀是否仍贴着上界 tight 。

漏状态反例:

$N = 523$ 。

处理第 2 位时:

前缀是 5, 下一位最多只能取 2;

前缀是 4, 下一位可以取 $0..9$ 。

如果不记 tight , 会把两种上界混在一起。

小模板:

```

int up = tight ? digits[pos] : 9;
for (int d = 0; d <= up; d++) {
    bool ntight = tight && (d == up);
    ans += dfs(pos + 1, ntight, next_state);
}

```

注意: $\text{tight} == \text{true}$ 的状态通常不要缓存; 只缓存 $\text{tight} == \text{false}$ 更稳。

维度 5: leading

题面信号:

数字可以有前导零补齐位数

统计是否出现某个数字

相邻数位限制

不允许数字 0 被前导零误算

状态句式:

$\text{dfs}(\text{pos}, \text{leading}, \text{state})$ 表示: 处理到第 pos 位, 当前是否仍没有放过非零数字。

漏状态反例:

统计 $1..100$ 中 “不含数字 0” 的数。

数字 7 在三位写法里是 007。

前两个 0 是补位, 不应该算作 “出现了数字 0”。

如果不记 leading , 就会错误排除 7。

小模板:

```

bool nleading = leading && (d == 0);
int nlast = nleading ? 10 : d; // 10 表示还没有真实上一位

```

维度 6: color

题面信号:

染色

相邻点颜色不同

某点属于某类

父子状态限制

状态句式:

$dp[u][color]$ 表示: u 的颜色为 $color$ 时, u 子树的方案数/最优值。

漏状态反例:

一棵树相邻点不能同色, 有 3 种颜色。

如果只写 $dp[u]$, 父亲是什么颜色不知道。

父亲是红色时 u 不能选红色; 父亲是蓝色时 u 不能选蓝色。

合法选择不同, 所以 $color$ 必须进入状态或在转移中枚举。

小模板:

```
for (int c = 0; c < C; c++) {
    dp[u][c] = 1;
    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        ll sum = 0;
        for (int vc = 0; vc < C; vc++) {
            if (vc == c) continue;
            sum = (sum + dp[v][vc]) % MOD;
        }
        dp[u][c] = dp[u][c] * sum % MOD;
    }
}
```

维度 7: parity/mod

题面信号:

和为偶数/奇数

能被 K 整除

余数为 r

路径长度模 M

状态句式:

$dp[i][rem]$ 表示: 处理完前 i 个元素, 当前结果模 K 为 rem 时的方案数/是否可行。

漏状态反例:

问选若干数, 使总和能被 3 整除。

处理到同一个 i 时, 当前和模 3 为 0、1、2 的后续完全不同。

只写 $dp[i]$ 无法知道再加一个数后会到哪个余数。

小模板:

```
vector<vector<ll>> dp(n + 1, vector<ll>(K, 0));
dp[0][0] = 1;
for (int i = 1; i <= n; i++) {
    for (int r = 0; r < K; r++) {
        dp[i][r] = (dp[i][r] + dp[i - 1][r]) % MOD;
        int nr = (r + a[i]) % K;
        dp[i][nr] = (dp[i][nr] + dp[i - 1][r]) % MOD;
    }
}
```

维度 8: parent/prev

题面信号:

无向树 DFS

从哪个方向进入会影响可走边

不能立刻走回上一点

换根或路径状态

状态句式:

dfs(u, parent) 表示: 当前在 u, 父亲是 parent, 只处理不走回 parent 的部分。

漏状态反例:

链 1-2-3。

如果写 dfs(2):

当 2 的父亲是 1 时, 2 的子树应该继续处理 3;

当 2 的父亲是 3 时, 2 的子树应该继续处理 1。

只用 u 不能区分这两种方向。

小模板:

```
void dfs(int u, int parent) {
    for (int v : g[u]) {
        if (v == parent) continue;
        dfs(v, u);
    }
}
```

注意: 固定根的树形 DP 中, parent 常作为 DFS 参数, 不一定作为 dp[u][...] 的下标。只有同一个 u 可能从不同方向进入并缓存时, 才需要把 parent/prev 放进 memo key。

维度 9: phase

题面信号:

买入/卖出/冷却

已经使用优惠券/尚未使用

正在匹配模式串第几步

第几阶段任务

状态句式:

dp[i][phase] 表示: 处理完第 i 天/第 i 个元素, 当前处于 phase 阶段时的最优值。

漏状态反例:

股票买卖题。

处理完第 i 天时, 手里有股票和手里没股票的后续选择不同:

有股票可以卖, 没股票可以买。

只写 dp[i] 会把两种状态混掉。

小模板:

```
vector<array<ll, 2>> dp(n + 1);
dp[0][0] = 0; // 0: 没持有
dp[0][1] = -LINF; // 1: 持有
for (int i = 1; i <= n; i++) {
    dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] + price[i]); // 不动或卖出
    dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] - price[i]); // 不动或买入
}
```

map<tuple> 完整状态救场

状态维度一多, 先用 map<tuple> 写对:

map<tuple<int,int,int,int>, ll> memo;

```
ll dfs(int i, int last, int rest, int rem) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return (rem == 0 ? 0 : -LINF);

    auto key = make_tuple(i, last, rest, rem);
    if (memo.count(key)) return memo[key];

    ll ans = -LINF;
    // 按题意枚举选择, 更新 last/rest/rem
```

```
    return memo[key] = ans;
}
```

能过样例后，再看每个维度范围，改成数组。

状态完整性检查卡

1. 同样的 dp 下标，未来合法选择是否完全一样？
2. 同样的 dp 下标，最终答案类型是否完全一样？
3. 转移中是否偷偷用了 path、sum、last、used 这类没进状态的变量？
4. memo key 是否包含所有会影响未来的参数？
5. 多加一维后，状态数是否还能承受？

常见坑：

- 为了省维度，把 last/cnt/rem 写成全局变量，memo 立即失效。
- tight/leading 没处理清楚，数位 DP 容易多算 0 或越过上界。
- mask 和 last 缺一个：访问集合知道了，但不知道当前停在哪里。
- parent 作为参数传了，但 memo 只按 u 缓存。
- 只因为题面出现一个词就盲目增维，没有检查这个信息是否影响未来。

暴力/部分替代：

- 状态不确定：先写不带 memo 的 DFS，用参数显式表达所有历史。
- 参数完整后：直接 map<tuple> memo。
- 状态数太大：只保留题面真正会影响未来的摘要，例如 sum % K，不要保留完整 sum。

升级方向：

- dfs(i,last) -> memo[i][last] -> dp[i][last]。
- dfs(i,cnt,rem) -> 三维数组或滚动数组。
- dfs(pos,tight,leading,last,rem) -> 数位 DP。
- dfs(mask,last) -> 状压 DP。
- dfs(i,j,lastMove/used) -> DP-26 网格升维例题。

最小测试样例：

题面摘要：长度为 3 的 0/1 串，相邻不能相同，问方案数。

正确状态：dfs(pos,last)

答案：2

合法串：010, 101

如果只写 dfs(pos)，说明状态漏了 last。

DP-04: 线性 DP

模型编号：DP-04

模型名称：线性 DP

标签：DP、序列、前 i 个、相邻关系、最优值

一句话用途：处理一条序列上“前 i 个/到第 i 个”的最优值、方案数或可行性。

题面触发词：

- “前 i 个” “到第 i 个位置”
- “不能相邻” “连续/不连续”
- “每天选择/不选择” “第 i 天状态”
- “最大收益” “最小代价” “方案数”

什么时候用：

- 选择按序列从左到右发生。
- 当前决策只依赖前面少量位置或状态。
- 不涉及明显容量时，优先考虑 dp[i] 或 dp[i][state]。

不要什么时候用：

- 每个元素有容量/预算限制，转背包。

- 转移需要查询前面某个值域范围，可能要接 DP + 数据结构优化。
- 最少步数且每步代价相同，优先 BFS。

复杂度：

- 常见： $O(n)$ 、 $O(n * \text{状态数})$ 。
- 朴素枚举前驱： $O(n^2)$ 。
- 优化后： $O(n \log n)$ 或 $O(n)$ 。

数据范围信号：

- $n \leq 5000$ ：可尝试 $O(n^2)$ 。
- $n \leq 2e5$ ：通常要 $O(n)$ 、 $O(n \log n)$ 或优化。

依赖的标准容器：

- `vector<ll> a(n + 1)`
- `vector<ll> dp(n + 1)`
- `vector<vector<ll>> dp(n + 1, vector<ll>(S))`

输入如何整理：

- 序列统一 1-index。
- 如果有“最后一步状态”，把状态编号成 $0..S-1$ 。

接口：

```
ll solve_linear(const vector<ll>& a);
```

输出能力：

- 前缀最优值、到达每个位置的最优值。
- 方案数和可行性也可套同一形状。

下游可接：

- DP-18 DP + 数据结构优化。
- DS 前缀和、树状数组、SegmentTree、单调队列。

可拼接模块：

- PrefixSum：快速计算一段代价。
- 树状数组/SegmentTree：优化 $\max/\min dp[j]$ 查询。
- Compressor：值域作为状态或查询条件时压缩。

状态句式：

`dp[i]` 表示：处理完前 i 个元素时，答案的最大值/最小值/方案数。

如果要求必须以 i 结尾：

`dp[i]` 表示：以第 i 个元素作为最后一步时的最优值。

初始化：

`dp[0] = 0`：还没有处理任何元素，收益/代价为 0。

最大值题其他状态设 `-LINF`；最小值题其他状态设 `LINF`。

如果题目允许什么都不选，最大值 DP 常常可以全 0 初始化；如果题目要求必须选至少一个、必须到达某个状态、或全是负数也要选，就不能全 0，必须用 `-LINF` 表示不可达。

转移模板：

```
dp[i] = max(dp[i - 1], (i >= 2 ? dp[i - 2] : 0) + a[i]); // 不选相邻的典型写法
```

更通用：

```
for (int i = 1; i <= n; i++) {
    dp[i] = best_from_previous_states(i);
}
```

答案位置：

- 前 n 个整体最优：`dp[n]`。
- 必须以某点结尾： $\max(dp[i])$ 或指定 `dp[t]`。
- 方案数：通常 `dp[n]`。

循环顺序：

- i 从小到大。
- 如果 $dp[i]$ 依赖 $i-k$, 先处理小下标。

暴力 DFS 版本:

```
11 dfs(int i) {
    if (i > n) return 0;
    11 ans = dfs(i + 1);           // 不选 i
    ans = max(ans, a[i] + dfs(i + 2)); // 选 i, 跳过相邻
    return ans;
}
```

记忆化版本:

```
vector<ll> memo;
vector<int> vis;

11 dfs(int i) {
    if (i > n) return 0;
    if (vis[i]) return memo[i];
    vis[i] = 1;
    11 ans = dfs(i + 1);
    ans = max(ans, a[i] + dfs(i + 2));
    return memo[i] = ans;
}
```

表推版本:

```
vector<ll> dp(n + 2, 0);
for (int i = 1; i <= n; i++) {
    dp[i] = max(dp[i - 1], (i >= 2 ? dp[i - 2] : 0) + a[i]);
}
cout << dp[n] << '\n';
```

常见变体:

- $dp[i][0/1]$: 第 i 个不选/选。
- $dp[i] = \min(dp[j] + \text{cost}(j + 1, i))$: 分段类线性 DP。
- $dp[i]$ 方案数: 加法转移并取模。

常见坑:

- $dp[i]$ 是“前 i 个最优”还是“以 i 结尾”混用。
- 最大值题有负数时, 初值不能随便设 0。
- $i-2$ 越界。
- 答案可能是 $\max(dp[i])$, 不是 $dp[n]$ 。

暴力/部分分替代:

- $n \leq 30$: 直接 DFS。
- $n \leq 1e5$ 且依赖固定前几项: 记忆化/表推都能过。
- 复杂转移先写 $O(n^2)$ 表推。

升级方向:

- $O(n^2)$ 前驱枚举 -> 树状数组/SegmentTree 优化。
- 分段代价 -> PrefixSum 预处理。
- 只依赖前几项 -> 滚动变量省空间。

最小测试样例:

```
5
2 7 9 3 1
输出: 12
说明: 选 2 + 9 + 1。
```

DP-05: 选/不选 DP

模型编号: DP-05

模型名称：选/不选 DP

标签：DP、子集、决策树、部分分、记忆化

一句话用途：每个元素只有“选/不选”两条路时，先写 DFS，再按状态维度升级成 DP。

题面触发词：

- “每个元素可以选或不选”
- “从若干物品中选择一些”
- “最多/恰好选 k 个”
- “满足限制的子集数量”
- “选择若干项目使收益最大/代价最小”

什么时候用：

- 决策是按元素逐个推进。
- 除了位置 i ，还要记住数量、容量、余数、上一状态等。
- 还没确定是不是背包、状压或线性 DP 时，它是通用入口。

不要什么时候用：

- $n \leq 20$ 且需要枚举集合关系，状压可能更直接。
- 物品有容量维度且每个最多一次，优先 0/1 背包。
- 选择顺序本身重要，不只是选集合。

复杂度：

- 暴力： $O(2^n)$ 。
- 记忆化/DP： $O(\text{状态数} * 2)$ 。

数据范围信号：

- $n \leq 25$ ：暴力 DFS 可能拿分。
- $n \leq 1000$ 且有 $k/W/\text{mod}$ 小维度：二维 DP。
- $n \leq 2e5$ ：通常要压缩状态或转其他模型。

依赖的标准容器：

- `vector<ll> a(n + 1)`
- `vector<vector<ll>> memo/dp`
- `map<tuple<...>, ll>`：状态复杂时使用。

输入如何整理：

- 统一每个元素的属性：`cost[i]`、`val[i]`、`type[i]`。
- 把限制整理成状态参数：`cnt`、`rest`、`mod`、`last`。

接口：

```
ll dfs(int i, int state);
```

输出能力：

- 最大值、最小值、方案数、可行性。

下游可接：

- DP-06 0/1 背包。
- DP-16 状压 DP。
- BRUTE 记忆化搜索。

可拼接模块：

- `map<tuple> memo`。
- Compressor：状态值域大时先压缩。
- PrefixSum：计算已选前缀贡献。

状态句式：

`dfs(i, rest)` 表示：从第 i 个元素开始，当前剩余资源为 `rest` 时，后续能得到的最优值。

表推句式：

`dp[i][j]` 表示：处理完前 i 个元素，附加状态为 j 时的答案。

初始化：

dp[0][初始状态] = 0/1/true: 还没处理元素时, 只有初始状态合法。
其他状态为不可达。

转移模板:

```
// 不选
dp[i][j] = merge(dp[i][j], dp[i - 1][j]);
// 选
dp[i][next(j, i)] = merge(dp[i][next(j, i)], dp[i - 1][j] + gain(i));
```

答案位置:

- 恰好达到状态: dp[n][target]。
- 任意合法状态: max/min/sum dp[n][j]。
- DFS 版本: dfs(1, initial_state)。

循环顺序:

- 二维表: i 从 1 到 n, j 枚举所有合法状态。
- 一维滚动: 看是否与背包一致; 0/1 场景通常倒序。

暴力 DFS 版本:

下面这份是“容量至多为 W”的最大值版本; 如果题目要求恰好用完容量, 终止条件要改成 rest == 0 ? 0 : -LINF。

```
ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    ll ans = dfs(i + 1, rest); // 不选
    ans = max(ans, val[i] + dfs(i + 1, rest - cost[i])); // 选
    return ans;
}
```

记忆化版本:

```
vector<vector<ll>> memo;
vector<vector<int>> vis;

ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    if (vis[i][rest]) return memo[i][rest];
    vis[i][rest] = 1;
    ll ans = dfs(i + 1, rest);
    ans = max(ans, val[i] + dfs(i + 1, rest - cost[i]));
    return memo[i][rest] = ans;
}
```

表推版本:

```
vector<vector<ll>> dp(n + 1, vector<ll>(W + 1, -LINF));
dp[0][W] = 0;
for (int i = 1; i <= n; i++) {
    for (int rest = 0; rest <= W; rest++) {
        dp[i][rest] = max(dp[i][rest], dp[i - 1][rest]);
        if (rest + cost[i] <= W && dp[i - 1][rest + cost[i]] != -LINF) {
            dp[i][rest] = max(dp[i][rest], dp[i - 1][rest + cost[i]] + val[i]);
        }
    }
}
cout << *max_element(dp[n].begin(), dp[n].end()) << '\n';
```

常见变体:

- 恰好选 k 个: 状态加 cnt。
- 余数限制: 状态加 mod。
- 上一个选择影响当前: 状态加 last。

常见坑:

- DFS 里用了全局 sum/path 却没有放进状态, memo 会错。

- 终止条件没有检查 “恰好” 还是 “至多”。
- 一维压缩时把同一元素重复选了。
- rest 为负后还访问数组。

暴力/部分分替代：

- $n \leq 25$: DFS。
- $n \leq 40$: 折半枚举可拿更多分。
- 状态范围不清: `map<tuple<int,int,int>, ll>`。

升级方向：

- 选/不选 + 容量 $\rightarrow$ 0/1 背包。
- 选/不选 + 集合关系 $\rightarrow$ 状压 DP。
- 选/不选 + 前驱范围最优 $\rightarrow$ DP + 数据结构。

最小测试样例：

```
3 5
2 3
3 4
4 5
输出：7
```

DP-06: 0/1 背包

模型编号: DP-06

模型名称: 0/1 背包

标签: DP、背包、容量、每个物品最多一次

一句话用途: 每个物品最多选一次，在容量/预算限制下求最大价值、方案数或可行性。

题面触发词：

- “每个物品只能选择一次”
- “重量/体积/花费不超过 W ”
- “价值最大”
- “能否凑出”
- “选一些物品”

什么时候用：

- 物品是独立的。
- 每个物品只有选一次或不选。
- 主要限制是一维容量/预算。

不要什么时候用：

- 物品可以重复选，转完全背包。
- 物品分组且每组最多一个，转分组背包。
- 容量太大但价值总和小，可能改用 “价值维度” 背包。

复杂度：

- $O(nW)$ 时间。
- 一维优化 $O(W)$ 空间。

数据范围信号：

- $n * W \leq 1e7$ 左右较稳。
- $W > 1e7$ 通常不能直接按容量开表。

依赖的标准容器：

- `vector<int> w(n + 1), v(n + 1)`
- `vector<ll> dp(W + 1)`

输入如何整理：

- `w[i]`: 消耗容量。

- $v[i]$: 收益。
- 容量默认 $0..W$ 。

接口:

```
ll zero_one_knapsack(int n, int W, vector<int>& w, vector<ll>& v);
```

输出能力:

- 最大价值。
- 可行性: 把 ll dp 改成 bool dp。
- 恰好容量的 0/1 方案数: $dp[0]=1$, 容量倒序, $dp[j]+=dp[j-w[i]]$; 不要只机械把 max 改加法。

下游可接:

- DP-05 选/不选 DP。
- DP-08 分组背包。
- DP-18 滚动数组/优化。
- DP-24 背包常见变体总表: 多重、二维、价值维度、计数、路径恢复。

可拼接模块:

- PrefixSum: 多重背包或分组预处理。
- Compressor: 容量值稀疏时辅助压缩, 但普通背包不能随意压缩加法容量。

状态句式:

$dp[j]$ 表示: 当前已处理若干物品, 总容量不超过/恰好为 j 时的最大价值。

若写二维:

$dp[i][j]$ 表示: 处理完前 i 个物品, 容量为 j 时的最大价值。

初始化:

最大值且允许不装满: $dp[j] = 0$ 。

最大值且要求恰好装满: $dp[0] = 0$, 其他 $dp[j] = -LINF$ 。

可行性: $dp[0] = true$ 。

方案数: $dp[0] = 1$ 。

转移模板:

```
dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

答案位置:

- 不超过容量最大值: $\max(dp[0..W])$, 若初始化全 0 通常 $dp[W]$ 也可代表 “不超过 W ”。
- 恰好装满: $dp[W]$ 。
- 可行性: $dp[target]$ 。

循环顺序:

物品 i 从 1 到 n 。

容量 j 必须从 W 倒序到 $w[i]$, 防止同一物品被重复使用。

暴力 DFS 版本:

下面这份是 “不超过容量” 的最大价值版本; 若要求恰好装满, 终止条件要改成 $i == n + 1$ 时返回 $rest == 0 ? 0 : -LINF$ 。

```
ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    return max(dfs(i + 1, rest),
               v[i] + dfs(i + 1, rest - w[i]));
}
```

记忆化版本:

```
vector<vector<ll>> memo;
vector<vector<int>> vis;
```

```
ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    if (vis[i][rest]) return memo[i][rest];
```

```

    vis[i][rest] = 1;
    ll ans = max(dfs(i + 1, rest),
                v[i] + dfs(i + 1, rest - w[i]));
    return memo[i][rest] = ans;
}

```

表推版本:

下面这段是“不超过容量 W 的最大价值”的常用版本: dp 全 0, 表示什么都不选也合法。此时 $dp[W]$ 代表容量不超过 W 时的最大值。

```

vector<ll> dp(W + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = W; j >= w[i]; j--) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
cout << dp[W] << '\n';

```

如果题目要求“恰好装满 W ”, 必须换初始化:

```

vector<ll> dp(W + 1, -LINF);
dp[0] = 0;
for (int i = 1; i <= n; i++) {
    for (int j = W; j >= w[i]; j--) {
        if (dp[j - w[i]] != -LINF) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
}
cout << (dp[W] == -LINF ? -1 : dp[W]) << '\n';

```

常见变体:

- 恰好装满最大值: 其他容量初始化 $-LINF$ 。
- 最小花费达到价值: 交换容量和价值维度。
- 方案数背包: 倒序加 $dp[j] += dp[j - w[i]]$ 。

常见坑:

- 容量正序导致一个物品被选多次。
- “恰好装满”和“不超过容量”初始化混淆。
- $dp[j - w[i]]$ 是可达时仍参与转移。
- 容量和价值用 int 溢出。

暴力/部分分替代:

- $n \leq 25$: DFS 枚举。
- $n * W$ 稍大: 记忆化可能因状态稀疏拿分。
- W 太大但 $\sum(v)$ 小: 改价值维度。

升级方向:

- 二维 $\rightarrow$ 一维倒序。
- 价值维度背包。
- 多维容量时谨慎扩维, 复杂度会乘上去。
- 有有限个数、多维费用、至少装满或路径恢复时, 优先翻 DP-24。

最小测试样例:

```

3 5
2 3
3 4
4 5
输出: 7

```

DP-07: 完全背包

模型编号: DP-07

模型名称：完全背包

标签：DP、背包、容量、无限次选择

一句话用途：每种物品可以重复选任意次，在容量限制下求最优值、方案数或可行性。

题面触发词：

- “每种物品有无限个”
- “可以重复购买/使用”
- “硬币凑金额”
- “不限数量”

什么时候用：

- 每种物品重复选择不受次数限制。
- 重复选同一种物品仍消耗同样容量并获得同样收益。

不要什么时候用：

- 每个物品最多一次，转 0/1 背包。
- 每种物品有有限个，需多重背包或拆分。
- 题目要求排列数/组合数时，循环顺序要额外确认。

复杂度：

- $O(nW)$ 时间。
- $O(W)$ 空间。

数据范围信号：

- $n * W \leq 1e7$ 左右较稳。
- “硬币数量不限，金额 W 不大” 是典型信号。

依赖的标准容器：

- `vector<int> w(n + 1)`
- `vector<ll> v(n + 1)`
- `vector<ll> dp(W + 1)`

输入如何整理：

- $w[i]$ ：每次选择的容量消耗。
- $v[i]$ ：每次选择的价值。

接口：

```
ll complete_knapsack(int n, int W, vector<int>& w, vector<ll>& v);
```

输出能力：

- 最大价值。
- 凑金额方案数。
- 能否凑出。

下游可接：

- 计数 DP。
- 最短/最少硬币数 DP。
- DP-24 背包常见变体总表：组合数/排列数、多重背包、至少装满。

可拼接模块：

- MOD 计数模板。
- GCD：硬币问题可先判断可达性必要条件。

状态句式：

$dp[j]$ 表示：当前已处理若干种物品，容量为 j 时的最大价值/方案数/最少数量。

初始化：

最大值不要求装满： $dp[j] = 0$ 。

恰好装满最大值： $dp[0] = 0$ ，其他为 $-LINF$ 。

最少数量： $dp[0] = 0$ ，其他为 $LINF$ 。

方案数： $dp[0] = 1$ 。

转移模板:

```
dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

前置条件: 普通完全背包默认 $w[i] > 0$ 。如果 $w[i] == 0$, 必须按题意单独处理; $v[i] > 0$ 的最优值可能无限大, 方案数也可能不是有限数。

答案位置:

- 最大价值: $dp[W]$ 。
- 恰好金额方案数: $dp[W]$ 。
- 任意不超过容量: 通常 $dp[W]$ 已包含 “不装满” 的最优。

循环顺序:

物品 i 从 1 到 n 。

容量 j 从 $w[i]$ 正序到 W , 允许当前物品被重复使用。

暴力 DFS 版本:

下面这份是 “不超过容量” 的最大价值版本; 若要求恰好装满, 终止条件要改成 $rest == 0 ? 0 : -LINF$ 。

```
ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    ll ans = dfs(i + 1, rest); // 不再选第 i 种
    if (rest >= w[i]) ans = max(ans, v[i] + dfs(i, rest - w[i])); // 继续选 i
    return ans;
}
```

记忆化版本:

```
vector<vector<ll>> memo;
vector<vector<int>> vis;

ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    if (vis[i][rest]) return memo[i][rest];
    vis[i][rest] = 1;
    ll ans = dfs(i + 1, rest);
    if (rest >= w[i]) ans = max(ans, v[i] + dfs(i, rest - w[i]));
    return memo[i][rest] = ans;
}
```

表推版本:

下面这段是 “不超过容量 W 的最大价值” 版本, dp 全 0, 表示不选也合法。

```
vector<ll> dp(W + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = w[i]; j <= W; j++) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
cout << dp[W] << '\n';
```

如果题目要求 “恰好装满 W ”, 必须换初始化:

```
vector<ll> dp(W + 1, -LINF);
dp[0] = 0;
for (int i = 1; i <= n; i++) {
    for (int j = w[i]; j <= W; j++) {
        if (dp[j - w[i]] != -LINF) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
}
cout << (dp[W] == -LINF ? -1 : dp[W]) << '\n';
```

方案数表推:


```
vector<ll> dp(W + 1, 0);
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    if (w[i] <= 0) throw runtime_error("complete knapsack count needs positive weight");
    for (int j = w[i]; j <= W; j++) {
        dp[j] = (dp[j] + dp[j - w[i]]) % MOD;
    }
}
```

常见变体:

- 最少硬币数: $dp[j] = \min(dp[j], dp[j - w[i]] + 1)$ 。
- 排列数: 通常容量外层、物品内层。
- 组合数: 物品外层、容量内层。
- 完全背包计数默认每种物品重量为正; 零重量物品要先特判, 不要直接套 $dp[j] += dp[j]$ 。

常见坑:

- 把容量写成倒序, 变成 0/1 背包。
- 计数题没区分组合数和排列数。
- 最少数量初始化成 0, 导致所有状态都像可达。
- $w[i]=0$ 且 $v[i]>0$ 会无限收益, 需特殊判断。

暴力/部分替代:

- DFS 枚举每种物品取几个。
- 小 W 时记忆化很稳。
- 不会一维时先写二维 $dp[i][j]$ 。

升级方向:

- 二维 -> 一维正序。
- 有数量上限 -> 多重背包/二进制拆分。
- 转移带滑动窗口限制 -> 单调队列优化。
- 计数题先到 DP-24 区分组合数和排列数。

最小测试样例:

```
2 5
2 3
3 4
```

输出: 7

说明: 2+3 容量, 价值 7。

DP-08: 分组背包

模型编号: DP-08

模型名称: 分组背包

标签: DP、背包、分组、每组最多选一个

一句话用途: 物品按组划分, 每组最多选择一个, 在容量限制下求最优值、方案数或可行性。

题面触发词:

- “每组最多选一个”
- “每类商品选一个/不选”
- “课程分组、套餐选择”
- “从每个集合中选择一个方案”

什么时候用:

- 物品天然分成若干组。
- 同一组内物品互斥。
- 组与组之间独立, 只有容量共享。

不要什么时候用:

- 所有物品互不互斥, 转 0/1 背包。

- 每组必须选一个时，初始化和答案要按“必须”改。
- 组内可以选多个，不能套“最多一个”。

复杂度：

- $O(\text{总物品数} * W)$ 。
- 空间 $O(W)$ 。

数据范围信号：

- 组数 * W * 平均组大小 可承受。
- 总物品数和容量都不大时直接套。

依赖的标准容器：

- `vector<vector<pair<int,ll>>> groups(G + 1)`
- `vector<ll> dp(W + 1), ndp(W + 1)`

输入如何整理：

- 把每件物品放入对应组：`groups[g].push_back({w, v})`。
- 组编号统一 $1..G$ 。

接口：

```
ll group_knapsack(int G, int W, vector<vector<pair<int,ll>>>& groups);
```

输出能力：

- 最大价值。
- 每组选择方案的可行性或方案数。

下游可接：

- 树形依赖背包。
- 课程/套餐类选择。
- DP-24 背包常见变体总表：每组必须选、分组计数、路径恢复。

可拼接模块：

- Compressor：组编号散乱时压缩。
- PrefixSum：组内候选由区间预处理生成时使用。

状态句式：

`dp[j]` 表示：已经处理完若干组，容量为 j 时的最大价值。

初始化：

最多选一个且可以不选任何组：`dp[j] = 0`。

必须精确容量：`dp[0] = 0`，其他为 `-LINF`。

每组必须选一个：每处理一组时 `ndp` 初始为 `-LINF`，不能直接继承“不选”。

转移模板：

```
ndp = dp; // 每组最多选一个：允许本组不选
for (auto [w, v] : group) {
    for (int j = w; j <= W; j++) {
        ndp[j] = max(ndp[j], dp[j - w] + v);
    }
}
dp.swap(ndp);
```

答案位置：

- 不超过容量：`dp[W]`。
- 恰好容量：`dp[W]`，但必须用不可达初始化。
- 任意容量：`max(dp[0..W])`。

循环顺序：

外层枚举组。

组内枚举物品。

容量用 `ndp` 从旧 `dp` 转移，防止同组多个物品被同时选。

暴力 DFS 版本：

```

11 dfs(int g, int rest) {
    if (rest < 0) return -LINF;
    if (g == G + 1) return 0;
    11 ans = dfs(g + 1, rest); // 本组不选
    for (auto [w, v] : groups[g]) {
        ans = max(ans, v + dfs(g + 1, rest - w));
    }
    return ans;
}

```

记忆化版本:

```

vector<vector<ll>> memo;
vector<vector<int>> vis;

```

```

11 dfs(int g, int rest) {
    if (rest < 0) return -LINF;
    if (g == G + 1) return 0;
    if (vis[g][rest]) return memo[g][rest];
    vis[g][rest] = 1;
    11 ans = dfs(g + 1, rest);
    for (auto [w, v] : groups[g]) {
        ans = max(ans, v + dfs(g + 1, rest - w));
    }
    return memo[g][rest] = ans;
}

```

表推版本:

下面这段是“每组最多选一个，容量不超过 W ”的最大价值版本；若要求恰好容量或每组必须选，要按初始化说明改 $-LINF$ 。

```

vector<ll> dp(W + 1, 0);
for (int g = 1; g <= G; g++) {
    vector<ll> ndp = dp;
    for (auto [w, v] : groups[g]) {
        for (int j = w; j <= W; j++) {
            ndp[j] = max(ndp[j], dp[j - w] + v);
        }
    }
    dp.swap(ndp);
}
cout << dp[W] << '\n';

```

常见变体:

- 每组必须选一个: ndp 初始化为 $-LINF$ ，不复制 dp 。
- 每组恰好选一个且容量不超过: 答案 $\max(dp[0..W])$ 。
- 组内候选是“方案”而非单物品，也可放进 $groups$ 。

常见坑:

- 直接在同一个 dp 上更新，导致同组多个物品被选。
- 忘记区分“最多选一个”和“必须选一个”。
- 组为空时，必须选一个会无解。
- 初始化全 0 导致恰好容量题错。

暴力/部分分替代:

- 组数小: DFS 枚举每组选择。
- 容量小: 记忆化 $dfs(g, rest)$ 。
- 组内物品多但有效容量少: 过滤 $w > W$ 的物品。

升级方向:

- 分组背包 $\rightarrow$ 树上分组/依赖背包。
- 组内候选通过其他 DP 先生成。
- 空间从二维压成一维 $dp + ndp$ 。
- 要恢复每组选了哪个方案时，优先写二维 $choice[g][j]$ ，见 DP-24。

最小测试样例:

2 5

组 1: (2,3) (3,4)

组 2: (2,4) (4,7)

输出: 8

说明: 选组 1 的 (3,4) 和组 2 的 (2,4)。

DP-09: LCS 最长公共子序列

模型编号: DP-09

模型名称: LCS

标签: DP、两个序列、字符串、公共子序列

一句话用途: 求两个序列的最长公共子序列长度, 或作为两个序列匹配类 DP 的入口。

题面触发词:

- “最长公共子序列”
- “两个字符串/两个序列”
- “保持相对顺序”
- “可以删除一些字符”
- “相似度/匹配长度”

什么时候用:

- 两个序列都只能按原相对顺序匹配。
- 不要求连续; 要求连续时是最长公共子串, 不是 LCS。

不要什么时候用:

- 允许插入、删除、替换并问最少操作, 转编辑距离。
- 只问一个序列的递增子序列, 转 LIS。
- 要求公共子串连续, 需另写 $dp[i][j]$ 表示以 i, j 结尾的连续长度。

复杂度:

- 时间 $O(nm)$ 。
- 空间 $O(nm)$, 可滚动成 $O(m)$ 。

数据范围信号:

- $n, m \leq 3000/5000$: $O(nm)$ 可能可接受, 视内存。
- $n, m \geq 1e5$: 普通 LCS 不可直接做。

依赖的标准容器:

- `string s, t`
- `vector<vector<int>>> dp`

输入如何整理:

- 字符串保持 0-index。
- DP 的 i/j 表示前缀长度, 访问字符用 $s[i-1]$ 、 $t[j-1]$ 。

接口:

```
int lcs(const string& s, const string& t);
```

输出能力:

- LCS 长度。
- 可扩展为输出一条 LCS 路径。

下游可接:

- 编辑距离。
- 路径恢复。
- DP-23 LIS/LCS 常见变体速查: 恢复一条 LCS、最长公共子串、只插删关系。

可拼接模块:

- 滚动数组。

- 字符串基础模块。

状态句式:

$dp[i][j]$ 表示: s 的前 i 个字符和 t 的前 j 个字符的 LCS 长度。

初始化:

$dp[0][j] = 0$: 空串和任何前缀的 LCS 长度为 0。

$dp[i][0] = 0$: 任何前缀和空串的 LCS 长度为 0。

转移模板:

```
if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
```

答案位置:

- $dp[n][m]$ 。

循环顺序:

- i 从 1 到 n 。
- j 从 1 到 m 。
- 因为依赖左、上、左上, 二维表自然从小到大。

暴力 DFS 版本:

```
int dfs(int i, int j) {
    if (i == n || j == m) return 0;
    int ans = max(dfs(i + 1, j), dfs(i, j + 1));
    if (s[i] == t[j]) ans = max(ans, 1 + dfs(i + 1, j + 1));
    return ans;
}
```

记忆化版本:

`vector<vector<int>> memo, vis;`

```
int dfs(int i, int j) {
    if (i == n || j == m) return 0;
    if (vis[i][j]) return memo[i][j];
    vis[i][j] = 1;
    int ans = max(dfs(i + 1, j), dfs(i, j + 1));
    if (s[i] == t[j]) ans = max(ans, 1 + dfs(i + 1, j + 1));
    return memo[i][j] = ans;
}
```

表推版本:

```
int n = s.size(), m = t.size();
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
        else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    }
}
cout << dp[n][m] << '\n';
```

滚动数组版本:

```
vector<int> dp(m + 1, 0), ndp(m + 1, 0);
for (int i = 1; i <= n; i++) {
    fill(ndp.begin(), ndp.end(), 0);
    for (int j = 1; j <= m; j++) {
        if (s[i - 1] == t[j - 1]) ndp[j] = dp[j - 1] + 1;
        else ndp[j] = max(dp[j], ndp[j - 1]);
    }
    dp.swap(ndp);
}
cout << dp[m] << '\n';
```

常见变体：

- 输出一条 LCS：从 $dp[n][m]$ 反向走。
- 最长公共子串：相等时 $dp[i][j]=dp[i-1][j-1]+1$ ，不等时 0。
- 三个序列 LCS：三维，复杂度很高。

常见坑：

- 把子序列误写成子串。
- 字符访问 $s[i]$ 与 $dp[i]$ 前缀长度混淆。
- 滚动数组更新时覆盖左上角。
- 内存 $n*m$ 太大。

暴力/部分分替代：

- 短字符串直接 DFS。
- $n*m$ 较大但样例小：记忆化版先交。
- 一个串很短时可考虑状压优化，但初学者低优先级。

升级方向：

- 二维表 -> 滚动数组。
- 路径恢复。
- 若字符集小且数据大，再查高级优化。
- 公共子串、只允许插删、输出具体序列时，优先翻 DP-23。

最小测试样例：

abcde

ace

输出：3

DP-10：编辑距离

模型编号：DP-10

模型名称：编辑距离

标签：DP、两个字符串、插入、删除、替换、最小操作数

一句话用途：求把一个字符串变成另一个字符串的最少插入、删除、替换次数。

题面触发词：

- “插入、删除、替换”
- “一个字符串变成另一个字符串”
- “最少操作次数”
- “编辑距离”
- “纠错/相似度”

什么时候用：

- 操作对象是两个字符串/序列的前缀。
- 每次操作只影响当前末尾或当前位置。

不要什么时候用：

- 只允许删除并求最长匹配，可能是 LCS。
- 操作有复杂全局限制，需要把限制加入状态。
- 操作代价不是常数时，仍可用，但转移要改权重。

复杂度：

- 时间 $O(nm)$ 。
- 空间 $O(nm)$ ，可滚动成 $O(m)$ 。

数据范围信号：

- $n, m \leq 3000/5000$ ：可尝试。
- n, m 很大：普通编辑距离不可直接做。

依赖的标准容器：

- string s, t
- vector<vector<int>> dp

输入如何整理:

- 字符串 0-index.
- dp[i][j] 的 i/j 表示前缀长度。

接口:

```
int edit_distance(const string& s, const string& t);
```

输出能力:

- 最少操作数。
- 可扩展为恢复操作路径。

下游可接:

- 字符串相似度。
- 路径恢复。

可拼接模块:

- LCS: 只允许删除时可用 LCS 转换。
- 滚动数组。
- DP-22: 编辑距离完整建模例题, 适合先看推导再抄模板。

状态句式:

dp[i][j] 表示: 把 s 的前 i 个字符变成 t 的前 j 个字符所需的最少操作数。

初始化:

dp[i][0] = i: 把前 i 个字符变成空串, 需要删除 i 次。

dp[0][j] = j: 把空串变成前 j 个字符, 需要插入 j 次。

转移模板:

```
if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1];
else {
    dp[i][j] = 1 + min({
        dp[i - 1][j],      // 删除 s[i-1]
        dp[i][j - 1],      // 插入 t[j-1]
        dp[i - 1][j - 1]   // 替换
    });
}
```

答案位置:

- dp[n][m]。

循环顺序:

- i 从 0 到 n 初始化第一列。
- j 从 0 到 m 初始化第一行。
- 主循环 i=1..n, j=1..m。

暴力 DFS 版本:

```
int dfs(int i, int j) {
    if (i == n) return m - j; // 只能插入剩余字符
    if (j == m) return n - i; // 只能删除剩余字符
    if (s[i] == t[j]) return dfs(i + 1, j + 1);
    int del = dfs(i + 1, j);
    int ins = dfs(i, j + 1);
    int rep = dfs(i + 1, j + 1);
    return 1 + min({del, ins, rep});
}
```

记忆化版本:

```
vector<vector<int>> memo, vis;
```

```
int dfs(int i, int j) {
```

```

    if (i == n) return m - j;
    if (j == m) return n - i;
    if (vis[i][j]) return memo[i][j];
    vis[i][j] = 1;
    if (s[i] == t[j]) return memo[i][j] = dfs(i + 1, j + 1);
    int ans = 1 + min({dfs(i + 1, j), dfs(i, j + 1), dfs(i + 1, j + 1)});
    return memo[i][j] = ans;
}

```

表推版本:

```

int n = s.size(), m = t.size();
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int j = 0; j <= m; j++) dp[0][j] = j;

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1];
        else dp[i][j] = 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]});
    }
}
cout << dp[n][m] << '\n';

```

常见变体:

- 插入/删除/替换代价不同: 把 +1 改成对应代价。
- 只允许插入删除: 转移去掉替换。
- 输出操作序列: 从 $dp[n][m]$ 反推。

常见坑:

- 第一行第一列没初始化。
- 相等字符仍加了操作代价。
- 插入和删除方向混乱, 但最小值仍通常能写对; 恢复路径时要小心。
- 字符串下标差 1。

暴力/部分分替代:

- 字符串很短时 DFS。
- 中等长度先记忆化。
- 只问删除次数时, 可能用 LCS 简化。

升级方向:

- 二维表 -> 滚动数组。
- 带阈值的编辑距离可只算带状区域。
- 操作路径恢复。

最小测试样例:

```

horse
ros
输出: 3

```

DP-11: LIS 最长递增子序列

模型编号: DP-11

模型名称: LIS

标签: DP、序列、递增、二分优化

一句话用途: 求一个序列中保持原相对顺序的最长递增/不下降子序列长度。

题面触发词:

- “最长递增子序列”
- “最长上升子序列”

- “不下降子序列”
- “保持原顺序”
- “最多能选多少个”

什么时候用：

- 只处理一个序列。
- 要求选出的元素下标递增，值也满足递增/不下降。

不要什么时候用：

- 要求连续，改成最长递增连续段。
- 有两个序列公共匹配，转 LCS。
- 有额外容量/代价限制，需要扩展状态。

复杂度：

- 朴素 DP: $O(n^2)$ 。
- 二分优化: $O(n \log n)$ 。

数据范围信号：

- $n \leq 5000$: 朴素 $O(n^2)$ 可写。
- $n \leq 2e5$: 用二分优化。

依赖的标准容器：

- `vector<ll> a(n + 1)`
- `vector<int> dp(n + 1)`
- `vector<ll> d`

输入如何整理：

- 序列读成 `a[1..n]`；二分辅助数组 `d` 是 STL 工作容器，可按 `push_back` 自然使用。

接口：

```
int lis_length(const vector<ll>& a);
```

输出能力：

- LIS 长度。
- 可扩展恢复一个序列。

下游可接：

- DP + 树状数组优化。
- 坐标压缩。
- DP-23 LIS/LCS 常见变体速查：路径恢复、二维偏序、公共子串/LCS 区分。

可拼接模块：

- Compressor: 值域大但要用树状数组。
- 树状数组/SegmentTree: 求以值为结尾的最大长度。

状态句式：

朴素 DP：

`dp[i]` 表示：必须以第 `i` 个元素结尾的最长递增子序列长度。

二分优化：

`d[len]` 表示：长度为 `len` 的递增子序列中，最小可能结尾值。

初始化：

朴素: `dp[i] = 1`，每个元素自己就是长度 1 的子序列。

二分: `d` 为空，逐个插入/替换。

转移模板：

朴素：

```
if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
```

二分：

```
auto it = lower_bound(d.begin(), d.end(), a[i]); // 严格递增
```

答案位置:

- 朴素: `max(dp[1..n])`。
- 二分: `d.size()`。

循环顺序:

- 朴素: i 从 1 到 n , j 从 1 到 $i-1$ 。
- 二分: 按原序列顺序扫描。

暴力 DFS 版本:

```
int dfs(int i, int last) {
    if (i == n + 1) return 0;
    int ans = dfs(i + 1, last);
    if (last == 0 || a[last] < a[i]) {
        ans = max(ans, 1 + dfs(i + 1, i));
    }
    return ans;
}
```

记忆化版本:

```
vector<vector<int>> memo, vis;
```

```
int dfs(int i, int last) {
    if (i == n + 1) return 0;
    if (vis[i][last]) return memo[i][last];
    vis[i][last] = 1;
    int ans = dfs(i + 1, last);
    if (last == 0 || a[last] < a[i]) ans = max(ans, 1 + dfs(i + 1, i));
    return memo[i][last] = ans;
}
```

表推版本:

```
vector<int> dp(n + 1, 1);
int ans = 0;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j < i; j++) {
        if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
    }
    ans = max(ans, dp[i]);
}
cout << ans << '\n';
```

二分优化版本:

```
vector<ll> d;
for (int i = 1; i <= n; i++) {
    auto it = lower_bound(d.begin(), d.end(), a[i]); // 严格递增
    if (it == d.end()) d.push_back(a[i]);
    else *it = a[i];
}
cout << (int)d.size() << '\n';
```

常见变体:

- 最长不下降子序列: 二分用 `upper_bound`。
- 求方案数: 通常用 DP + 树状数组。
- 二维偏序: 排序后一维 LIS。

常见坑:

- 严格递增和不下降混淆。
- 朴素 DP 的答案是 `max(dp[i])`, 不是 `dp[n]`。
- 二分数组 d 不一定是一条真实子序列, 只能用于长度。
- 有重复值时排序和二分规则容易错。

暴力/部分替代:

- $n \leq 25$: 选/不选 DFS。
- $n \leq 5000$: 朴素 DP。
- n 大: 二分优化。

升级方向:

- 朴素 $O(n^2)$ $\rightarrow$ 二分 $O(n \log n)$ 。
- 需要计数/带权: 坐标压缩 + 树状数组/SegmentTree。
- 需要恢复序列: 记录前驱。
- 二维偏序、重复值 tie-break、输出一条 LIS 时, 优先翻 DP-23。

最小测试样例:

6
10 9 2 5 3 7
输出: 3
说明: 2 5 7 或 2 3 7。

DP-12: 网格 DP

模型编号: DP-12

模型名称: 网格 DP

标签: DP、网格、路径数、最小路径和

一句话用途: 在二维网格中按固定方向移动, 求路径数、最大收益或最小代价。

题面触发词:

- “从左上角到右下角”
- “只能向右/向下”
- “网格路径数量”
- “障碍物”
- “最小路径和/最大金币”

什么时候用:

- 移动方向保证无环, 例如只向右、向下、右下。
- 到达一个格子的答案只依赖前面方向的格子。

不要什么时候用:

- 可以上下左右任意走且求最短步数, 优先 BFS。
- 网格有权最短路且可回头, 可能是 Dijkstra。
- 状态还包含钥匙/访问集合, 可能要 BFS 或状压。

复杂度:

- 常见 $O(nm)$ 。
- 空间 $O(nm)$, 可滚动成 $O(m)$ 。

数据范围信号:

- $n*m \leq 1e7$ 左右可尝试。
- 网格特别大但障碍少, 可能要组合数学或压缩。

依赖的标准容器:

- 上限明确时用 `static ll a[MAXN][MAXM] / dp[MAXN][MAXM]`
- 字符网格可用 `vector<string> grid(n+1)` 且每行前面补一个空格, 保持 `grid[i][j]`

输入如何整理:

- 网格坐标统一 $1..n, 1..m$ 。
- 障碍格用 `blocked[i][j]` 或字符 '#'。

接口:

```
ll grid_dp(int n, int m);
```

输出能力:

- 路径数。
- 最小路径和。
- 最大收益。

下游可接：

- BFS 状态搜索。
- 状压钥匙/访问状态。
- DP-03B/DP-26 状态升维：上一步方向、是否用过机会等会影响未来时使用。

可拼接模块：

- PrefixSum：快速计算矩形/路径段贡献。
- BFS：方向不受限时替代。

状态句式：

$dp[i][j]$ 表示：走到格子 (i, j) 时的方案数/最小代价/最大收益。

初始化：

$dp[1][1] =$ 起点贡献或 1。

障碍格不转移。

最小值题其他状态为 $LINF$ ；最大值题其他状态为 $-LINF$ ；计数题为 0。

转移模板：

$dp[i][j] = \text{merge}(dp[i - 1][j], dp[i][j - 1]) + \text{cost}[i][j];$

计数：

$dp[i][j] = dp[i - 1][j] + dp[i][j - 1];$

答案位置：

- 通常 $dp[n][m]$ 。
- 终点不固定时取边界或全表最优。

循环顺序：

- i 从 1 到 n 。
- j 从 1 到 m 。
- 因为只依赖上方和左方。

暴力 DFS 版本：

```
ll dfs(int i, int j) {
    if (i > n || j > m || blocked[i][j]) return 0;
    if (i == n && j == m) return 1;
    return dfs(i + 1, j) + dfs(i, j + 1);
}
```

记忆化版本：

```
const int MAXN = 1000 + 5;
const int MAXM = 1000 + 5;
static ll memo[MAXN][MAXM];
static char vis[MAXN][MAXM];
static bool blocked[MAXN][MAXM];

ll dfs(int i, int j) {
    if (i > n || j > m || blocked[i][j]) return 0;
    if (i == n && j == m) return 1;
    if (vis[i][j]) return memo[i][j];
    vis[i][j] = 1;
    return memo[i][j] = dfs(i + 1, j) + dfs(i, j + 1);
}
```

表推版本：

```
static ll dp[MAXN][MAXM];
for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= m; j++) dp[i][j] = 0;
}
```

```

if (!blocked[1][1]) dp[1][1] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (blocked[i][j] || (i == 1 && j == 1)) continue;
        dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
}
cout << dp[n][m] << '\n';

```

最小路径和版本:

下面是无障碍版; 若有障碍格, 在循环里先 `if (blocked[i][j]) continue;` 并且起点/终点障碍要特判无解。

```

static ll dp[MAXN][MAXM];
for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= m; j++) dp[i][j] = LINF;
}
dp[1][1] = a[1][1];
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (i == 1 && j == 1) continue;
        ll best = min(dp[i - 1][j], dp[i][j - 1]);
        if (best != LINF) dp[i][j] = best + a[i][j];
    }
}

```

常见变体:

- 允许三方向: 多加一个前驱。
- 有障碍: 障碍格跳过。
- 路径取模计数: 每次加法取模。
- 上一步方向影响下一步: 状态升成 `dp[i][j][lastMove]`。
- 最多使用一次技能/翻倍: 状态升成 `dp[i][j][used]`。

常见坑:

- 第一行第一列初始化错误。
- 起点或终点是障碍没有特判。
- 允许回头时仍套普通网格 DP。
- 最小值题 `LINF + cost` 溢出, 转移前判断可达。

暴力/部分分替代:

- 小网格 DFS。
- 中等网格记忆化。
- 无环方向明确后改表推。

升级方向:

- 二维 -> 滚动数组。
- 加状态: `dp[i][j][k]` 表示还剩/已用资源。
- 可自由移动 -> BFS/Dijkstra。
- 发现 `dp[i][j]` 有后效性 -> DP-26 例题卡。

最小测试样例:

```

2 3
...
...
输出: 3

```

DP-13: 区间 DP

模型编号: DP-13

模型名称: 区间 DP

标签: DP、区间、合并、分割点、回文、两端取

一句话用途：处理“只考虑区间 $[l, r]$ ”的最优合并、删除、匹配或博弈类问题。

题面触发词：

- “区间合并”
- “合并石子”
- “括号匹配”
- “回文删除”
- “每次从两端取”
- “删除一段/合并相邻”

什么时候用：

- 子问题天然连续区间。
- 大区间可以由小区间合并。
- 转移枚举分割点 k 或两端决策。

不要什么时候用：

- 只是区间查询/修改，转数据结构。
- 子集不要求连续，可能是状压或背包。
- 数据 $n > 1000$ 且需要 $O(n^3)$ 时，必须找优化或别的模型。

复杂度：

- 常见 $O(n^3)$ ：长度 * 左端点 * 分割点。
- 两端转移可为 $O(n^2)$ 。

数据范围信号：

- $n \leq 300/500$ ： $O(n^3)$ 常见。
- $n \leq 5000$ ：只能做 $O(n^2)$ 或优化。

依赖的标准容器：

- `vector<vector<ll>> dp`
- `vector<ll> prefix`：区间和代价。

输入如何整理：

- 序列 1-index。
- 预处理 `sum(l, r)`。

接口：

```
ll interval_dp(int n, vector<ll>& a);
```

输出能力：

- 区间最小/最大合并代价。
- 回文/括号匹配最优值。
- 两端取最优差值。

下游可接：

- PrefixSum。
- 记忆化 DFS。

可拼接模块：

- PrefixSum: `cost(l, r)`。
- 记忆化搜索：先写 `dfs(l, r)`。

状态句式：

`dp[l][r]` 表示：只考虑闭区间 $[l, r]$ 时的最优值/方案数。

初始化：

`len = 1` 的区间通常为 0 或 `a[l]`，按题意。

最小值题其他状态为 `LINF`；最大值题为 `-LINF`。

转移模板：

枚举分割点：

```
dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + cost(l, r));
```

两端选择:

```
dp[l][r] = max(a[l] - dp[l + 1][r], a[r] - dp[l][r - 1]);
```

答案位置:

- 整段答案: `dp[1][n]`。

循环顺序:

```
for len = 1..n
  for l = 1..n-len+1
    r = l + len - 1
    枚举转移
```

暴力 DFS 版本:

```
ll dfs(int l, int r) {
  if (l == r) return 0;
  ll ans = LINF;
  for (int k = l; k < r; k++) {
    ans = min(ans, dfs(l, k) + dfs(k + 1, r) + sum(l, r));
  }
  return ans;
}
```

记忆化版本:

```
vector<vector<ll>> memo;
vector<vector<int>> vis;

ll dfs(int l, int r) {
  if (l == r) return 0;
  if (vis[l][r]) return memo[l][r];
  vis[l][r] = 1;
  ll ans = LINF;
  for (int k = l; k < r; k++) {
    ans = min(ans, dfs(l, k) + dfs(k + 1, r) + sum(l, r));
  }
  return memo[l][r] = ans;
}
```

表推版本:

```
vector<vector<ll>> dp(n + 2, vector<ll>(n + 2, 0));
for (int len = 2; len <= n; len++) {
  for (int l = 1; l + len - 1 <= n; l++) {
    int r = l + len - 1;
    dp[l][r] = LINF;
    for (int k = l; k < r; k++) {
      dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + sum(l, r));
    }
  }
}
cout << dp[1][n] << '\n';
```

常见变体:

- 环形区间 DP: 复制一倍序列, 枚举长度 n 的区间。
- 回文匹配: 转移可能看 `a[l] == a[r]`。
- 两端取博弈: `dp[l][r]` 常表示当前玩家领先分差。

常见坑:

- 没按长度从小到大, 导致依赖未算。
- 分割点范围写成 `k <= r`。
- `sum(l, r)` 没有 $O(1)$ 预处理, 复杂度多乘一层。
- `dp[l][r]` 初值忘记按最小/最大设置。

暴力/部分替代:

- $n \leq 20$: 递归枚举。
- $n \leq 300$: 区间表推。
- 表推顺序想不清: 先记忆化 $\text{dfs}(1, r)$ 。

升级方向:

- PrefixSum 优化代价。
- 四边形不等式/Knuth 优化属于高阶, 低优先级。
- 环形问题复制数组。

最小测试样例:

```
3
1 2 3
输出: 9
说明: 先合并 1+2 花 3, 再和 3 合并花 6, 总 9。
```

DP-14: 树形 DP

模型编号: DP-14

模型名称: 树形 DP

标签: DP、树、子树、选点、覆盖、染色

一句话用途: 在树上按子树合并状态, 处理选点、覆盖、染色、最大独立集等问题。

题面触发词:

- “树上选择若干点”
- “父子不能同时选”
- “覆盖所有点”
- “树上染色”
- “以子树为单位”

什么时候用:

- 输入是一棵树或森林。
- 一个节点的答案可以由儿子子树合并。
- 状态通常描述 u 自己选不选、颜色或覆盖情况。

不要什么时候用:

- 图有环且不是树, 普通树形 DP 不适用。
- 问树上路径多次查询, 可能是 LCA/树链/数据结构。
- 子树之间有额外横向边, 状态不独立。

复杂度:

- 常见 $O(n * \text{状态数}^2)$ 或 $O(n * \text{状态数})$ 。

数据范围信号:

- $n \leq 2e5$ 且状态少: 树形 DP 可做。
- 状态含背包容量: 复杂度可能变成 $O(nw^2)$ 或 $O(nw)$ 。

依赖的标准容器:

- 标准 Graph G 。
- 本页后续 $\text{vector}<\text{vector}<\text{int}>> g(n + 1)$ 是速抄变体; 与图论卷拼接时优先从 $G.g$ 取 $e.to/e.w$ 。
- $\text{vector}<\text{array}<\text{ll}, 2>> dp$
- $\text{vector}<\text{int}> parent, order$

输入如何整理:

- 建无向树。
- DFS 时传 $parent$ 防止走回父亲。
- 权值统一 $w[u]$ 。

接口:

```
void dfs(int u, int p);
```


输出能力:

- 树上最大/最小选择值。
- 覆盖、染色、方案数。

下游可接:

- Graph 标准建图。
- 树上背包。
- DP-03B 状态增维: 不同方向进入同一节点、parent/prev 影响未来时检查 memo key。

可拼接模块:

- Graph: 统一邻接表。
- 0/1 背包: 子树容量合并。
- 记忆化 DFS: 状态复杂时先写。

状态句式:

$dp[u][state]$ 表示: 只考虑 u 的子树, 并且 u 自己处于 $state$ 状态时的最优值/方案数。

为什么这个状态够用: 固定根以后, 不同儿子子树之间没有横向边; 父亲对子树的影响只通过 u 的状态传下来。只要 u 的状态确定, 每个儿子子树就可以独立求解, 再合并。

最大独立集常用:

$dp[u][0]$ 表示: u 不选时, u 子树最大权值。

$dp[u][1]$ 表示: u 选时, u 子树最大权值。

初始化:

$dp[u][0] = 0$: 不选 u , 初始没有贡献。

$dp[u][1] = w[u]$: 选 u , 先拿 u 的权值。

转移模板:

```
dp[u][0] += max(dp[v][0], dp[v][1]);
dp[u][1] += dp[v][0];
```

标准 Graph G 循环写法:

```
for (auto e : G.g[u]) {
    int v = e.to;
    ll w = e.w;
    if (v == p) continue;
    // 树边权不用时忽略 w
}
```

速抄局部 g 变体:

```
for (int v : g[u]) {
    if (v == p) continue;
}
```

答案位置:

- 根为 1: $\max(dp[1][0], dp[1][1])$ 。
- 森林: 对每棵树答案相加。

循环顺序:

- 后序 DFS: 先算儿子, 再合并到父亲。
- 或先拿 DFS 序, 再按逆序表推。

暴力 DFS 版本:

```
ll brute(int u, int p, int parentChosen) {
    ll ans0 = 0; // 不选 u
    for (int v : g[u]) if (v != p) ans0 += brute(v, u, 0);

    ll ans1 = -LINF;
    if (!parentChosen) {
        ans1 = w[u];
        for (int v : g[u]) if (v != p) ans1 += brute(v, u, 1);
    }
}
```

```

    return max(ans0, ans1);
}

```

记忆化版本:

注意: 下面 memo 版本只适用于固定根以后, u 的父亲唯一的树形 DP。若换根, 或同一个 u 可能带不同 p, 必须把 p/parent 纳入状态, 或者不要缓存。

```

vector<array<ll, 2>> memo;
vector<array<int, 2>> vis;

ll dfs_memo(int u, int p, int parentChosen) {
    if (vis[u][parentChosen]) return memo[u][parentChosen];
    vis[u][parentChosen] = 1;

    ll ans0 = 0;
    for (int v : g[u]) if (v != p) ans0 += dfs_memo(v, u, 0);

    ll ans1 = -LINF;
    if (!parentChosen) {
        ans1 = w[u];
        for (int v : g[u]) if (v != p) ans1 += dfs_memo(v, u, 1);
    }
    return memo[u][parentChosen] = max(ans0, ans1);
}

```

表推版本:

```

vector<array<ll, 2>> dp(n + 1);

void dfs(int u, int p) {
    dp[u][0] = 0;
    dp[u][1] = w[u];
    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}

dfs(1, 0);
cout << max(dp[1][0], dp[1][1]) << '\n';

```

常见变体:

- 最小点覆盖: $dp[u][0] += dp[v][1]$, $dp[u][1] += \min(dp[v][0], dp[v][1])$ 。
- 树上染色: $dp[u][color]$ 合并儿子颜色。
- 树上背包: $dp[u][k]$ 表示子树选 k 个。

常见坑:

- 忘记传父亲, 递归走回去。
- 根的父亲状态处理错。
- 递归深度过大可能爆栈。
- 子树合并时更新顺序覆盖旧值。
- 固定根树形 DP 中 parent 通常是 DFS 参数; 如果同一个 u 可能从不同方向进入并被缓存, parent/prev 必须进入状态。
- n 接近 $2e5$ 时递归可能爆栈; 如果现场栈限制严格, 改用父数组 + DFS 序逆序表推更稳。

暴力/部分替代:

- $n \leq 20$: 枚举点集检查边约束。
- 树结构明确但转移复杂: 先写递归 + memo。
- 多个连通块: 逐个根 DFS。

升级方向:

- 简单选点 -> $dp[u][0/1]$ 。
- 覆盖问题 -> $dp[u][0/1/2]$ 。

- 容量限制 -> 树上背包。
- 换根或无固定父亲的记忆化 -> 先检查 DP-03B 的 parent/prev 维度。

最小测试样例：

```
3
w: 1 2 3
edges: 1-2, 1-3
输出: 5
说明: 选 2 和 3。
```

DP-15: DAG DP

模型编号: DP-15

模型名称: DAG DP

标签: DP、拓扑排序、有向无环图、路径计数、依赖关系

一句话用途: 在有向无环图上按拓扑序转移, 求路径最优值、方案数或依赖完成方式。

题面触发词:

- “有向无环图”
- “依赖关系”
- “先修课程”
- “任务 A 必须在 B 前”
- “从起点到终点的路径数/最长路”

什么时候用:

- 状态之间是有向依赖且无环。
- 可以拓扑排序。
- $dp[u]$ 能由前驱或后继转移。

不要什么时候用:

- 图有环, 普通 DAG DP 不适用。
- 边权非负最短路在一般图上, 转 Dijkstra。
- 无权最少步数, 转 BFS。

复杂度:

- 拓扑排序 $O(n+m)$ 。
- DP 转移 $O(n+m)$ 。

数据范围信号:

- $n, m \leq 2e5$: DAG DP 很适合。
- 若 $n \leq 20$ 且集合访问, 可能是状压。

依赖的标准容器:

- 标准 Graph G。
- 本页后续 `vector<vector<pair<int,ll>>> g` 是速抄变体; 与图论卷拼接时优先使用 `G.g[u]` 中的 `e.to/e.w`。
- `vector<int> indeg, topo`
- `queue<int>`
- `vector<ll> dp`

输入如何整理:

- 建有向边 $u \rightarrow v$ 。
- 统计入度。
- 如果题意是 “u 依赖 v”, 先确认边方向。

接口:

```
vector<int> topo_sort(const Graph& G);
```

输出能力:

- 到达每点的最长/最短路径。

- 路径方案数。
- 依赖完成最早时间。

下游可接：

- Graph 标准建图。
- Topo 模块。

可拼接模块：

- Graph。
- 拓扑排序。
- 取模计数。

状态句式：

$dp[u]$ 表示：到达 u 的最优值/方案数。

为什么这个状态够用：DAG 的拓扑序保证所有影响 u 的前驱都已经处理完；未来只需要知道“到达 u 的当前最优/方案数”，不需要知道具体路径历史。

也可反向：

$dp[u]$ 表示：从 u 出发到终点的最优值/方案数。

初始化：

起点 s ： $dp[s] = 0/1$ 。

最大值题其他点为 $-LINF$ ；最小值题为 $LINF$ ；方案数为 0 。

转移模板：

```
for (auto e : G.g[u]) {
    int v = e.to;
    ll w = e.w;
    dp[v] = max(dp[v], dp[u] + w);
}
```

速抄局部 g 变体：

```
for (auto [v, w] : g[u]) {
    dp[v] = max(dp[v], dp[u] + w);
}
```

方案数：

$ways[v] = (ways[v] + ways[u]) \% MOD$;

答案位置：

- 指定终点： $dp[t]$ 。
- 任意终点： $\max/\min dp[u]$ 。
- 路径总数：通常 $ways[t]$ 。

循环顺序：

- 先拓扑排序。
- 按拓扑序从前往后转移。
- 反向定义时按逆拓扑序。

暴力 DFS 版本：

```
ll dfs(int u) {
    if (u == t) return 0;
    ll ans = -LINF;
    for (auto [v, w] : g[u]) {
        ll sub = dfs(v);
        if (sub != -LINF) ans = max(ans, w + sub);
    }
    return ans;
}
```

记忆化版本：

```

vector<ll> memo;
vector<int> vis;

ll dfs(int u) {
    if (u == t) return 0;
    if (vis[u]) return memo[u];
    vis[u] = 1;
    ll ans = -LINF;
    for (auto [v, w] : g[u]) {
        ll sub = dfs(v);
        if (sub != -LINF) ans = max(ans, w + sub);
    }
    return memo[u] = ans;
}

```

表推版本:

```

queue<int> q;
for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
vector<int> topo;
while (!q.empty()) {
    int u = q.front(); q.pop();
    topo.push_back(u);
    for (auto [v, w] : g[u]) {
        if (--indeg[v] == 0) q.push(v);
    }
}
if ((int)topo.size() != n) {
    // 有环时普通 DAG DP 不适用; 按题意输出无解, 或重新路由到图算法。
    return;
}

```

```

vector<ll> dp(n + 1, -LINF);
dp[s] = 0;
for (int u : topo) {
    if (dp[u] == -LINF) continue;
    for (auto [v, w] : g[u]) dp[v] = max(dp[v], dp[u] + w);
}
cout << dp[t] << '\n';

```

常见变体:

- 最短路 DAG: min 转移, 初值 LINF。
- 路径计数: ways[s]=1。
- 任务最早完成时间: 边/点权表示耗时。

常见坑:

- 图有环但仍递归, 导致死循环。
- 边方向建反。
- 多个入度 0 起点没有初始化。
- 修改 indeg 后还要复用原入度, 需备份。

暴力/部分分替代:

- 小图 DFS 枚举路径。
- 无环但顺序不清: 记忆化 DFS。
- 大图: 拓扑表推。

升级方向:

- DFS memo -> 拓扑 DP。
- 与 Graph 标准容器统一。
- 若有环且问最长路, 通常不是普通 DP, 要重新路由。

最小测试样例:

```

4 4
1 2 3

```

```
1 3 2
2 4 4
3 4 5
s=1 t=4
输出: 7
```

DP-16: 状压 DP

模型编号: DP-16

模型名称: 状压 DP

标签: DP、集合、二进制、mask、TSP

一句话用途: 当 n 很小且状态是“已选择/已访问集合”时, 用二进制 mask 表示集合。

题面触发词:

- $n \leq 20$
- “访问所有点”
- “每个点选或不选且集合状态重要”
- “钥匙/开关集合”
- “哈密顿路径/TSP”

什么时候用:

- 元素数量小。
- 需要知道哪些元素已经被选过/访问过。
- 状态由集合和最后位置等少量信息决定。

不要什么时候用:

- $n > 25$ 通常不能直接 2^n 。
- 不需要记集合, 只需容量/数量时, 用背包。
- 状态图边权非负且集合不是关键, 可能是最短路。

复杂度:

- 常见 $O(2^n * n^2)$ 。
- 空间 $O(2^n * n)$ 。

数据范围信号:

- $n \leq 20$: 标准状压。
- $n \leq 21/22$: 不是默认模板可直接套的范围; 必须重新估算内存、改 KMAX/SMAX, 必要时压缩或折半。
- $n \geq 25$: 通常要剪枝、折半或其他模型。

依赖的标准容器:

- 逻辑点编号统一 $1..k$ 。
- 位运算用 $1 \ll (i - 1)$ 表示第 i 个关键点。
- 上限明确时优先全局静态数组: `dist[KMAX+1][KMAX+1]`、`dp[SMAX][KMAX+1]`。

输入如何整理:

- 关键点编号保持 $1..k$, 不要改成 0-index。
- mask 只是集合编码; 第 i 个关键点对应第 $i-1$ 位。
- 若原图有边, 先算关键点之间距离。

1-index Graph/Floyd 到 1-index 关键点矩阵适配食谱:

```
const int KMAX = 20;
const int SMAX = 1 << KMAX;
const int MAXN = 305;
ll all_dist[MAXN][MAXN]; // 按 Floyd/Dijkstra 预处理好的原图距离, 原图点号 1..n
ll dist[KMAX + 1][KMAX + 1];
int key[KMAX + 1];

// G 是标准 1-index Graph, key[1..k] 是关键点在原图中的 1-index 点号。
for (int i = 1; i <= k; i++) {
```

```

    for (int j = 1; j <= k; j++) {
        dist[i][j] = all_dist[key[i]][key[j]];
    }
}

```

状压 DP 里的 last/nxt 全部用 1..k 的关键点编号；原图点号只留在 key[i] 里。若 dist[i][j] == LINF，表示两个关键点不可达，转移时跳过。

接口：

```
ll tsp(int k, int start);
```

输出能力：

- 访问全部点的最小代价。
- 集合方案数。
- 最后停在某点的最优值。

下游可接：

- Floyd/Dijkstra：预处理关键点距离。
- BFS：网格钥匙状态。
- DP-03B 状态增维：只记 mask 不够时，加 last 表示当前位置。

可拼接模块：

- Floyd：全源最短路。
- Dijkstra：关键点间最短路。
- Bitset/位运算。

状态句式：

dp[mask][last] 表示：已经访问的点集为 mask，并且当前最后停在 last 时的最小代价/最大收益。
last 是 1..k 的关键点编号；mask 的第 last-1 位表示 last 是否已访问。

为什么这个状态够用：访问过哪些点决定“以后还能去哪些点”，当前停在哪个点决定“下一步代价从哪里出发”。历史访问顺序本身不再影响未来，所以 mask + last 可以吸收全部关键历史。

初始化：

dp[1 << (start - 1)][start] = 0：一开始只访问起点，代价为 0。
其他状态为 LINF/-LINF。

转移模板：

```

nmask = mask | (1 << (nxt - 1));
dp[nmask][nxt] = min(dp[nmask][nxt], dp[mask][last] + dist[last][nxt]);

```

答案位置：

- 不要求回起点：min dp[(1<<k)-1][last]。
- 要求回起点：min dp[full][last] + dist[last][start]。

循环顺序：

- mask 从 0 到 full。
- 枚举 last。
- 枚举未访问 nxt。

暴力 DFS 版本：

```

ll dfs(int mask, int last) {
    if (mask == full) return 0;
    ll ans = LINF;
    for (int nxt = 1; nxt <= k; nxt++) {
        if (mask >> (nxt - 1) & 1) continue;
        if (dist[last][nxt] == LINF) continue;
        ll sub = dfs(mask | (1 << (nxt - 1)), nxt);
        if (sub != LINF) ans = min(ans, dist[last][nxt] + sub);
    }
    return ans;
}

```

记忆化版本：

```

ll memo[SMAX][KMAX + 1];
bool vis[SMAX][KMAX + 1];

ll dfs(int mask, int last) {
    if (mask == full) return 0;
    if (vis[mask][last]) return memo[mask][last];
    vis[mask][last] = 1;
    ll ans = LINF;
    for (int nxt = 1; nxt <= k; nxt++) {
        if (mask >> (nxt - 1) & 1) continue;
        if (dist[last][nxt] == LINF) continue;
        ll sub = dfs(mask | (1 << (nxt - 1)), nxt);
        if (sub != LINF) ans = min(ans, dist[last][nxt] + sub);
    }
    return memo[mask][last] = ans;
}

```

表推版本:

```

ll dp[SMAX][KMAX + 1];

int total = 1 << k;
int full = total - 1;
for (int mask = 0; mask < total; mask++) {
    for (int i = 1; i <= k; i++) dp[mask][i] = LINF;
}
dp[1 << (start - 1)][start] = 0;

for (int mask = 0; mask < total; mask++) {
    for (int last = 1; last <= k; last++) {
        if (dp[mask][last] == LINF) continue;
        for (int nxt = 1; nxt <= k; nxt++) {
            if (mask >> (nxt - 1) & 1) continue;
            if (dist[last][nxt] == LINF) continue;
            int nmask = mask | (1 << (nxt - 1));
            dp[nmask][nxt] = min(dp[nmask][nxt], dp[mask][last] + dist[last][nxt]);
        }
    }
}

ll ans = LINF;
for (int last = 1; last <= k; last++) {
    if (dp[full][last] == LINF) continue;
    ans = min(ans, dp[full][last]); // 如果要求回到 start, 再检查 dist[last][start] 后加上它
}
cout << ans << '\n';

```

常见变体:

- `dp[mask]`: 不关心最后位置。
- 子集枚举: `for (sub = mask; sub; sub = (sub-1)&mask)`。
- 网格钥匙: BFS 状态 (x, y, mask) , 不一定是表推 DP。

常见坑:

- $1 << n$ 用 `int` 时 n 太大溢出; 必要时用 `1LL << n`。
- 逻辑点编号保持 1-index; 取位时统一写 $(i - 1)$, 不要把关键点数组改成 0-index。
- `full` 变量在 DFS 前未初始化。
- 内存 $2^n * n$ 爆掉。
- 只记 `mask` 但转移还需要知道当前停在哪个点时, 必须写成 `dp[mask][last]`。

暴力/部分分替代:

- $n \leq 10$: 排列枚举。
- $n \leq 20$: 状压 DP。
- 图上访问关键点: 先算关键点最短路, 再状压。

升级方向：

- 暴力排列 -> DFS mask -> memo -> 表推。
- 与 Floyd/Dijkstra 拼接。
- 子集 DP 用子集枚举优化。
- 状态漏当前位置/钥匙/阶段时，回 DP-03B/DP-26 检查需要加哪一维。

最小测试样例：

k=3 start=1

dist:

0 1 5

1 0 2

5 2 0

输出：3

说明：1 -> 2 -> 3。

DP-17: 数位 DP

模型编号：DP-17

模型名称：数位 DP

标签：DP、数字位、上界限制、计数、记忆化搜索

一句话用途：统计 $1..N$ 或 $L..R$ 中满足数位条件的正整数数量；如果题目把 0 也算合法，在 base case 里打开 count_zero。

题面触发词：

- “1 到 N 中有多少个数”
- “数字各位满足条件”
- “不含某个数字”
- “数位和”
- “上界很大，N 可到 10^{18} 或 10^{100} ”

什么时候用：

- 判断条件和数字的每一位有关。
- 上界太大，不能枚举所有数。
- 状态能用 “当前位 + 是否贴上界 + 前导零 + 附加信息” 描述。

不要什么时候用：

- N 很小，直接枚举更简单。
- 条件不是数位局部状态能表达。
- 需要统计的是排列/组合对象，不是数字范围。

复杂度：

- $O(\text{位数} * \text{状态数} * 10)$ 。

数据范围信号：

- $N \leq 1e18$ ：位数最多 19。
- N 是长字符串：位数可达几千，也要看状态大小。

依赖的标准容器：

- `vector<int> digits`
- `map<tuple<...>, ll>` 或多维数组 memo

输入如何整理：

- 把 N 拆成从高位到低位的 digits。
- 计算 $[L, R]$ 时用 `solve(R) - solve(L-1)`。

接口：

`ll solve(long long N);`

输出能力：

- 满足数位条件的数量。

- 可扩展为数位和总和、最大/最小数字。

下游可接：

- 取模计数。
- 记忆化 DFS。

可拼接模块：

- 数学取模。
- `map<tuple> memo`。

状态句式：

`dfs(pos, tight, leading, state)` 表示：处理到第 `pos` 位，是否贴着上界 `tight`，是否仍是前导零 `leading`，附加状态为 `state` 时，后面能

初始化：

从 `pos = 0` 开始，`tight = true`，`leading = true`，`state = 初始状态`。

到 `pos == 位数` 时，按 `state` 判断是否计 1。

转移模板：

```
int up = tight ? digits[pos] : 9;
for (int d = 0; d <= up; d++) {
    ntight = tight && (d == up);
    nleading = leading && (d == 0);
    nstate = update(state, d, nleading);
}
```

答案位置：

- `solve(N)`。
- 区间：`solve(R) - solve(L - 1)`。

循环顺序：

- 记忆化 DFS 通常最稳。
- 表推可按 `pos` 从高位到低位推进，状态带 `tight/leading`。

暴力 DFS 版本：

```
ll dfs_plain(int pos, bool tight, bool leading, int sum) {
    bool count_zero = false; // 如果 0 也是合法数字，改成 true
    if (pos == len) return ((!leading || count_zero) && sum % K == 0) ? 1 : 0;
    int up = tight ? digits[pos] : 9;
    ll ans = 0;
    for (int d = 0; d <= up; d++) {
        bool ntight = tight && (d == up);
        bool nleading = leading && (d == 0);
        int nsum = nleading ? 0 : (sum + d) % K;
        ans += dfs_plain(pos + 1, ntight, nleading, nsum);
    }
    return ans;
}
```

记忆化版本：

```
const int MAXD = 20; // Long Long 上界最多 19 位；大数字字符串按题目改大
const int MAXK = 205; // 按题目 K 调大
static ll memo[MAXD][MAXK][2];
static char vis[MAXD][MAXK][2];
bool count_zero = false; // 如果 0 也是合法数字，改成 true

ll dfs(int pos, bool tight, bool leading, int sum) {
    if (pos == len) return ((!leading || count_zero) && sum % K == 0) ? 1 : 0;
    if (!tight && vis[pos][sum][leading]) return memo[pos][sum][leading];

    int up = tight ? digits[pos] : 9;
    ll ans = 0;
    for (int d = 0; d <= up; d++) {
        bool ntight = tight && (d == up);
```

```

        bool nleading = leading && (d == 0);
        int nsum = nleading ? 0 : (sum + d) % K;
        ans += dfs(pos + 1, ntight, nleading, nsum);
    }

    if (!tight) {
        vis[pos][sum][leading] = 1;
        memo[pos][sum][leading] = ans;
    }
    return ans;
}

ll solve(long long N) {
    if (N < 0) return 0;
    digits.clear();
    string s = to_string(N);
    for (char c : s) digits.push_back(c - '0');
    len = (int)digits.size();

    // 每次 solve 都清空 vis, 避免 solve(R) 与 solve(L-1) 串缓存。
    memset(vis, 0, sizeof(vis));
    return dfs(0, true, true, 0);
}

```

如果 N 是长字符串, 把 solve(long long N) 改成 solve_string(const string& s), 不要再 to_string:

```

ll solve_string(const string& s) {
    digits.clear();
    for (char c : s) digits.push_back(c - '0');
    len = (int)digits.size();
    memset(vis, 0, sizeof(vis)); // MAXD 必须开到最大位数 + 1
    return dfs(0, true, true, 0);
}

```

如果位数或 K 由输入决定, 固定 MAXD/MAXK 不够时改用动态 vector 或 map<tuple<...>, ll>, 别硬抄固定数组。

表推版本:

```

// 统计 1..N 中数位和 mod K == 0 的正整数数量; 若 0 合法, 另行按题意处理
static ll dp[MAXD][MAXK][2][2];
memset(dp, 0, sizeof(dp));
dp[0][0][1][1] = 1; // pos=0, sum=0, tight=true, leading=true

for (int pos = 0; pos < len; pos++) {
    for (int sum = 0; sum < K; sum++) {
        for (int tight = 0; tight <= 1; tight++) {
            for (int leading = 0; leading <= 1; leading++) {
                ll cur = dp[pos][sum][tight][leading];
                if (!cur) continue;
                int up = tight ? digits[pos] : 9;
                for (int d = 0; d <= up; d++) {
                    int ntight = tight && (d == up);
                    int nleading = leading && (d == 0);
                    int nsum = nleading ? 0 : (sum + d) % K;
                    dp[pos + 1][nsum][ntight][nleading] += cur;
                }
            }
        }
    }
}

ll ans = dp[len][0][0][0] + dp[len][0][1][0];

```

常见变体:

- 不含数字 x: 遇到 d==x 跳过。
- 相邻位不能相同: 状态加 lastDigit。

- 数字模 M: 状态加 rem。

常见坑:

- tight 状态直接缓存, 导致不同上界前缀混淆; 通常只缓存 tight=false。
- 前导零是否算入条件没想清。
- 默认模板不统计数字 0; 如果题目要求统计 $0..N$ 且 0 合法, 设置 count_zero 或对答案单独 +1。
- $[L, R]$ 忘记处理 $L=0$ 。
- 若 N 是上百位长字符串, 答案可能超过 long long; 此时必须按题目要求取模, 或改用大整数。
- ntight 写成 $d == up$ 在 tight=false 时也为真; 正确是 $tight \ \&\& \ d == up$ 。
- K 可能很大时不要用固定数组; 按 $len + 1$ 和 K 动态开 memo/vis 更稳。
- 每次 solve(N) 要重建 digits 并清空 memo/vis, 区间 solve(R)-solve(L-1) 不能串缓存。

暴力/部分替代:

- $N \leq 1e6$: 直接枚举检查数位。
- 位数小但状态复杂: 先写无 memo DFS。
- 状态清楚后, 只缓存 tight=false 的状态。

升级方向:

- 暴力枚举数字 -> 数位 DFS。
- DFS -> 记忆化。
- 需要避免递归时改表推。

最小测试样例:

$N=20 \ K=3$

输出: 6

说明: 3, 6, 9, 12, 15, 18 的数位和能被 3 整除。

DP-18: DP + 数据结构优化

模型编号: DP-18

模型名称: DP + 数据结构优化

标签: DP、树状数组、SegmentTree、单调队列、前缀和、优化转移

一句话用途: 当普通 DP 需要枚举大量前驱 j 时, 用数据结构快速查询前驱最优值。

题面触发词:

- $n \leq 2e5$ 但朴素 DP 是 $O(n^2)$
- “从前面某个范围转移”
- “满足 $a[j] \leq a[i]$ 的最大 $dp[j]$ ”
- “滑动窗口内最优”
- “区间最大/最小前缀”

什么时候用:

- 已经写出朴素转移 $dp[i] = \text{merge}(dp[j] + \text{gain})$ 。
- 前驱 j 的合法条件能变成区间查询、前缀查询或滑窗最值。
- 数据范围要求优化到 $O(n \log n)$ 或 $O(n)$ 。

不要什么时候用:

- 朴素 DP 已能过, 不必强行优化。
- 前驱条件无法用单调性、区间或值域表达。
- 转移有复杂二维条件, 数据结构维护不了。

复杂度:

- PrefixSum 优化: 通常 $O(n^2)$ 降一层或减少求代价常数。
- 树状数组/SegmentTree: $O(n \log n)$ 。
- 单调队列: $O(n)$ 。

数据范围信号:

- $n \leq 5000$: 朴素 $O(n^2)$ 可先交。
- $n \leq 2e5$: 优先找 树状数组/SegmentTree/单调队列。

依赖的标准容器：

- `vector<ll> dp`
- `Compressor`
- 树状数组最大值版 或 `SegmentTree`
- `deque<int>`: 单调队列

输入如何整理：

- 把前驱限制写成查询条件: $key[j] \leq key[i]$ 、 $L \leq j \leq R$ 、 $value \in [l,r]$ 。
- 值域大时先坐标压缩。

接口：

```
// 查询前面  $key \leq x$  的最大  $dp$   
best = bit.prefix(pos(x));  
bit.add(pos(key[i]), dp[i]);
```

输出能力：

- 优化后的最大值/最小值 DP。
- 带权 LIS、分段 DP、滑窗 DP。

下游可接：

- 树状数组。
- `SegmentTree`。
- `PrefixSum`。
- `MonotonicQueue`。
- `Compressor`。

可拼接模块：

- DS-树状数组：前缀最大/最小。
- DS-`SegmentTree`：区间最大/最小。
- DS-`MonotonicQueue`：滑动窗口最值。
- `PrefixSum`：`cost(l,r)`。

状态句式：

$dp[i]$ 表示：以第 i 个元素作为最后一步时的最优值。

优化查询句式：

需要从所有满足条件的 j 中取 $\max/\min dp[j]$ ，把这个集合交给数据结构查询。

先问：这个 DP 真的能用数据结构优化吗？

考场上不要一看到 $O(n^2)$ 就硬套树状数组或线段树。先把朴素式子写完整：

$dp[i] = \max/\min \text{ over } j < i \text{ and ok}(j,i) \text{ of } dp[j] + \text{gain}(j,i)$

然后按下面四问判断：

问题	如果答案是“是”	常用模块
合法前驱是不是一段下标区间？	$L(i) \leq j \leq R(i)$	单调队列 / <code>SegmentTree</code>
合法前驱是不是一段值域区间？	$a[j] \leq a[i]$ 或 $L \leq a[j] \leq R$	树状数组 / <code>SegmentTree</code> + 离散化
$\text{gain}(j,i)$ 能不能拆成“只和 j 有关 + 只和 i 有关”？	$dp[j] + A[j] + B[i]$	维护 $dp[j]+A[j]$ 的最值
区间代价是不是反复求和？	$\text{cost}(l,r)$	<code>PrefixSum</code> / 二维前缀和

如果四个问题都答不上来，就先交朴素 $O(n^2)$ 或记忆化版本。强行上数据结构通常会把状态含义写乱。

初始化：

$dp[i]$ 至少可以单独选择 i ，初值为 $\text{base}(i)$ 。

树状数组/`SegmentTree` 初始为不可达值；如果允许空前驱，先加入空状态。

转移模板：

朴素：

```

dp[i] = base(i);
for (int j = 1; j < i; j++) {
    if (ok(j, i)) dp[i] = max(dp[i], dp[j] + gain(j, i));
}

```

树状数组优化:

```

dp[i] = base(i) + bit.prefix(pos(key[i]));
bit.add(pos(key[i]), dp[i]);

```

这段只适用于 $gain(j, i)$ 能拆成 “前驱可维护的值 + 当前 i 的值” 的情况。若真实转移是 $dp[j] + cost(j, i)$ 且 $cost$ 同时复杂依赖 j 和 i , 树状数组不能直接替代枚举。

答案位置:

- 常见: $\max(dp[1..n])$ 。
- 如果必须以 n 结尾: $dp[n]$ 。

循环顺序:

- 通常 i 从 1 到 n 。
- 查询只看已经加入的数据结构的前驱。
- 算完 $dp[i]$ 后再把 i 加入数据结构。

暴力 DFS 版本:

```

ll dfs(int i) {
    if (vis[i]) return memo[i];
    vis[i] = 1;
    ll ans = base(i);
    for (int j = 1; j < i; j++) {
        if (ok(j, i)) ans = max(ans, dfs(j) + gain(j, i));
    }
    return memo[i] = ans;
}

```

记忆化版本:

```

vector<ll> memo(n + 1);
vector<int> vis(n + 1, 0);

ll dfs(int i) {
    if (vis[i]) return memo[i];
    vis[i] = 1;
    ll ans = base(i);
    for (int j = 1; j < i; j++) {
        if (ok(j, i)) ans = max(ans, dfs(j) + gain(j, i));
    }
    return memo[i] = ans;
}

```

表推版本 (朴素):

```

vector<ll> dp(n + 1, -LINF);
ll ans = -LINF;
for (int i = 1; i <= n; i++) {
    dp[i] = base(i);
    for (int j = 1; j < i; j++) {
        if (ok(j, i)) dp[i] = max(dp[i], dp[j] + gain(j, i));
    }
    ans = max(ans, dp[i]);
}
cout << ans << '\n';

```

树状数组最大值版本:

```

struct BITMax {
    int n;
    vector<ll> bit;
    BITMax(int n = 0) { init(n); }
}

```

```

void init(int n_) { n = n_; bit.assign(n + 1, -LINF); }
void add(int idx, ll val) {
    for (; idx <= n; idx += idx & -idx) bit[idx] = max(bit[idx], val);
}
ll prefix(int idx) {
    ll ans = -LINF;
    for (; idx > 0; idx -= idx & -idx) ans = max(ans, bit[idx]);
    return ans;
}
};

// 例: 带权 LIS, 要求  $a[j] < a[i]$ , 收益为  $w[i]$ 
vector<ll> vals = a;
sort(vals.begin() + 1, vals.end());
vals.erase(unique(vals.begin() + 1, vals.end()), vals.end());
auto pos = [&](ll x) {
    return int(lower_bound(vals.begin() + 1, vals.end(), x) - vals.begin());
};

BITMax bit((int)vals.size() + 2);
ll ans = 0; // 如果必须选至少一个且权值可能全负, 改成 -LINF
for (int i = 1; i <= n; i++) {
    int p = pos(a[i]);
    ll best = bit.prefix(p - 1);
    ll dp_i = w[i] + max(0LL, best);
    bit.add(p, dp_i);
    ans = max(ans, dp_i);
}
cout << ans << '\n';

```

如果题目要求至少选一个且 $w[i]$ 可能为负, 保留 $dp_i = w[i] + \max(0LL, best)$ 仍然可以让每个元素单独开头, 但 ans 必须初始化为 $-LINF$, 不能用 0 当空方案答案。

单调队列优化句式:

如果 $dp[i] = \min(dp[j] + cost(i))$ 且 j 只在 $[i-K, i-1]$, 维护窗口内最小 $dp[j]$ 。

常见变体:

- $key[j] \leq key[i]$: 树状数组前缀查询。
- $L \leq key[j] \leq R$: SegmentTree 区间查询。
- $i-K \leq j \leq i-1$: 单调队列。
- $cost(l, r)$ 多次出现: PrefixSum。

常见坑:

- 先 $add(i)$ 再 $query$, 把自己当成前驱。
- 坐标压缩后严格 $<$ 和 $\leq$ 的查询位置不同。
- 树状数组默认做前缀, 区间查询要用 SegmentTree 或两个前缀处理。
- 最大值树初始值不能是 0 , 除非空前驱合法。

暴力/部分替代:

- $n \leq 5000$: 朴素 $O(n^2)$ 。
- 优化写不出: 先交朴素表推。
- 查询条件复杂: 先用记忆化/朴素确认答案。

升级方向:

- 朴素前驱枚举 $\rightarrow$ 树状数组/SegmentTree 查询。
- 固定窗口 $\rightarrow$ 单调队列。
- 区间代价重复计算 $\rightarrow$ PrefixSum。
- 空间大 $\rightarrow$ 滚动数组。

最小测试样例:

```

5
a: 1 3 2 4 3
w: 2 5 4 6 7

```

带权严格递增 LIS 输出: 13
说明: 1(2) -> 2(4) -> 3(7)。

DP-19: DP 模型拼接食谱

模型编号: DP-19

模型名称: DP 模型拼接食谱

标签: DP、拼接、PrefixSum、树状数组、Tree、Graph、Topo、Digit

一句话用途: 当题目不是单纯 DP, 而是 “DP + 图论/数据结构/数学预处理” 时, 按固定食谱把模块接起来。

题面触发词:

- “区间代价反复出现”
- “从前面满足条件的位置转移”
- “树上选 k 个/容量限制”
- “访问关键点, 原图有边权”
- “依赖关系、有向无环”
- “1..N 计数, 要求取模/余数/组合数量”

什么时候用:

- 已经识别出主 DP 模型, 但转移里的 cost/query/dist/order/count 需要别的模块提供。
- 朴素转移能写出来, 但复杂度差一截。
- 题面同时出现 DP 信号和图论/数据结构/数学信号。

不要什么时候用:

- 单个 DP 模型已经足够, 不要强行拼接。
- 辅助模块会改变问题本质, 例如一般图最长路不能靠 DAG DP 解决。
- 还没写出状态句式, 就先不要急着上数据结构。

复杂度:

总复杂度 = 预处理复杂度 + DP 状态数 * 转移复杂度

数据范围信号:

- $n \leq 300/500$ 且区间: 区间 DP + PrefixSum。
- $n \leq 2e5$ 且前驱范围查询: 线性 DP/LIS + 树状数组/SegmentTree。
- $n \leq 2000$, $K \leq 2000$ 且树上选数量: 树形 DP + 背包合并。
- 关键点 $k \leq 20$, 原图大: 最短路预处理 + 状压 DP。
- 有向无环依赖 $n, m \leq 2e5$: Topo + DAG DP。
- N 很大且数位条件: 数位 DP + 取模/组合。

依赖的标准容器:

- vector
- queue
- priority_queue
- array
- tuple

输入如何整理:

- 先确定主 DP 状态。
- 再问转移里缺什么: 区间和、前驱最值、子树合并、关键点距离、拓扑序、组合数。
- 辅助模块只负责提供这个缺口, 不要把状态含义写乱。

接口:

主 DP 状态句式 + 辅助模块预处理 + 转移骨架 + 答案位置

输出能力:

- 一组可直接复用的拼接套路。
- 每个套路都给 “先做什么, 再做什么” 的考场顺序。

下游可接:

- DP-13 区间 DP
- DP-14 树形 DP
- DP-15 DAG DP
- DP-16 状压 DP
- DP-17 数位 DP
- DP-18 DP + 数据结构优化

可拼接模块：

- DS-01 PrefixSum
- DS-02 树状数组
- GRAPH-03 Dijkstra
- GRAPH-04 Floyd
- GRAPH-05 Topo
- MATH-03 组合数

拼接总原则

先写 DP 状态句式，再写辅助模块负责什么。

不要一上来就写大模板。考场顺序固定为：

1. 主模型：这题的 dp 下标是什么？
2. 缺口：转移里反复需要算什么？
3. 辅助模块：用谁把缺口变成 $O(1)$ 、 $O(\log n)$ 或有序处理？
4. 拼接点：辅助模块的输出喂给哪一行转移？
5. 答案位置：最后从哪个 dp 位置取？

食谱 1：区间 DP + PrefixSum

题面信号：

区间合并、区间删除、合并石子

转移里反复用 $\text{sum}(l, r)$ 或 $\text{cost}(l, r)$

$n \leq 300/500$

拼接顺序：

1. 先预处理 $\text{prefix}[i] = a[1] + \dots + a[i]$
2. 写 $\text{sum}(l, r) = \text{prefix}[r] - \text{prefix}[l-1]$
3. 区间 DP 按 len 从小到大
4. 转移里直接调用 $\text{sum}(l, r)$

状态句式：

$\text{dp}[l][r]$ 表示：合并/处理区间 $[l, r]$ 的最小代价。

代码骨架：

```
const int MAXN = 505;
static ll prefix[MAXN], dp[MAXN][MAXN];

for (int i = 1; i <= n; i++) prefix[i] = prefix[i - 1] + a[i];

auto sum = [&](int l, int r) {
    return prefix[r] - prefix[l - 1];
};

for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        dp[l][r] = LINF;
        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + sum(l, r));
        }
    }
}

cout << dp[1][n] << '\n';
```

常见坑:

- 没有 PrefixSum, 每次 $\text{sum}(l,r)$ 再循环求和, 复杂度多乘一层。
- prefix 是 1-index, $\text{sum}(l,r)$ 要用 $\text{prefix}[l - 1]$ 。
- 区间 DP 必须小区间先算, 大区间后算。

食谱 2: LIS/线性 DP + 树状数组

题面信号:

$n \leq 2e5$

$\text{dp}[i]$ 需要从前面满足 $a[j] < a[i]$ 或 $a[j] \leq a[i]$ 的 j 转移

要求最大权值递增子序列、前缀最优、值域范围最优

拼接顺序:

1. 先写朴素转移: $\text{dp}[i] = \max(\text{dp}[j] + \text{gain})$
2. 把前驱条件改写成 $\text{key}[j]$ 的前缀/区间查询
3. 值域大时坐标压缩
4. 树状数组维护“已经处理过的 key 的最大 dp ”
5. 每个 i 先 query, 再 add

状态句式:

$\text{dp}[i]$ 表示: 以第 i 个元素作为最后一个元素时的最大收益。

树状数组最大值模板:

```
struct BITMax {
    int n;
    vector<ll> bit;
    BITMax(int n = 0) { init(n); }
    void init(int n_) {
        n = n_;
        bit.assign(n + 1, -LINF);
    }
    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) bit[idx] = max(bit[idx], val);
    }
    ll prefix(int idx) {
        ll ans = -LINF;
        for (; idx > 0; idx -= idx & -idx) ans = max(ans, bit[idx]);
        return ans;
    }
};
```

拼接代码:

```
vector<ll> vals;
for (int i = 1; i <= n; i++) vals.push_back(a[i]);
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());

auto pos = [&](ll x) {
    return int(lower_bound(vals.begin(), vals.end(), x) - vals.begin()) + 1;
};

BITMax bit((int)vals.size());
ll ans = 0; // 如果必须选至少一个且权值可能全负, 改成 -LINF
for (int i = 1; i <= n; i++) {
    int p = pos(a[i]);
    ll best = bit.prefix(p - 1); // 严格递增:  $a[j] < a[i]$ 
    ll dp_i = w[i] + max(0LL, best);
    bit.add(p, dp_i);
    ans = max(ans, dp_i);
}
cout << ans << '\n';
```

常见坑:

- 严格递增用 $p - 1$, 不下降用 p 。
- 必须先 query 再 add, 否则会把自己当成前驱。
- 如果空前驱不合法, 不能写 $\max(0LL, best)$, 要按题意处理。
- 如果必须选至少一个且权值可能全负, ans 初始化为 $-LINF$, 每个 dp_i 至少可以等于 $w[i]$ 。

食谱 3: 树形 DP + 背包合并

题面信号:

树上选择 k 个点
 每个子树贡献若干容量/数量
 父亲要合并多个儿子的方案
 $n * K$ 能接受

拼接顺序:

1. 树形 DP 后序 DFS
2. $dp[u][j]$ 表示 u 子树选 j 个的最优值/方案数
3. 每合并一个儿子 v , 就做一次背包合并
4. 用 ndp 防止同一个儿子被重复使用

状态句式:

$dp[u][j]$ 表示: 只考虑 u 的子树, 选 j 个点时的最大收益。

代码骨架:

```
const int MAXN = 2000 + 5;
const int MAXK = 2000 + 5; // dp[MAXN][MAXK] 约 32MB, 按题目内存调小
static ll dp[MAXN][MAXK], val[MAXN];
static int sz[MAXN];

for (int u = 1; u <= n; u++) {
    for (int j = 0; j <= K; j++) dp[u][j] = -LINF;
}

void dfs(int u, int p) {
    sz[u] = 1;
    dp[u][0] = 0;
    if (K >= 1) dp[u][1] = val[u]; // 选择 u

    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u);

        static ll ndp[MAXK];
        for (int x = 0; x <= K; x++) ndp[x] = -LINF;
        for (int i = 0; i <= min(sz[u], K); i++) {
            if (dp[u][i] == -LINF) continue;
            for (int j = 0; j <= min(sz[v], K - i); j++) {
                if (dp[v][j] == -LINF) continue;
                ndp[i + j] = max(ndp[i + j], dp[u][i] + dp[v][j]);
            }
        }
        sz[u] += sz[v];
        for (int i = 0; i <= min(sz[u], K); i++) dp[u][i] = ndp[i];
    }
}

dfs(1, 0);
cout << dp[1][K] << '\n';
```

常见坑:

- 合并儿子时直接写回 $dp[u]$, 导致一个儿子贡献被重复用。
- $sz[u]$ 没维护, 循环跑太大。
- $dp[u][0]$ 忘记初始化为 0, 导致“不选子树点”的状态不可达。

食谱 4: 状压 DP + Floyd/Dijkstra

题面信号:

访问所有关键点

关键点 $k \leq 20$

原图上有边权或障碍

要求最短访问代价

拼接顺序:

1. 找出关键点 $\text{key}[1..k]$
2. 在原图上求关键点两两最短路
 - $n \leq 300$: Floyd
 - 稀疏大图: 从每个关键点跑 Dijkstra
3. 得到 $\text{dist}[i][j]$, 其中 i, j 都是 $1..k$ 的关键点编号
4. 在关键点编号 $1..k$ 上做状压 DP; 第 i 个关键点对应 mask 的第 $i-1$ 位

状态句式:

$\text{dp}[\text{mask}][\text{last}]$ 表示: 已经访问关键点集合 mask , 最后停在 last 时的最小代价。
 last 是 $1..k$ 的关键点编号, 不要改成 0-index。

Floyd 拼接:

```
const int MAXN = 305;
ll d[MAXN][MAXN];
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) d[i][j] = (i == j ? 0 : LINF);
}
for (auto [u, v, w] : edges) {
    d[u][v] = min(d[u][v], w);
    d[v][u] = min(d[v][u], w);
}
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (d[i][k] == LINF || d[k][j] == LINF) continue;
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}

const int KMAX = 20;
ll dist[KMAX + 1][KMAX + 1];
for (int i = 1; i <= k; i++) {
    for (int j = 1; j <= k; j++) {
        dist[i][j] = d[key[i]][key[j]];
    }
}
```

Dijkstra 拼接:

```
const int MAXN = 200000 + 5;
vector<pair<int, ll>> g[MAXN];

void dijkstra(int s, ll one[]) {
    for (int i = 1; i <= n; i++) one[i] = LINF;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    one[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != one[u]) continue;
        for (auto [v, w] : g[u]) {
            if (one[v] > du + w) {
```

```

        one[v] = du + w;
        pq.push({one[v], v});
    }
}
}
}

```

```

const int KMAX = 20;
ll dist[KMAX + 1][KMAX + 1];
ll one[MAXN];
for (int i = 1; i <= k; i++) {
    dijkstra(key[i], one);
    for (int j = 1; j <= k; j++) {
        dist[i][j] = one[key[j]];
    }
}

```

状压拼接:

```

const int SMAX = 1 << KMAX;
ll dp[SMAX][KMAX + 1];

int total = 1 << k;
int full = total - 1;
for (int mask = 0; mask < total; mask++) {
    for (int i = 1; i <= k; i++) dp[mask][i] = LINF;
}
dp[1 << (start - 1)][start] = 0;

for (int mask = 0; mask < total; mask++) {
    for (int last = 1; last <= k; last++) {
        if (dp[mask][last] == LINF) continue;
        for (int nxt = 1; nxt <= k; nxt++) {
            if (mask >> (nxt - 1) & 1) continue;
            if (dist[last][nxt] == LINF) continue;
            int nmask = mask | (1 << (nxt - 1));
            dp[nmask][nxt] = min(dp[nmask][nxt], dp[mask][last] + dist[last][nxt]);
        }
    }
}

```

常见坑:

- 原图点号和关键点编号都保持 1-index; 只有 mask 取位时写 (i - 1)。
- 两个关键点不可达时, dist == LINF 要跳过。
- 1 << k 要确认 k <= 20 左右, 否则内存爆。

食谱 5: DAG DP + Topo

题面信号:

依赖关系

任务先后顺序

有向无环图

路径计数/最长路/最早完成时间

拼接顺序:

1. 建有向图, 确认边方向
2. 拓扑排序
3. 按拓扑序做 dp 转移
4. 如果拓扑点数不足 n, 说明有环, 不能套普通 DAG DP

状态句式:

dp[u] 表示: 到达 u 的最大收益/最短时间/方案数。

代码骨架:

```

const int MAXN = 200000 + 5;
static vector<pair<int,ll>> g[MAXN];
static int indeg[MAXN];
static ll dp[MAXN];

for (auto [u, v, w] : edges) {
    g[u].push_back({v, w});
    indeg[v]++;
}

queue<int> q;
for (int i = 1; i <= n; i++) {
    if (indeg[i] == 0) q.push(i);
}

vector<int> topo;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    topo.push_back(u);
    for (auto [v, w] : g[u]) {
        if (--indeg[v] == 0) q.push(v);
    }
}

if ((int)topo.size() < n) {
    cout << "-1\n"; // 有环, 不能直接用 DAG DP.
    return;
}

for (int i = 1; i <= n; i++) dp[i] = -LINF;
dp[s] = 0;
for (int u : topo) {
    if (dp[u] == -LINF) continue;
    for (auto [v, w] : g[u]) {
        dp[v] = max(dp[v], dp[u] + w);
    }
}
cout << dp[t] << '\n';

```

方案数改法:

```

vector<ll> ways(n + 1, 0);
ways[s] = 1;
for (int u : topo) {
    for (auto [v, w] : g[u]) {
        ways[v] = (ways[v] + ways[u]) % MOD;
    }
}

```

常见坑:

- “u 依赖 v” 时, 边可能应建成 v -> u。
- 有多个起点时, 方案数/最早时间要初始化所有起点。
- 拓扑排序会修改 indeg, 后面还要用原入度就先备份。

食谱 6: 数位 DP + 取模/组合

题面信号:

默认统计 1..N 或 L..R 中的正整数; 如果 0 合法, base case 要单独打开
 数字本身模 M 为某个余数

数位和模 K

恰好有 k 个非零位/某数字出现 k 次

答案对 MOD 取模

拼接顺序:

1. 把 N 拆成 digits
2. 数位 DP 保留 tight/leading
3. 需要整除或余数时, 加 rem 维度
4. 需要出现次数时, 加 cnt/rest 维度
5. 如果 tight=false 后剩余位只和数量有关, 可以用组合数加速; 不会加速就保留 DP

状态句式:

dfs(pos, tight, leading, rem, cnt) 表示: 处理到 pos 位, 当前余数 rem、已用数量 cnt 时的方案数。

取模 DP 骨架:

```
ll dfs(int pos, bool tight, bool leading, int rem) {
    if (pos == len) {
        bool count_zero = false; // 如果 0 是合法数字, 改成 true
        return ((!leading || count_zero) && rem == 0) ? 1 : 0;
    }

    if (!tight && vis[pos][leading][rem]) return memo[pos][leading][rem];

    int up = tight ? digits[pos] : 9;
    ll ans = 0;
    for (int d = 0; d <= up; d++) {
        bool nleading = leading && (d == 0);
        bool ntight = tight && (d == up);
        int nrem = nleading ? 0 : (rem * 10 + d) % M;
        ans = (ans + dfs(pos + 1, ntight, nleading, nrem)) % MOD;
    }

    if (!tight) {
        vis[pos][leading][rem] = 1;
        memo[pos][leading][rem] = ans;
    }
    return ans;
}
```

组合数加速句式:

如果后面还剩 len 位, 且只要求 “恰好再放 need 个非零数字”, 数量可以是 $C[\text{len}][\text{need}] * 9^{\text{need}}$ 。

组合数预处理:

```
const int MAXL = 105;
static ll C[MAXL][MAXL];

for (int i = 0; i <= maxLen; i++) {
    C[i][0] = C[i][i] = 1;
    for (int j = 1; j < i; j++) {
        C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % MOD;
    }
}
```

常见坑:

- rem 是数字本身的余数时, 更新应是 $(\text{rem} * 10 + d) \% M$, 不是 $(\text{rem} + d) \% M$ 。
- 数位和余数才写 $(\text{rem} + d) \% M$ 。
- leading 时是否更新 rem/cnt 要按题意决定。
- tight=true 的状态不要随便缓存。
- 区间 $[L, R]$ 用 $\text{solve}(R) - \text{solve}(L-1)$, 取模时要加回 MOD。

拼接检查卡

1. 主 DP 状态是否已经写清楚?
2. 辅助模块的输出是什么变量?
3. 这个变量进入哪一行转移?

4. 预处理下标和 DP 下标是否一致？
5. 复杂度是否是“预处理 + DP”，没有暗中多一层循环？

暴力/部分替代：

- 拼接写不出来时，先交朴素 DP：区间和暴力、前驱枚举、DFS 路径枚举。
- 图上关键点距离不会预处理时，先只处理完全图输入的版本。
- 数位 DP 想不清组合加速时，保留完整 cnt/rem 记忆化。

升级方向：

- 区间转移代价重复计算 -> PrefixSum。
- 线性 DP 前驱枚举 -> 树状数组/SegmentTree。
- 树上多个子树数量分配 -> 背包合并。
- 原图访问关键点 -> 最短路预处理 + 状压。
- 依赖图 -> Topo 后 DP。
- 数位范围计数 -> tight/leading + mod/cnt/comb。

最小测试样例：

检查任意拼接题时，先写这三行：

主状态：dp[...]

辅助模块：负责提供 ...

转移拼接点：dp[...] = merge(dp[...], 辅助模块输出 + ...)

DP-20: 计数 / 可行性 DP

模型编号：DP-20

模型名称：计数 / 可行性 DP

标签：DP、方案数、可行性、取模、初始化、判定

一句话用途：把“有多少种方案”和“是否存在方案”从最优值 DP 中单独拆出来，避免初值、转移和取模混用。

题面触发词：

- “方案数” “有多少种”
- “对 $1e9+7$ 取模”
- “是否存在” “能否凑出”
- “可不可以” “是否可达”
- “恰好” “至少” “至多”

什么时候用：

- dp 不是最大/最小值，而是数量或真假。
- 背包、网格、DAG、数位、区间里问方案数。
- 题目同时要求“是否存在”和“方案数”。

不要什么时候用：

- 明确要求最大收益/最小代价时，先按最优值 DP 写。
- 方案数可能巨大但题目没有要求取模时，要确认是否需要高精度或 long long。
- 只问最短路条数时，先路由到最短路，再在最短路路上计数。

复杂度：

状态数 * 转移数

计数/可行性不会改变状态数量，只改变 dp 的含义和 merge 方式。

数据范围信号：

- 容量 $W \leq 1e5$ ：一维背包计数/可行性常见。
- 网格 $n*m \leq 1e6$ ：路径计数/可达性常见。
- DAG $n,m \leq 2e5$ ：拓扑计数/可达性常见。
- 数位上界很大：数位 DP 计数。

依赖的标准容器：

- vector<ll>：方案数。

- `vector<char>` 或 `vector<int>`: 可行性。
- `vector<vector<ll>>`、`vector<vector<char>>`。

输入如何整理:

- 先确认目标是“计数”还是“可行性”。
- 再确认要求是“恰好达到”还是“不超过/任意合法”。
- 如果有取模, 统一写 MOD 和加法函数。

接口:

```
const ll MOD = 1000000007LL;
```

输出能力:

- 方案数。
- 是否存在合法方案。
- 同时维护可行性和方案数, 避免取模后误判。

下游可接:

- DP-05 选/不选 DP
- DP-06/07/08 背包
- DP-12 网格 DP
- DP-15 DAG DP
- DP-17 数位 DP
- DP-19 拼接食谱

可拼接模块:

- Math 取模。
- Topo。
- PrefixSum。

计数和可行性的本质区别

类型	dp 含义	初始值	合并方式	答案判断
计数 DP	有多少种方案到达状态	0	加法	输出数量
可行性 DP	状态是否能到达	false	或运算	true/false
最大值 DP	到达状态的最大收益	-LINF	max	最大值
最小值 DP	到达状态的最小代价	LINF	min	最小值

最重要的初始化:

计数: 起点 `dp[start] = 1`, 表示“空方案”有 1 种。

可行性: 起点 `ok[start] = true`, 表示起点可达。

其他状态: 计数为 0, 可行性为 false。

从暴力到计数/可行性

计数和可行性 DP 的动机不是“换一套模板”, 而是把暴力搜索的返回值换掉:

暴力问最优值: 所有选择里取 max/min。

计数问方案数: 所有能到达的前驱方案数相加。

可行性问存在: 所有能到达的前驱做 OR。

最后一步视角:

背包计数: 最后一个动作是“选第 i 个物品”, 于是从 $j-w[i]$ 来, `ways[j] += ways[j-w[i]]`。

网格计数: 最后一步从上或左走入 (i, j) , 于是 `ways[i][j] = ways[i-1][j] + ways[i][j-1]`。

DAG 计数: 最后一条边是 $u \rightarrow v$, 于是 `ways[v] += ways[u]`。

可行性: 同样的前驱关系, 把加法换成 `ok[v] |= ok[u]`。

取模套路

```
using ll = long long;
const ll MOD = 1000000007LL;
```

```
void add_mod(ll& a, ll b) {
```

```

    b %= MOD;
    a += b;
    if (a >= MOD) a -= MOD;
}

```

如果一次可能加很多或有乘法：

```

a = (a + b) % MOD;
c = a * b % MOD; // a, b 已经在 MOD 内时 long long 足够装 1e18 以内

```

减法取模：

```

ans = (solve(R) - solve(L - 1) + MOD) % MOD;

```

常见坑：

- 计数题忘记每次加法后取模。
- 可行性题不需要取模。
- `dp[target] == 0` 只表示“方案数模 MOD 为 0”，不一定表示没有方案。若题目还问是否存在，要另开 `ok`。

判定套路

计数题：

输出 `dp[target]`。

可行性题：

```

if (ok[target]) cout << "Yes\n";
else cout << "No\n";

```

同时要数量和是否存在：

```

vector<ll> ways(W + 1, 0);
vector<char> ok(W + 1, 0);
ways[0] = 1;
ok[0] = 1;

```

// 转移时 `ways` 做加法, `ok` 做或运算。

不要用取模后的 `ways[target] != 0` 代替可行性。

套路 1: 0/1 背包方案数

题面信号：

每个数最多用一次

问凑出恰好 `W` 的方案数

状态句式：

`dp[j]` 表示：当前已处理若干物品，凑出和 `j` 的方案数。

初始化：

`dp[0] = 1`：什么都不选，凑出 0，有 1 种方案。

其他 `dp[j] = 0`：暂时没有方案。

转移：

```

vector<ll> dp(W + 1, 0);
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = W; j >= a[i]; j--) {
        dp[j] = (dp[j] + dp[j - a[i]]) % MOD;
    }
}
cout << dp[W] << '\n';

```

循环方向原因：

0/1 背包倒序，防止同一个物品在本轮被重复使用。

套路 2：完全背包方案数

题面信号：

每种物品可以用无限次

问凑出 W 的方案数

转移：

```
vector<ll> dp(W + 1, 0);
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = a[i]; j <= W; j++) {
        dp[j] = (dp[j] + dp[j - a[i]]) % MOD;
    }
}
```

循环方向原因：

完全背包正序，允许当前物品在本轮被继续使用。

注意：

上面写法统计“组合数”：物品顺序不重要。

如果题目把不同排列也算不同方案，外层通常改成 j ，内层枚举物品。

排列计数写法：

```
vector<ll> dp(W + 1, 0);
dp[0] = 1;
for (int j = 1; j <= W; j++) {
    for (int i = 1; i <= n; i++) {
        if (j >= a[i]) dp[j] = (dp[j] + dp[j - a[i]]) % MOD;
    }
}
```

套路 3：可行性背包

题面信号：

能否凑出 W

是否存在子集满足条件

状态句式：

$ok[j]$ 表示：当前已处理若干物品，是否能凑出 j 。

0/1 版本：

```
vector<char> ok(W + 1, 0);
ok[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = W; j >= a[i]; j--) {
        ok[j] = ok[j] || ok[j - a[i]];
    }
}
```

```
cout << (ok[W] ? "Yes\n" : "No\n");
```

完全背包版本：

```
vector<char> ok(W + 1, 0);
ok[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = a[i]; j <= W; j++) {
        ok[j] = ok[j] || ok[j - a[i]];
    }
}
```

常见坑：

- `vector<bool>` 有特殊压缩行为，初学者更推荐 `vector<char>`。
- `ok[0]=true` 不是说选了东西，而是空方案可达。

套路 4：分组计数 / 可行性

题面信号：

每组最多选一个

从每组中选 0 或 1 个

计数版本：

```
vector<ll> dp(W + 1, 0);
dp[0] = 1;
for (const auto &group : groups) {
    vector<ll> ndp = dp; // 本组一个也不选
    for (auto item : group) {
        int cost = item.first; // group: vector<pair<int, ll>>, first 是费用
        for (int j = cost; j <= W; j++) {
            ndp[j] = (ndp[j] + dp[j - cost]) % MOD;
        }
    }
    dp.swap(ndp);
}
```

可行性版本：

```
vector<char> ok(W + 1, 0);
ok[0] = 1;
for (const auto &group : groups) {
    vector<char> nok = ok; // 本组不选
    for (auto item : group) {
        int cost = item.first; // group: vector<pair<int, ll>>, first 是费用
        for (int j = cost; j <= W; j++) {
            if (ok[j - cost]) nok[j] = 1;
        }
    }
    ok.swap(nok);
}
```

常见坑：

- 分组题必须从上一组的 dp 转到新组 ndp。
- 直接在同一个 dp 上更新，可能一组里选了多个物品。

套路 5：网格路径计数 / 可达性

题面信号：

只能向右/下走

障碍格

问路径条数或能否到达

计数：

```
vector<vector<ll>> dp(n + 1, vector<ll>(m + 1, 0));
if (!blocked[1][1]) dp[1][1] = 1;

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (blocked[i][j]) {
            dp[i][j] = 0;
            continue;
        }
        if (i == 1 && j == 1) continue;
        dp[i][j] = ((i > 1 ? dp[i - 1][j] : 0) + (j > 1 ? dp[i][j - 1] : 0)) % MOD;
    }
}
```

可达性：

```
vector<vector<char>> ok(n + 1, vector<char>(m + 1, 0));
if (!blocked[1][1]) ok[1][1] = 1;
```

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (blocked[i][j]) continue;
        if (i > 1) ok[i][j] = ok[i][j] || ok[i - 1][j];
        if (j > 1) ok[i][j] = ok[i][j] || ok[i][j - 1];
    }
}

```

常见坑:

- 起点或终点是障碍时答案直接为 0/No。
- 初始化第一行第一列时，障碍后面不能继续继承。

套路 6: DAG 路径计数

题面信号:

有向无环图

从 s 到 t 有多少条路径

依赖关系方案数

状态句式:

$ways[u]$ 表示: 从 s 到 u 的路径数量。

转移:

```

vector<ll> ways(n + 1, 0);
ways[s] = 1;
for (int u : topo) {
    for (auto [v, w] : g[u]) {
        ways[v] = (ways[v] + ways[u]) % MOD;
    }
}
cout << ways[t] << '\n';

```

可达性:

```

vector<char> ok(n + 1, 0);
ok[s] = 1;
for (int u : topo) {
    if (!ok[u]) continue;
    for (auto [v, w] : g[u]) ok[v] = 1;
}

```

常见坑:

- 没有拓扑序，普通图上直接这么计数会因为环出错。
- 多个起点时要把所有起点 $ways[x]=1$ 或 $ok[x]=1$ 。

套路 7: 数位 DP 计数

题面信号:

统计 $1..N$ 中满足数位条件的数

答案取模

状态句式:

$dfs(pos, tight, leading, state)$ 返回当前状态后面能组成的合法数量。

返回边界:

```

if (pos == len) {
    return legal(state, leading) ? 1 : 0;
}

```

计数转移:

```

ll ans = 0;
for (int d = 0; d <= up; d++) {

```

```

    ans = (ans + dfs(pos + 1, ntight, nleading, nstate)) % MOD;
}

```

常见坑:

- 终点返回的是 1 或 0, 不是最优值。
- [L,R] 答案做减法时要补 MOD。
- 前导零是否算合法数字, 要按题面决定。

初始化总表

场景	初始化
计数, 空方案合法	dp[0] = 1
计数, 起点为 s	ways[s] = 1
可行性, 空状态可达	ok[0] = true
可行性, 起点为 s	ok[s] = true
恰好选 0 个	dp[0][0] = 1 或 ok[0][0] = true
其他状态	计数为 0, 可行性为 false

转移总表

类型	merge 写法
计数	dp[to] = (dp[to] + dp[from]) % MOD
可行性	ok[to] = ok[to] ok[from]
计数带权选择	dp[next] += dp[cur], 权值只影响 next
最优值	max/min, 不要和计数加法混用

答案位置总表

题面	答案
恰好凑出 W	dp[W] / ok[W]
不超过 W 任意合法	sum(dp[0..W]) 或 any(ok[0..W])
恰好选 K 个	dp[K] / ok[K]
网格到终点	dp[n][m] / ok[n][m]
DAG 从 s 到 t	ways[t] / ok[t]
数位范围	solve(R) - solve(L - 1)

常见坑:

- “至多 W” 却只输出 dp[W]。
- “恰好 W” 却把 dp[0..W] 全加了。
- 方案数题把初值设为 -LINF。
- 可行性题把 ok 默认成 true。
- 组合数和排列数循环顺序写反。

暴力/部分分替代:

- 小数据先 DFS 枚举所有选择, 遇到合法方案 ans++ 或设置 found=true。
- DFS 能重复到同一状态时, 把返回值从最优值改成 “方案数/是否可行”。
- 背包循环方向拿不准时, 先写二维 dp[i][j], 确认后再压成一维。

升级方向:

- 计数 DFS -> 记忆化计数 -> 表推计数。
- 可行性 DFS -> ok 表。
- 二维背包 -> 一维滚动。
- DAG DFS 计数 -> Topo 计数。
- 数位暴力枚举 -> 数位 DP 计数。

最小测试样例:

物品: 1 2 3, 目标 $W=3$

0/1 方案数: 2

方案: {3}, {1,2}

0/1 可行性: Yes

完全背包组合方案数: 3

方案: {1+1+1}, {1+2}, {3}

DP-21: P1874 快速求和建模例题

模块编号: DP-21

模块名称: 从暴力切分到 DP 建模: P1874 快速求和

标签: DP、建模过程、字符串切分、最小加号数、暴力到表推

一句话用途: 用一道完整例题演示 “为什么这样定义状态”, 帮助初学者从暴力搜索自然推到 DP。

题面触发词:

- 数字字符串中插入加号。
- 让表达式结果等于目标数。
- 求最少加号数量。
- 字符串长度不大但所有切法是指数级。
- 前导零不影响数字大小。

什么时候用:

- 题目是在序列/字符串中切分若干段, 并让段值的和、代价或方案数满足目标。
- 暴力枚举每个空隙切或不切, 复杂度是 $2^{(len-1)}$ 。
- 后续是否能成功只取决于 “处理到的位置” 和 “当前累计值”, 不关心前面具体怎么切。

不要什么时候用:

- 段值可以为负, 或后续操作能让和变小, 此时 $sum > target$ 不能直接剪枝。
- 目标值很大到 $dp[len][target]$ 开不下, 需要改记忆化搜索、map 状态或其他优化。
- 每一段的贡献不只是段值, 还依赖前一段形态, 此时需要额外状态, 例如 last。

复杂度:

- 状态数: $O(len * target)$ 。
- 转移: 枚举下一段, 整体约 $O(len^2 * target)$ 。
- 本题 $len \leq 40$ 、 $target \leq 1e5$, 约 $1.6e8$ 级别简单整数操作, C++ 可接受。
- 空间: $(len + 1) * (target + 1)$ 个 int, 约 16MB。

依赖的标准容器:

- string: 数字串, 0-index。
- static int dp[MAXL][MAXT]: DP 表, 竞赛速写更稳。
- long long: 临时解析子串值, 防止中间乘 10 溢出; 本题会在 target 以上立即停止。

输入如何整理:

```
string s;
int target;
cin >> s >> target;
int len = (int)s.size();
```

接口:

solve_p1874(s, target) -> 最少加号数量, 不可达返回 -1

输出能力:

- 输出最少加号数。
- 不可达时输出 -1。

下游可接:

- DP-03 DFS -> 记忆化 -> 表推升级图。
- DP-03B 状态增维路由。
- DP-20 计数/可行性 DP, 如果目标从“最少加号”改成“方案数/是否存在”。
- DP-25 暴力 DFS 到记忆化搜索: 如果考场上先写出 `dfs(pos, sum)`, 直接加 memo 也能得到同一批状态。

可拼接模块:

- CPP-011 string: 子串和字符处理。
- BRUTE-07: 如果先写 DFS, 可加 memo。
- DP-04: 线性前缀 DP 思路。

题意压缩

给定一个数字字符串 `s` 和整数 `target`。可以在相邻字符之间插入若干个 `+`, 把字符串拆成若干个非负整数段。前导零不影响值, 例如 `030` 的值是 `30`。要求所有段相加等于 `target`, 并输出最少需要插入多少个加号; 如果无论如何都不能等于 `target`, 输出 `-1`。

样例:

99999

45

输出:

4

解释: $9+9+9+9+9=45$, 需要 4 个加号。

第一阶段: 先写暴力, 找决策点

长度为 `len` 的字符串有 `len - 1` 个空隙。每个空隙只有两种选择:

切: 插入一个 `+`

不切: 继续把后面的数字拼到当前段里

所以朴素暴力是:

枚举每个空隙是否切开

计算每种表达式的和

取能等于 `target` 的最少加号数

复杂度是 $O(2^{(len-1)})$ 。本题 `len <= 40`, 最大约 2^{39} , 远远超过考场可承受范围。

结论: 暴力可以帮助理解题目, 但必须找重复状态, 用记忆化或 DP 降复杂度。

第二阶段: 找重叠子问题

从左到右处理字符串。假设已经处理完前 `i` 个字符, 并且前面切出来的段之和是 `sum`。

这时后续还没处理的部分只关心两件事:

1. 现在处理到哪个位置 `i`
2. 当前累计和是多少 `sum`

它不关心前面到底是:

`1 + 23`

`12 + 3`

还是其他切法。只要位置和累计和相同, 后续能不能凑到 `target`、还需要多少加号, 都是同一个子问题。

这就是 DP 的信号:

历史具体路径不重要, 只保留会影响未来的摘要信息。

第三阶段: 定义状态并验可行性

自然状态:

`dp[i][sum]` = 把 `s` 的前 `i` 个字符切成若干段, 段和等于 `sum` 时, 最少需要多少个加号

其中:

- `i` 表示已经用掉前 `i` 个字符。
- `sum` 表示这些段的总和。
- `dp` 值表示优化目标: 最少加号数。

可行性检查：

```
i: 0..40
sum: 0..target, target <= 100000
状态数约 41 * 100001
int 表约 16MB
```

空间可行。因为每段都是非负数，如果当前和已经超过 target，后面再加也不会变小，所以 sum 只需要开到 target。

第四阶段：推导转移

看最后一段怎么来。

假设用数学上的 1-index 位置描述，最后一段是 $s[start..end]$ ，它的值是 val。在加上这一段之前，已经处理完前 $start - 1$ 个字符，累计和是 sum。

于是：

```
前面状态: dp[start - 1][sum]
当前段值: val
新状态: dp[end][sum + val]
```

落到 C++ 代码里，字符串本身仍然是 0-index：如果当前已经处理完前 i 个字符，下一段从 $s[i]$ 开始，到 $s[j]$ 结束，新状态就是 $dp[j + 1][sum + val]$ 。

每切出一个新段，段数加一。加号数 = 段数 - 1。

最直观的写法是：

```
dp[0][0] = 0
其他状态 = INF
```

当从位置 i 新开一段 $s[i..j]$ 时：

```
int add = (i == 0 ? 0 : 1);
dp[j + 1][sum + val] = min(dp[j + 1][sum + val], dp[i][sum] + add);
```

解释：

- 如果 $i == 0$ ，这是第一段，前面没有加号，所以加 0。
- 如果 $i > 0$ ，这段前面必须插入一个加号，所以加 1。

这个版本最贴近题目问法：dp 表里直接存“最少加号数”，不需要最后再减一。

第五阶段：细节与陷阱

1. 大数字剪枝：

子串长度可能很长，不能直接 `stoll`。边枚举边计算 $val = val * 10 + digit$ ，一旦 $sum + val > target$ 就停止扩展这一段。

2. 前导零：

题目说前导零不影响大小。逐位计算天然支持：“003”会得到 3。

3. 不可达：

$dp[len][target]$ 仍是 INF 时输出 -1。

4. 加号数量：

不要把段数当答案。 $a+b+c$ 有 3 段但只有 2 个加号。

5. 目标为正：

本题 $target \geq 1$ 。如果遇到目标可为 0 的变体，也能用同一代码；只是注意全零字符串会有很多 0 段。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;
const int MAXL = 45;
const int MAXT = 100000 + 5;
static int dp[MAXL][MAXT];
```

```

int solve_p1874(const string &s, int target) {
    int len = (int)s.size();

    for (int i = 0; i <= len; i++) {
        for (int sum = 0; sum <= target; sum++) {
            dp[i][sum] = INF;
        }
    }
    dp[0][0] = 0;

    for (int i = 0; i < len; i++) {
        for (int sum = 0; sum <= target; sum++) {
            if (dp[i][sum] == INF) continue;

            long long val = 0;
            for (int j = i; j < len; j++) {
                val = val * 10 + (s[j] - '0');
                if (sum + val > target) break;

                int add = (i == 0 ? 0 : 1);
                int ns = (int)(sum + val);
                dp[j + 1][ns] = min(dp[j + 1][ns], dp[i][sum] + add);
            }
        }
    }

    if (dp[len][target] == INF) return -1;
    return dp[len][target];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    int target;
    cin >> s >> target;

    cout << solve_p1874(s, target) << '\n';
    return 0;
}

```

调用示例:

```

cout << solve_p1874("99999", 45) << '\n'; // 4
cout << solve_p1874("12", 3) << '\n'; // 1: 1+2
cout << solve_p1874("303", 6) << '\n'; // 1: 3+03
cout << solve_p1874("0003", 3) << '\n'; // 0: 整段 0003

```

常见坑:

- 用 `stoll(s.substr(...))`: 长子串可能越界或抛异常, 不适合考场稳写。
- 忘记前导零: `303 -> 3+03` 是合法的。
- 每开一段都无脑 +1, 会把第一段前面也算一个加号; 要写 `i == 0 ? 0 : 1`。
- 只初始化 `dp[i][整段值] = 0`, 却忘记从 `dp[0][0]` 或前缀状态统一转移, 容易漏情况。
- `sum + v` 越界: 循环条件写成 `sum + v <= target`。
- `sum + val > target` 后继续扩展: 既浪费时间, 也可能让 `val` 变大后溢出。

暴力/部分分替代:

本题非常适合从暴力 DFS 升级成记忆化搜索。纯 DFS 只适合小数据; 若给 `dfs(pos, sum)` 加 `memo/vis`, 每个位置和累计和只计算一次, 复杂度会降到 $O(\text{len}^2 * \text{target})$ 。完整可抄代码见 DP-25。

```

#include <bits/stdc++.h>
using namespace std;

```

```

const int INF = 1000000000;

string s;
int target_value;
int len;
int best_answer;

void dfs_bruteforce(int pos, long long sum, int plus_count) {
    if (sum > target_value) return;
    if (plus_count >= best_answer) return;

    if (pos == len) {
        if (sum == target_value) best_answer = min(best_answer, plus_count);
        return;
    }

    long long val = 0;
    for (int end = pos; end < len; end++) {
        val = val * 10 + (s[end] - '0');
        if (sum + val > target_value) break;
        int add = (pos == 0 ? 0 : 1);
        dfs_bruteforce(end + 1, sum + val, plus_count + add);
    }
}

int solve_fast_sum_bruteforce(const string& input, int target) {
    s = input;
    target_value = target;
    len = (int)s.size();
    best_answer = INF;
    dfs_bruteforce(0, 0, 0);
    return best_answer == INF ? -1 : best_answer;
}

```

这个 DFS 可以过很小数据。若改成 `memo[pos][sum]` = 当前位置和当前累计和下最少还要多少加号，就是 DP-25 的记忆化版本。

最小测试样例：

输入：
99999

45
输出：
4

输入：
12
12
输出：
0

输入：
12
3
输出：
1

输入：
303
6
输出：
1

输入：
0003

3

输出:

0

输入:

111

100

输出:

-1

建模口令:

先问: 我处理到哪里?

再问: 未来还需要知道什么?

本题答案: 处理到 `start/end`, 未来只需要知道当前 `sum`。

所以状态是 `dp[i][sum]`, 值是最少加号数。

DP-22: 编辑距离建模例题

模块编号: DP-22

模块名称: 从双序列匹配到 DP 建模: 编辑距离

标签: DP、双序列、编辑距离、Levenshtein、插入、删除、替换、建模过程

一句话用途: 用编辑距离完整演示“双序列 DP”的建模过程: 为什么状态是 `dp[i][j]`, 为什么最后一步只有三类操作。

题面触发词:

- 把字符串 A 变成字符串 B。
- 每次可以删除一个字符、插入一个字符、替换一个字符。
- 求最少操作次数。
- 两个字符串长度都在几千以内。
- 字符串相似度、拼写纠错、最小编辑次数。

什么时候用:

- 操作对象是两个字符串或两个序列。
- 当前选择只影响末尾/当前位置, 不需要记住更久的历史。
- 目标是求最小操作次数。
- $|A| * |B|$ 的二维表能开下。

不要什么时候用:

- 只允许删除, 常常可以转成 LCS。
- 操作代价不同, 可以用编辑距离框架, 但转移里的 +1 要改成对应代价。
- 操作带复杂限制, 例如不能连续替换、某些字符不能删, 需要额外状态。
- 字符串长度到 $1e5$ 级别, 普通 $O(nm)$ 编辑距离不可直接做。

复杂度:

- 时间: $O(nm)$ 。
- 空间: $O(nm)$; 若只要答案, 可滚动为 $O(m)$ 。
- 本题长度小于等于 2000, 状态数约 $4e6$, 二维 `int` 表约 16MB, 可行。

依赖的标准容器:

- `string a, b`。
- `static int dp[MAXN][MAXN]`, 题目长度上限明确时比嵌套 `vector` 更适合速写。

输入如何整理:

```
string a, b;
cin >> a >> b;
int n = (int)a.size();
int m = (int)b.size();
```

接口:

`edit_distance(a, b)` -> 把 a 变成 b 的最少编辑次数

输出能力：

- 最少插入/删除/替换次数。
- 可扩展为恢复一条操作路径。

下游可接：

- DP-09 LCS：只允许插入/删除时常与 LCS 相关。
- DP-02 状态句式库：双序列前缀句式。
- DP-20 计数/可行性 DP：若题目改成“是否能在 k 步内完成”。
- DP-25 暴力 DFS 到记忆化搜索：如果先写出 $\text{dfs}(i, j)$ ，加 memo 后就是同一批二维状态。

可拼接模块：

- CPP-011 string。
- 滚动数组优化。
- 路径恢复。

题意压缩

给定两个只含小写字母的字符串 A 和 B。每次可以执行三种操作之一：

删除 A 中的一个字符

在 A 中插入一个字符

把 A 中的一个字符替换成另一个字符

求把 A 变成 B 的最少操作次数。

样例：

sfdqxbw

gfdgw

输出：

4

第一阶段：从暴力尝试开始，找决策点

如果完全不用 DP，面对 A 和 B 的末尾字符，我们大概会递归尝试：

如果末尾相同：两个末尾都跳过

如果末尾不同：

尝试删除 A 的末尾

尝试在 A 末尾插入 B 的末尾

尝试把 A 的末尾替换成 B 的末尾

这个暴力会不断遇到相同的子问题。例如很多操作序列最后都会变成：

把 A 的前 i 个字符变成 B 的前 j 个字符

只要 i 和 j 一样，后续最优答案就一样，不关心之前到底怎么删、插、替换过。

结论：有重叠子问题，可以记忆化；因为依赖顺序很清楚，也可以表推 DP。

第二阶段：找到无后效性

当我们讨论“把 $A[1..i]$ 变成 $B[1..j]$ ”时，未来只关心两个进度：

A 已经考虑到前 i 个字符

B 已经考虑到前 j 个字符

前面具体操作历史不重要。比如你是先删再替，还是先替再插，只要最后剩下的是同一个前缀转换问题，后续最少代价相同。

这就是双序列 DP 的典型信号：

两个对象各有一个进度，下标就是 i 和 j。

第三阶段：定义状态并验可行性

状态定义：

$\text{dp}[i][j]$ = 把 A 的前 i 个字符变成 B 的前 j 个字符，最少需要多少次操作

注意：

- i 和 j 表示前缀长度，不是字符下标。
- 访问字符时用 $a[i - 1]$ 和 $b[j - 1]$ 。
- dp 值是最少操作次数。

可行性检查：

$n, m \leq 2000$

状态数 = $(n + 1) * (m + 1)$ 约 $4e6$

每个状态 $O(1)$ 转移

时间 $O(nm)$ ，空间 $O(nm)$

这个规模对 C++17 很稳。

第四阶段：倒推最后一步

站在状态 $dp[i][j]$ ，观察最后一个字符：

A 的最后字符： $a[i - 1]$

B 的最后字符： $b[j - 1]$

情况一：最后字符相等

如果 $a[i - 1] == b[j - 1]$ ，最后一个字符已经匹配，不需要操作。问题缩小成：

把 A 的前 $i-1$ 个字符变成 B 的前 $j-1$ 个字符

转移：

$dp[i][j] = dp[i - 1][j - 1];$

情况二：最后字符不相等

必须从三种操作中选一个。

1. 删除 A 的最后字符

先把 $A[1..i-1]$ 变成 $B[1..j]$ ，再删除多出来的 $A[i]$ 。

$dp[i - 1][j] + 1$

2. 插入 B 的最后字符

先把 $A[1..i]$ 变成 $B[1..j-1]$ ，再在末尾插入 $B[j]$ 。

$dp[i][j - 1] + 1$

3. 替换最后字符

先把 $A[1..i-1]$ 变成 $B[1..j-1]$ ，再把 $A[i]$ 替换成 $B[j]$ 。

$dp[i - 1][j - 1] + 1$

取最小：

$dp[i][j] = 1 + \min(\{dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]\});$

第五阶段：初始化

必须处理空串。

$dp[0][0] = 0$

$dp[i][0] = i$ ：把 A 的前 i 个字符变成空串，只能删 i 次

$dp[0][j] = j$ ：把空串变成 B 的前 j 个字符，只能插入 j 次

这一步不是形式主义。因为主转移会访问 $i-1$ 和 $j-1$ ，第 0 行和第 0 列就是所有递推的地基。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 2000 + 5;
static int dp[MAXN][MAXN];

int edit_distance(const string &a, const string &b) {
```

```

int n = (int)a.size();
int m = (int)b.size();

for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int j = 0; j <= m; j++) dp[0][j] = j;

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (a[i - 1] == b[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        } else {
            dp[i][j] = 1 + min({
                dp[i - 1][j],      // 删除 a[i-1]
                dp[i][j - 1],      // 插入 b[j-1]
                dp[i - 1][j - 1]   // 替换 a[i-1] 为 b[j-1]
            });
        }
    }
}

return dp[n][m];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    cin >> a >> b;

    cout << edit_distance(a, b) << '\n';
    return 0;
}

```

调用示例:

```

cout << edit_distance("sfdqxbw", "gfdgw") << '\n'; // 4
cout << edit_distance("abc", "abc") << '\n';        // 0
cout << edit_distance("", "abc") << '\n';           // 3

```

从 P1874 到编辑距离: DP 五步法

步骤	P1874 快速求和	编辑距离
识别对象	一个字符串, 切成数字段	两个字符串, 互相匹配
进度维度	用到前 i 个字符	A 前 i 个, B 前 j 个
额外约束	当前和 sum	无额外约束
dp 值	最少加号数	最少操作次数
最后一步	最后一段从哪里开始	最后字符相等/删/插/替
初始化	dp[0][0]=0	空串行列
复杂度	$O(\text{len}^2 * \text{target})$	$O(nm)$

通用心法:

1. 画坐标系: 一个进度、两个进度、区间、树、集合?
2. 写状态句: dp[下标] 表示在什么前提下, 最优值是什么。
3. 倒问最后一步: 最后切在哪里? 最后字符怎么处理? 最后选了谁?
4. 处理空状态: 0 个字符、空区间、空集合、容量 0。
5. 估状态数 * 转移数, 确认能跑。

常见坑:

- dp[i][j] 的 i/j 是前缀长度, 访问字符要用 i-1/j-1。
- 第一行/第一列不初始化, 样例可能过, 大数据会错。
- 字符相等时还加 1, 会把无需操作的匹配算错。

- 插入和删除的解释容易混，但公式要记住：删除看 $dp[i-1][j]$ ，插入看 $dp[i][j-1]$ 。
- $n, m=2000$ 时二维表可开；如果更大，需要滚动数组或其他算法。

暴力/部分分替代：

如果没能马上写出二维表，也可以先顺着“末尾/当前位置怎么处理”写 DFS。纯 DFS 是指数级；一旦给 $dfs(i, j)$ 加 $memo/vis$ ，状态数立刻变成 $n*m$ ，本题规模下通常可以直接满分。完整可抄记忆化代码见 DP-25。

```
#include <bits/stdc++.h>
using namespace std;

string a, b;
int n, m;

int dfs_edit_bruteforce(int i, int j) {
    if (i == n) return m - j;
    if (j == m) return n - i;

    if (a[i] == b[j]) return dfs_edit_bruteforce(i + 1, j + 1);

    int del = dfs_edit_bruteforce(i + 1, j);
    int ins = dfs_edit_bruteforce(i, j + 1);
    int rep = dfs_edit_bruteforce(i + 1, j + 1);
    return 1 + min({del, ins, rep});
}

int solve_edit_distance_bruteforce(const string& x, const string& y) {
    a = x;
    b = y;
    n = (int)a.size();
    m = (int)b.size();
    return dfs_edit_bruteforce(0, 0);
}
```

小数据可以先写这个 DFS。加上 $memo[i][j]$ 后，就变成表推 DP 的同一批状态。

最小测试样例：

输入：
sfdqxbw
gfdgw
输出：
4

输入：
abc
abc
输出：
0

输入：
a
b
输出：
1

输入：
abc
def
输出：
3

DP-23: LIS/LCS 常见变体速查

模型编号: DP-23

模型名称: LIS/LCS 常见变体速查

标签: DP、LIS、LCS、子序列、路径恢复、变体路由

一句话用途: 把“最长递增子序列”和“最长公共子序列”的常见变体集中到一张卡, 考场上先判别变体, 再抄对应短模板。

题面触发词:

- “最长递增/上升/不下降子序列”
- “最长公共子序列/公共子串”
- “删除最少字符使序列递增/两个串相同”
- “输出一个方案”
- “有多少个最长方案”
- “二维偏序、信封嵌套、最多能选多少个”

什么时候用:

- 题目要求保持原相对顺序, 但不要求连续。
- 一个序列内部比较大小, 优先看 LIS。
- 两个序列之间做公共匹配, 优先看 LCS。
- 需要输出路径、统计方案数或处理重复值时, 先查本卡的变体表。

不要什么时候用:

- 要求连续子段/子串, 不能直接套 LIS/LCS。
- 允许插入、删除、替换三种操作并问最少次数, 优先看编辑距离。
- 有容量、预算、次数限制时, 可能要和背包/线性 DP 拼接。
- 需要在线修改序列, 普通 LIS/LCS 表不适合直接维护。

复杂度:

- 朴素 LIS: $O(n^2)$ 。
- 二分 LIS: $O(n \log n)$ 。
- 普通 LCS: $O(nm)$ 。
- LCS 滚动数组: $O(nm)$ 时间, $O(m)$ 空间。
- 计数/带权 LIS: 常用树状数组/SegmentTree, $O(n \log n)$ 。

数据范围信号:

- $n \leq 5000$: 朴素 LIS 或 LCS 常可交。
- $n \leq 2e5$ 且只求 LIS 长度: 二分优化。
- $n, m \leq 3000/5000$: 普通 LCS。
- $n, m \geq 1e5$: 普通 LCS 大概率不行, 先看是否有特殊字符集、短串或别的结构。

依赖的标准容器:

- `vector<int> dp, pre, tail_idx`
- `vector<ll> a`
- `string s, t`
- `vector<vector<int>> lcs_dp`

输入如何整理:

- LIS: 数组推荐读成 `vector<ll> a(n + 1)`, 1-index。
- LCS: 字符串保持 0-index, 但 DP 表用前缀长度 i/j , 访问 `s[i-1]`。
- 二维偏序: 把点整理成 (x, y) , 先排序, 再对 y 做 LIS。

接口:

```
int lis_strict_length(const vector<ll>& a);
int lnds_length(const vector<ll>& a);
string restore_lcs(const string& s, const string& t);
```

输出能力:

- LIS/LNDS 长度。
- 输出一条 LIS 或 LCS。
- 最长公共子串长度。
- 二维偏序最大链长度。
- 最少删除次数类答案。

下游可接：

- DP-11 LIS
- DP-09 LCS
- DP-10/22 编辑距离
- DP-18 DP + 树状数组/SegmentTree 优化

可拼接模块：

- Compressor: 带权/计数 LIS 的值域压缩。
- 树状数组/SegmentTree: 求前缀最大值或方案数。
- Sorting: 二维偏序先排序。

1. 变体路由表

题面	模型	关键区别
最长严格递增子序列	LIS	二分用 lower_bound
最长不下降子序列	LNDS	二分用 upper_bound
求一条具体 LIS	LIS + 前驱	d 只给长度, 要另存前驱
求 LIS 方案数	DP/树状数组	状态存 {长度, 方案数}
每个元素有收益, 递增且收益最大	带权 LIS	$dp[i] = w[i] + \max(dp[j])$
二维点 (x,y) 最长链	排序 + 一维 LIS	第一维升序, 第二维按规则处理重复
最少删除使序列递增	n - LIS	看严格还是不下降
两个序列最长公共子序列	LCS	不要求连续
两个字符串最长公共子串	公共子串 DP	不相等时清零
只允许删除使两串相同	LCS	删除次数 $n + m - 2 * lcs$
最短公共超序列长度	LCS	$n + m - lcs$
两个序列最长公共上升子序列	LCIS	扫描第二个序列维护可接到当前 a[i] 的 best

1A. 变体建模口令

LIS/LCS 变体最容易 “看着像模板, 实际状态没想清楚”。每次套模板前先补三句话：

1. 状态是什么：以哪里结尾？处理到哪个前缀？是否还要记住长度/方案数/最后值？
2. 最后一步是什么：最后选了哪个元素？最后匹配了哪两个字符？最后一个公共元素是谁？
3. 答案在哪里： $\max(dp[i])$ 、 $dp[n][m]$ 、还是 $\max(dp[j])$ ？

常用模型对照：

模型	状态来源	最后一步	答案
LIS $O(n^2)$	$dp[i]$ 以第 i 个数结尾	从某个 $j < i, a[j] < a[i]$ 接到 i	$\max dp[i]$
LCS	$dp[i][j]$ 两个前缀	$s[i]$ 与 $t[j]$ 匹配或跳过一边	$dp[n][m]$
公共子串	$dp[i][j]$ 以 $s[i], t[j]$ 同时结尾	相等就从左上延长, 不等清零	$\max dp[i][j]$
LCIS	$dp[j]$ 以 $b[j]$ 结尾且已处理当前 a 前缀	当前 $a[i]$ 匹配某个 $b[j]$	$\max dp[j]$

如果说不清 “以哪里结尾”，优先写暴力 DFS 或二维表推，不要直接抄二分 LIS。

2. LIS 严格/不下降短模板

注意：本卡的序列函数默认 $a[1..n]$ ，调用时要保留 $a[0]$ 这个 dummy 位，例如 `vector<ll> a(n + 1)`。如果你把普通 0-index vector 直接传进来，会漏掉第一个元素。

```
int lis_strict_length(const vector<ll>& a) {
    int n = (int)a.size() - 1;
    vector<ll> d;
    for (int i = 1; i <= n; i++) {
        auto it = lower_bound(d.begin(), d.end(), a[i]);
        if (it == d.end()) d.push_back(a[i]);
        else *it = a[i];
    }
}
```

```

    return (int)d.size();
}

int lnds_length(const vector<ll>& a) {
    int n = (int)a.size() - 1;
    vector<ll> d;
    for (int i = 1; i <= n; i++) {
        auto it = upper_bound(d.begin(), d.end(), a[i]);
        if (it == d.end()) d.push_back(a[i]);
        else *it = a[i];
    }
    return (int)d.size();
}

```

3. 输出一条 LIS

```

vector<ll> restore_lis(const vector<ll>& a) {
    int n = (int)a.size() - 1;
    vector<ll> tail_value;
    vector<int> tail_idx;
    vector<int> pre(n + 1, 0);

    for (int i = 1; i <= n; i++) {
        int pos = lower_bound(tail_value.begin(), tail_value.end(), a[i] - tail_value.begin());
        if (pos == (int)tail_value.size()) {
            tail_value.push_back(a[i]);
            tail_idx.push_back(i);
        } else {
            tail_value[pos] = a[i];
            tail_idx[pos] = i;
        }
        if (pos > 0) pre[i] = tail_idx[pos - 1];
    }

    vector<ll> ans;
    int cur = tail_idx.empty() ? 0 : tail_idx.back();
    while (cur != 0) {
        ans.push_back(a[cur]);
        cur = pre[cur];
    }
    reverse(ans.begin(), ans.end());
    return ans;
}

```

4. 二维偏序: 排序后一维 LIS

常见于信封嵌套: w 和 h 都要严格变大。

```

struct Point {
    int x;
    int y;
};

int two_dim_lis(vector<Point> p) {
    sort(p.begin(), p.end(), [](const Point& a, const Point& b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y > b.y; // x 相同不能互选, 所以 y 降序
    });

    vector<int> d;
    for (auto e : p) {
        auto it = lower_bound(d.begin(), d.end(), e.y);
        if (it == d.end()) d.push_back(e.y);
        else *it = e.y;
    }
}

```

```

    }
    return (int)d.size();
}

```

5. 输出一条 LCS

```

string restore_lcs(const string& s, const string& t) {
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    string ans;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s[i - 1] == t[j - 1]) {
            ans.push_back(s[i - 1]);
            i--;
            j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    reverse(ans.begin(), ans.end());
    return ans;
}

```

6. 统计 LIS 个数

朴素 $O(n^2)$ 版本最适合考场先写。len[i] 表示以 i 结尾的 LIS 长度，cnt[i] 表示这种长度的方案数。

```

pair<int, long long> count_lis(const vector<ll>& a) {
    int n = (int)a.size() - 1;
    vector<int> len(n + 1, 1);
    vector<long long> cnt(n + 1, 1);

    int best = 0;
    long long ways = 0;
    for (int i = 1; i <= n; i++) {
        len[i] = 1;
        cnt[i] = 1;
        for (int j = 1; j < i; j++) {
            if (a[j] < a[i]) {
                if (len[j] + 1 > len[i]) {
                    len[i] = len[j] + 1;
                    cnt[i] = cnt[j];
                } else if (len[j] + 1 == len[i]) {
                    cnt[i] += cnt[j];
                }
            }
        }

        if (len[i] > best) {
            best = len[i];
            ways = cnt[i];
        } else if (len[i] == best) {
            ways += cnt[i];
        }
    }
}

```

```

    }
}
return {best, ways};
}

```

若题目需要“值相同的序列只算一次”，计数规则会复杂很多；这份模板统计的是按下标选择的方案数。

7. 带权 LIS

每个元素有收益 $w[i]$ ，要求选出的值递增，最大化收益。

```

long long weighted_lis_n2(const vector<ll>& a, const vector<ll>& weight) {
    int n = (int)a.size() - 1;
    vector<long long> dp(n + 1, 0);
    long long ans = -LINF; // 若允许空子序列，可改成 0
    for (int i = 1; i <= n; i++) {
        dp[i] = weight[i];
        for (int j = 1; j < i; j++) {
            if (a[j] < a[i]) {
                dp[i] = max(dp[i], dp[j] + weight[i]);
            }
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}

```

$n \leq 5000$ 时可先交这个。 n 大时升级为坐标压缩 + 树状数组/SegmentTree 查询前缀最大值，见 DP-18。

8. 山形/合唱队形

求先严格上升再严格下降的最长长度。做两次 LIS: $up[i]$ 表示以 i 结尾的上升长度， $down[i]$ 表示以 i 开始的下降长度。

```

int bitonic_length_n2(const vector<ll>& a) {
    int n = (int)a.size() - 1;
    vector<int> up(n + 1, 1), down(n + 1, 1);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j < i; j++) {
            if (a[j] < a[i]) up[i] = max(up[i], up[j] + 1);
        }
    }

    for (int i = n; i >= 1; i--) {
        for (int j = n; j > i; j--) {
            if (a[j] < a[i]) down[i] = max(down[i], down[j] + 1);
        }
    }

    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = max(ans, up[i] + down[i] - 1);
    }
    return ans;
}

```

最少删除变成山形: $n - \text{bitonic_length}$ 。

9. 最长公共子串

公共子串要求连续，不是 LCS。

```

int longest_common_substring(const string& s, const string& t) {
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    int ans = 0;
}

```

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (s[i - 1] == t[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1] + 1;
            ans = max(ans, dp[i][j]);
        } else {
            dp[i][j] = 0;
        }
    }
}
return ans;
}

```

10. LCIS 最长公共上升子序列

两个序列既要“公共”，又要“严格上升”。dp[j] 表示当前处理到 a[i] 时，以 b[j] 结尾的 LCIS 长度。

为什么不是“先求 LCS 再求 LIS”：LCS 的某一条公共子序列不一定保留所有可能的上升选择，先求出来的一条 LCS 可能把更优 LCIS 需要的元素丢掉。必须在匹配两个序列的同时维护“上升”限制。

最后一步视角：

如果 $a[i] == b[j]$ ，并且 LCIS 最后一个公共元素选 $b[j]$ ，那么前一个公共元素必须来自某个 $k < j$ 且 $b[k] < a[i]$ 。扫描 $b[j]$ 时维护 $best = \max(dp[k])$ ，就能 $O(nm)$ 转移。

```

int lcis_length(const vector<int>& a, const vector<int>& b) {
    int n = (int)a.size() - 1;
    int m = (int)b.size() - 1;
    vector<int> dp(m + 1, 0);

    for (int i = 1; i <= n; i++) {
        int best = 0;
        for (int j = 1; j <= m; j++) {
            if (a[i] == b[j]) {
                dp[j] = max(dp[j], best + 1);
            }
            if (b[j] < a[i]) {
                best = max(best, dp[j]);
            }
        }
    }

    int ans = 0;
    for (int j = 1; j <= m; j++) ans = max(ans, dp[j]);
    return ans;
}

```

触发词是“两个序列”“公共”“上升/递增”。如果只说公共，不说递增，用 LCS；如果只说递增，不说两个序列，用 LIS。

常见坑：

- lower_bound 是严格 LIS，upper_bound 是不下降 LIS。
- 二分 LIS 的 d 不是真实答案序列，恢复路径要另存前驱。
- 二维偏序有重复第一维时，第二维通常要降序，防止同一第一维被选两次。
- LCS 是子序列，公共子串是连续子串，两者转移不同。
- LCS 路径恢复中，相等走左上，不等走更大的方向。
- LIS 计数模板默认按下标计数；如果题目按值去重，要另做去重逻辑；如果题目要求取模，每次累加 cnt 都要取模，精确大数要转高精度。
- LCIS 的 best 必须在扫描 $b[j]$ 时维护，不能简单套 LCS 后再 LIS。

暴力/部分分替代：

- LIS: $n \leq 25$ 先 DFS 选/不选， $n \leq 5000$ 用 $O(n^2)$ 。
- LCS: 短串先 DFS/记忆化， $n*m$ 可承受再二维表推。
- 二维偏序不确定排序规则时，用小数据暴力枚举子集验证。

升级方向：

LIS 长度 -> 输出路径 -> 计数/带权 -> 树状数组/SegmentTree

LCS 长度 -> 输出一条 LCS -> 滚动数组节省空间

公共子序列 -> 公共子串: 相等延长, 不等清零

二维选择 -> 排序 -> 一维 LIS

最小测试样例:

LIS:

6

10 9 2 5 3 7

输出: 3

LCS:

abcde

ace

输出: ace

DP-24: 背包常见变体总表

模型编号: DP-24

模型名称: 背包常见变体总表

标签: DP、背包、多重背包、二维背包、价值维度、计数、可行性

一句话用途: 把 0/1、完全、分组之外的背包变体集中成一张考场路由卡, 先判循环顺序和初始化, 再抄短模板。

题面触发词:

- “每种物品最多选 k 个”
- “重量和体积两个限制”
- “价值总和较小, 容量很大”
- “问方案数/能否凑出/最少硬币数”
- “每组必须选一个”
- “依赖关系、选父才能选子”

什么时候用:

- 题目本质是“从若干物品/方案中选择”, 并受容量、预算、数量、体积等资源限制。
- 资源维度较小, 或可以把较小的维度作为 DP 下标。
- 正解不明显时, 可先写 $\text{dfs}(i, \text{rest})$ 记忆化, 再改表推。

不要什么时候用:

- 容量上限巨大且没有可替代小维度, 普通背包会爆内存。
- 选择之间有复杂图关系, 不能只靠容量表达。
- 明显是区间合并、最短路、排序贪心, 不要硬套背包。
- 物品价值可能为负时, 初始化和答案位置要重新检查。

复杂度:

- 0/1、完全、分组: 常见 $O(nW)$ 。
- 多重背包二进制拆分: $O(W * \sum(\log \text{cnt}[i]))$ 。
- 二维背包: $O(n * W * V)$ 。
- 价值维度背包: $O(n * \sum \text{value})$ 。
- bitset 可行性: 约 $O(nW/\text{word})$, 代码很短。

数据范围信号:

- $n * W \leq 1e7$: 容量维度背包较稳。
- W 很大但 $\sum(v) \leq 1e5$: 价值维度背包。
- $W1 * W2 * n \leq 1e7$: 二维背包可考虑。
- $\text{cnt}[i]$ 很大但有上限: 二进制拆分或单调队列优化, 多数考场先用二进制拆分。

依赖的标准容器:

- `vector<int> w, cnt`
- `vector<ll> v, dp`
- `bitset<MAXW + 1>` 或 `vector<char> reachable`

输入如何整理：

- 统一 $w[i]$ 表示容量消耗, $v[i]$ 表示价值或收益。
- 多重背包额外读 $cnt[i]$ 。
- 二维背包读 $w1[i], w2[i], val[i]$ 。
- 分组/依赖题先整理成组或树, 再做背包合并。

接口：

```
ll multiple_knapsack_binary(int n, int W, vector<int>& w, vector<ll>& v, vector<int>& cnt);
ll value_dimension_knapsack(int n, long long W, vector<int>& w, vector<int>& v);
```

输出能力：

- 最大价值。
- 最小花费。
- 是否可达。
- 方案数。
- 每组最多/必须选一个。

下游可接：

- DP-06 0/1 背包
- DP-07 完全背包
- DP-08 分组背包
- DP-14 树形 DP
- DP-20 计数/可行性 DP

可拼接模块：

- BRUTE 选/不选 DFS：小数据或先写记忆化。
- PrefixSum/MonotonicQueue：高级多重背包优化。
- TreeDP：依赖背包、树上背包。

1. 背包变体路由表

题面	模型	核心写法
每个物品最多一次	0/1 背包	容量倒序
每种物品无限个	完全背包	容量正序
每种物品最多 $cnt[i]$ 个	多重背包	二进制拆成若干 0/1 物品
每组最多选一个	分组背包	每组用 ndp
每组必须选一个	必选分组背包	ndp 初始化为 $-LINF$
两个容量限制	二维背包	两层容量都倒序
容量巨大, 价值和小	价值维度背包	$dp[value]=$ 最小重量
问能否凑出	可行性背包	$bool\ dp[j]$ 或 $bitset$
问方案数	计数背包	注意组合数/排列数循环顺序
问最少硬币数	最短背包	$dp[j]=\min(dp[j], dp[j-w]+1)$
树上选 K 个点、子树容量合并	树上背包	DP-14 + DP-19, 容量合并
选父才能选子	依赖背包	先建依赖树; 和“树上任选 K 个”不是同一题

1A. 背包变体建模口令

背包的共同核心不是“背模板”，而是把题目翻译成资源坐标：

$dp[\text{资源用量}] =$ 当前能得到的最好答案 / 方案数 / 是否可行

最后一步 = 最后考虑的那个物品到底选了几次

然后按题面把资源轴和物品规则调整：

发现的问题	对状态/循环的影响	动机
每件最多一次	容量倒序	防止同一件在本轮被重复使用
每件无限次	容量正序	允许本轮刚更新的状态继续使用同一件
每件最多 cnt 次	拆成若干 0/1 物品或单调队列	把“有限次数”转回熟悉的 0/1 规则
有两个资源限制	$dp[w1][w2]$ 升维	单独知道重量不够, 还要知道体积/时间
容量很大但价值小	$dp[value]=$ 最小重量 换维度	用小的那一维做下标
要计数	$dp[j] += dp[j-w]$	最后一步从能到达 $j-w$ 的方案接过来

如果一维 `dp[j]` 写不明白，先写二维 `dp[i][j]`；二维清楚后再滚动成一维。

2. 多重背包：二进制拆分成 0/1

二进制拆分的动机：如果一种物品最多取 13 个，不必枚举 0..13 每个数量。把它拆成 1,2,4,6 四包，每包只能选或不选，就能表示 0..13 的任意数量，同时直接套 0/1 背包倒序循环。

```
11 multiple_knapsack_binary(int n, int W, vector<int>& w, vector<ll>& v, vector<int>& cnt) {
    vector<int> nw;
    vector<ll> nv;
    nw.push_back(0);
    nv.push_back(0);

    for (int i = 1; i <= n; i++) {
        long long c = cnt[i];
        for (long long k = 1; k <= c; k <= 1) {
            long long packed_w = 1LL * w[i] * k;
            nw.push_back(packed_w > W ? W + 1 : (int)packed_w);
            nv.push_back(v[i] * k);
            c -= k;
        }
        if (c > 0) {
            long long packed_w = 1LL * w[i] * c;
            nw.push_back(packed_w > W ? W + 1 : (int)packed_w);
            nv.push_back(v[i] * c);
        }
    }

    vector<ll> dp(W + 1, 0);
    for (int i = 1; i < (int)nw.size(); i++) {
        for (int j = W; j >= nw[i]; j--) {
            dp[j] = max(dp[j], dp[j - nw[i]] + nv[i]);
        }
    }
    return dp[W];
}
```

3. 二维背包

例如同时限制重量 A 和体积 B。

```
vector<vector<ll>> dp(A + 1, vector<ll>(B + 1, 0));
for (int i = 1; i <= n; i++) {
    for (int a = A; a >= w1[i]; a--) {
        for (int b = B; b >= w2[i]; b--) {
            dp[a][b] = max(dp[a][b], dp[a - w1[i]][b - w2[i]] + val[i]);
        }
    }
}
cout << dp[A][B] << '\n';
```

4. 价值维度背包

当 W 很大但总价值小，用 `dp[value]` = 达到该价值的最小重量。

```
11 value_dimension_knapsack(int n, long long W, vector<int>& w, vector<int>& v) {
    int V = 0;
    for (int i = 1; i <= n; i++) V += v[i];

    const long long INF = numeric_limits<long long>::max() / 4;
    vector<long long> dp(V + 1, INF);
    dp[0] = 0;

    for (int i = 1; i <= n; i++) {
        for (int val = V; val >= v[i]; val--) {

```

```

        if (dp[val - v[i]] != INF) {
            dp[val] = min(dp[val], dp[val - v[i]] + w[i]);
        }
    }
}

int ans = 0;
for (int val = 0; val <= V; val++) {
    if (dp[val] <= W) ans = val;
}
return ans;
}

```

5. 可行性背包 bitset

只问某个和能不能凑出时非常好抄。

```

bitset<100005> can;
can[0] = 1;
for (int i = 1; i <= n; i++) {
    can |= (can << w[i]); // 0/1 可达性
}
cout << (can[target] ? "Yes" : "No") << '\n';

```

如果 target 不是编译期常量, 改用 vector<char> 普通背包。

6. 组合数 vs 排列数

完全背包计数最容易错。

组合数: 不关心顺序, 1+2 和 2+1 算同一种。

```

vector<int> dp(W + 1, 0);
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int j = w[i]; j <= W; j++) {
        dp[j] = (dp[j] + dp[j - w[i]]) % MOD;
    }
}

```

排列数: 关心顺序, 1+2 和 2+1 算两种。

```

vector<int> dp(W + 1, 0);
dp[0] = 1;
for (int j = 1; j <= W; j++) {
    for (int i = 1; i <= n; i++) {
        if (j >= w[i]) dp[j] = (dp[j] + dp[j - w[i]]) % MOD;
    }
}

```

7. 至少装满的最小代价

题面常见说法: “至少达到 W 的容量/攻击力/分数, 花费最小”。推荐容量封顶: 超过 W 的状态都合并到 W。

```

long long at_least_knapsack_min_cost(int n, int W, vector<int>& gain, vector<int>& cost) {
    const long long INF = numeric_limits<long long>::max() / 4;
    vector<long long> dp(W + 1, INF);
    dp[0] = 0;

    for (int i = 1; i <= n; i++) {
        vector<long long> ndp = dp;
        for (int j = 0; j <= W; j++) {
            if (dp[j] == INF) continue;
            int nj = min(W, j + gain[i]);
            ndp[nj] = min(ndp[nj], dp[j] + cost[i]);
        }
        dp.swap(ndp);
    }
}

```

```

    }
    return dp[W] == INF ? -1 : dp[W];
}

```

如果每个物品可以无限选，把 `ndp` 改成原地正序更新，但考场不确定时先写 0/1 版本保稳。

8. 0/1 背包路径恢复：二维 take

初学者要输出选了哪些物品时，优先用二维表，别急着一维恢复。

```

vector<int> restore_zero_one_items(int n, int W, vector<int>& w, vector<int>& v) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    vector<vector<char>> take(n + 1, vector<char>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp[i][j] = dp[i - 1][j];
            if (j >= w[i] && dp[i - 1][j - w[i]] + v[i] > dp[i][j]) {
                dp[i][j] = dp[i - 1][j - w[i]] + v[i];
                take[i][j] = 1;
            }
        }
    }

    vector<int> chosen;
    int j = W;
    for (int i = n; i >= 1; i--) {
        if (take[i][j]) {
            chosen.push_back(i);
            j -= w[i];
        }
    }
    reverse(chosen.begin(), chosen.end());
    return chosen;
}

```

若题目要求“恰好装满”，二维 `dp` 要用不可达初始化，不能默认全 0。

常见坑：

- 多重背包直接正序更新，会把数量上限冲掉。
- 二维背包如果每个物品最多一次，两个容量都要倒序。
- 价值维度里 `dp[value]` 存重量，不是价值。
- 可行性/恰好装满必须初始化不可达状态，不能全 0。
- 完全背包计数要先区分组合数和排列数。
- 分组“必须选一个”不能 `ndp = dp`。
- “至少装满”不要直接访问超过 `W` 的下标，推荐容量封顶。
- 一维背包路径恢复容易被覆盖；初学者优先二维 `take`。

暴力/部分分替代：

- $n \leq 25$ ：DFS 枚举每件选/不选。
- 多重背包数量小：DFS 枚举每种取几个。
- 容量小：`dfs(i, rest)` 加记忆化。
- 不确定一维循环方向时，先写二维 `dp[i][j]`。

升级方向：

DFS 选物品 -> 0/1 背包

数量无限 -> 完全背包

数量有限 -> 二进制拆分

两种资源 -> 二维背包

容量太大 -> 价值维度

只问能否 -> `bitset`/可行性背包

问方案数 -> 计数背包，先判组合/排列

最小测试样例：

多重背包:

2 10

2 3 3

3 4 2

输出: 14

说明: 3 个第一种 + 1 个第二种, 容量 9, 价值 13; 2 个第一种 + 2 个第二种, 容量 10, 价值 14。

DP-25: 两道例题的暴力 DFS 到记忆化搜索

模块编号: DP-25

模块名称: 两道例题的暴力 DFS 到记忆化搜索

标签: DP、记忆化搜索、暴力升级、P1874、编辑距离、自顶向下

一句话用途: 如果考场上没能直接推导出递推 DP, 就先写最自然的暴力 DFS, 再把 DFS 的可变参数变成 memo 下标, 快速拿到部分分甚至满分。

题面触发词:

- “我能递归枚举所有选择, 但复杂度爆炸”
- “同一个位置/同一组参数会被反复算”
- “字符串切分、双序列匹配、选或不选、区间递归”
- “求最大/最小/方案数/是否可行”
- “DP 方程一时想不出来”

什么时候用:

- 暴力 DFS 很容易按题意写出来。
- DFS 的参数个数较少, 且每个参数范围能估算。
- 同样的参数代表同一个子问题, 答案不依赖 “怎么走到这里”。
- 递归深度不大, 或至少不会明显到 $1e5$ 以上。

不要什么时候用:

- 状态有环且没有处理 “正在访问” 的标记, 容易递归死循环。
- DFS 参数漏掉了影响未来的全局变量, 例如 `used[]`、`last`、`path`。
- 状态数量远超内存, 数组 memo 开不下且哈希也会太慢。
- 递归深度极深, 例如链状 $n=5e5$, 可能爆栈。

复杂度:

- 纯暴力: 通常是指数级, 例如 2^n 或 3^n 。
- 记忆化搜索: 约等于 状态数 * 每个状态的转移数。
- P1874 快速求和: 约 $O(\text{len}^2 * \text{target})$ 。
- 编辑距离: $O(nm)$ 。

数据范围信号:

- P1874: $\text{len} \leq 40$, $\text{target} \leq 1e5$, `memo[pos][sum]` 可开。
- 编辑距离: $n, m \leq 2000$, `memo[i][j]` 可开。
- 一般题: 先估 状态总数, 超过 $1e7$ 要谨慎, 超过 $1e8$ 通常危险。

依赖的标准容器:

- `vector<vector<int>>` memo
- `vector<vector<char>>` vis
- `string`
- `tuple/map/unordered_map`: 状态稀疏或参数不好做下标时备用。

输入如何整理:

- 把不变的输入放成全局或传引用: 字符串、数组、目标值。
- DFS 参数只放 “会变化且决定未来” 的量。
- 先写清楚一句话: `dfs(...)` 返回什么。

接口:

暴力 `dfs`(可变参数) -> 加 memo/vis -> 每个状态只算一次

输出能力:

- 最小代价。
- 最大收益。
- 方案数。
- 是否可行。
- 小数据精确解和中数据记忆化部分分。

下游可接：

- DP-03 DFS -> 记忆化 -> 表推升级图
- DP-21 P1874 快速求和
- DP-22 编辑距离
- BRUTE-07/08/09 记忆化搜索实现

可拼接模块：

- vector memo: 参数范围小且整数下标。
- map<tuple<...>, int>: 状态复杂但求稳。
- unordered_map<long long, int>: 状态稀疏且能编码。

1. 考场口令

先写暴力 DFS。

看 DFS 函数参数。

去掉全局不变参数。

剩下的可变参数就是状态。

同样状态答案相同，就加 memo。

要特别问自己一句：

同样的 dfs 参数，未来答案是不是永远一样？

如果答案是“是”，就可以记忆化。

注意：表推 DP 和记忆化 DFS 可能访问同一批“位置 + 约束”状态，但 dp 值的含义不一定完全一样。

题目	表推含义	记忆化含义
P1874 快速求和	dp[i][sum]: 前 i 个字符已经切完且和为 sum 的最少已用加号数	dfs(pos, sum): 已经处理到 pos、当前和为 sum，后面最少还要多少加号
编辑距离	dp[i][j]: A 前 i 个字符变成 B 前 j 个字符的最少操作数	dfs(i, j): 从 A[i..] 变成 B[j..] 的最少后续操作数

考场记法：状态维度相同代表可以互相启发；值的方向要按函数定义重新写清楚。

2. P1874 快速求和：记忆化 DFS 完整代码

定义：

dfs(pos, sum) = 已经处理到 s[pos]，前面数字段总和为 sum 时，从 pos 往后凑到 target 还需要的最少加号数。

注意第一段前面没有加号，所以选择一段 s[pos..end] 后，新增加号数是：

```
int add = (pos == 0 ? 0 : 1);
```

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;

string s;
int target_value;
int len;
vector<vector<int>>> memo;
vector<vector<char>>> vis;

int dfs_fast_sum(int pos, int sum) {
    if (sum > target_value) return INF;
```

```

    if (pos == len) {
        return sum == target_value ? 0 : INF;
    }

    if (vis[pos][sum]) return memo[pos][sum];
    vis[pos][sum] = 1;

    int ans = INF;
    long long val = 0;
    for (int end = pos; end < len; end++) {
        val = val * 10 + (s[end] - '0');
        if (sum + val > target_value) break;

        int add = (pos == 0 ? 0 : 1);
        int got = dfs_fast_sum(end + 1, (int)(sum + val));
        if (got != INF) {
            ans = min(ans, add + got);
        }
    }

    memo[pos][sum] = ans;
    return ans;
}

int solve_fast_sum_memo(const string& input, int target) {
    s = input;
    target_value = target;
    len = (int)s.size();
    memo.assign(len + 1, vector<int>(target_value + 1, INF));
    vis.assign(len + 1, vector<char>(target_value + 1, 0));

    int ans = dfs_fast_sum(0, 0);
    return ans == INF ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string input;
    int target;
    cin >> input >> target;

    cout << solve_fast_sum_memo(input, target) << '\n';
    return 0;
}

```

为什么复杂度降下来:

纯 DFS 会反复进入相同的 (pos, sum)。

加 memo 后, 每个 (pos, sum) 只算一次。

每个状态枚举后面一段的 end, 最多 len 次。

所以约 $O(len * target * len)$ 。

这道题的递归深度最多 $len \leq 40$, 不用担心爆栈。

3. 编辑距离: 记忆化 DFS 完整代码

定义:

$dfs(i, j)$ = 把 a 从 i 开始的后缀变成 b 从 j 开始的后缀, 最少需要多少次操作。

如果 $a[i] == b[j]$, 当前字符不用动, 直接进入 $dfs(i+1, j+1)$ 。

如果不同, 有三种选择:

删除 a[i]: dfs(i+1, j) + 1
 插入 b[j]: dfs(i, j+1) + 1
 替换 a[i]: dfs(i+1, j+1) + 1

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

string a, b;
int n, m;
vector<vector<int>> memo;
vector<vector<char>> vis;

int dfs_edit_distance(int i, int j) {
    if (i == n) return m - j;
    if (j == m) return n - i;

    if (vis[i][j]) return memo[i][j];
    vis[i][j] = 1;

    int ans;
    if (a[i] == b[j]) {
        ans = dfs_edit_distance(i + 1, j + 1);
    } else {
        int del_cost = dfs_edit_distance(i + 1, j) + 1;
        int ins_cost = dfs_edit_distance(i, j + 1) + 1;
        int rep_cost = dfs_edit_distance(i + 1, j + 1) + 1;
        ans = min({del_cost, ins_cost, rep_cost});
    }

    memo[i][j] = ans;
    return ans;
}

int solve_edit_distance_memo(const string& x, const string& y) {
    a = x;
    b = y;
    n = (int)a.size();
    m = (int)b.size();
    memo.assign(n + 1, vector<int>(m + 1, 0));
    vis.assign(n + 1, vector<char>(m + 1, 0));
    return dfs_edit_distance(0, 0);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string x, y;
    cin >> x >> y;

    cout << solve_edit_distance_memo(x, y) << '\n';
    return 0;
}
```

为什么复杂度降下来:

纯 DFS 每次最多分出 3 个分支, 复杂度接近指数级。

加 memo 后, 状态只有 (i,j), 总数约 $n*m$ 。

每个状态 $O(1)$ 转移, 所以 $O(nm)$ 。

本题 $n, m \leq 2000$, 递归深度大约 $n+m \leq 4000$, 通常安全。

4. 什么时候记忆化能直接满分

满足这四条，记忆化搜索经常和表推 DP 一样强：

1. 状态没有环，或者递归天然走向更小/更后的状态。
2. 状态数能承受。
3. 每个状态转移不太多。
4. 递归深度不爆栈。

P1874 和编辑距离都满足：

题目	状态	递归方向	状态数	结果
P1874	(pos, sum)	pos 变大	len * target	可满分
编辑距离	(i, j)	i/j 变大	n * m	可满分

5. 什么时候只能拿部分分

记忆化不一定总是满分，但仍然很值得写：

- 状态范围大，但实际访问状态少，用 map/unordered_map 可拿中档分。
- 转移很多，状态数乘转移数仍偏大，但比纯暴力好很多。
- 状态漏了一些信息，先补全状态可能会变大，但小数据仍可过。
- 递归深度太深时可能 RE，要准备表推或迭代版。

6. 数组 memo 与 map memo 的选择

优先级：

参数范围小、连续整数 -> vector/数组

状态是 tuple 且范围不清 -> map

状态稀疏且能编码成 long long -> unordered_map

复杂状态求稳模板：

```
#include <bits/stdc++.h>
using namespace std;

map<tuple<int, int, int>, int> memo;

int dfs(int i, int sum, int last) {
    auto key = make_tuple(i, sum, last);
    auto it = memo.find(key);
    if (it != memo.end()) return it->second;

    int ans = 0;
    // 在这里枚举选择，更新 ans。

    memo[key] = ans;
    return ans;
}
```

7. 考场策略总结

第一步：写最直白 DFS，不要一开始硬想 for 循环 DP。

第二步：把函数参数写成一句状态定义。

第三步：删掉不变参数，留下决定未来的参数。

第四步：估算状态数，能开数组就开数组。

第五步：先判非法和终止，确认下标合法后查 memo，在返回前写 memo。

第六步：样例过后交一版，后面再考虑表推、滚动数组或数据结构优化。

关键判断：

如果同一个 dfs 参数再次出现，答案完全一样，就能 memo。

如果答案还依赖 used/path/last/当前方向，这些也必须进状态。

常见坑：

- 用 -1 表示没算过，但合法答案也可能是 -1；稳妥用 vis。

- base case 放在查 memo 后面，导致空状态访问越界；正确顺序通常是先判非法/终止，再查 memo。
- 参数漏信息，例如只写 `dfs(pos)`，但其实还依赖 `sum` 或 `last`。
- `sum` 可能超过目标，没剪枝导致数组越界。
- 多组数据忘记清空 `memo/vis`。
- 递归深度很深时爆栈。

暴力/部分替代：

- 完全不会 DP：先交纯 DFS 小数据版。
- DFS 会重复：加 memo。
- 递归深度危险：把 memo DFS 改成表推。
- 状态太大：删掉无关历史，或只用 map 存实际访问状态。

升级方向：

纯 DFS -> DFS + memo -> 表推 DP -> 滚动数组/数据结构优化

最小测试样例：

P1874：
99999
45
输出：4

编辑距离：

sfdqxbw
gfdgw
输出：4

DP-26：发现有后效性怎么办：升维吸收关键历史

模块编号：DP-26

模块名称：发现有后效性怎么办：升维吸收关键历史

标签：DP、无后效性、有后效性、状态升维、last、used、phase、网格 DP

一句话用途：当你发现 `dp[i]` 或 `dp[i][j]` 不够用，因为“前面怎么来的”会影响后面选择时，把那一点关键历史加进状态，让新状态重新满足无后效性。

题面触发词：

- “不能连续两次做某操作”
- “相邻不能相同”
- “上一步方向/颜色/动作影响下一步”
- “最多使用一次机会”
- “是否已经使用过某个技能/优惠/钥匙”
- “当前处于持有/冷却/空闲状态”
- “恰好选 K 个，同时还有相邻限制”

什么时候用：

- 两个局面的位置相同，但因为历史不同，后续合法选择不同。
- 转移时你忍不住想看 `path`、`lastMove`、`used`、`previousColor` 这类变量。
- 记忆化搜索中，同样的 `dfs(pos)` 得不到唯一答案，必须改成 `dfs(pos, extra_state)`。

不要什么时候用：

- 历史信息能由当前位置唯一推出，不需要重复加维。
- 历史只影响当前得分，不影响未来合法选择，可能直接在转移里处理即可。
- 升维后状态数爆炸，例如 $n * m * 2^k$ 中 k 太大，要考虑换模型或只保留更小摘要。
- 图上可以任意回头且有环时，不一定是 DP，可能要 BFS/Dijkstra/状态搜索。

复杂度：

升维后复杂度 = 原状态数 * 新维度大小 * 每个状态转移数

例如：

- `dp[i][j]` 加上一维上一步方向 2：状态数变成 $2 * n * m$ 。

- $dp[i]$ 加上是否选择当前元素 2: 状态数变成 $2*n$ 。
- $dp[i][j]$ 加上是否用过一次机会 2: 状态数变成 $2*n*m$ 。
- $dp[i]$ 加上已选数量 K 和当前是否选 2: 状态数变成 $2*n*K$ 。

数据范围信号:

- 新维度只有 2/3/10 种: 通常很稳。
- 新维度是容量 W : 看 nW 是否可承受。
- 新维度是集合 $mask$: 要求元素数通常 $n \leq 20$ 。

依赖的标准容器:

- `vector`
- `array<ll, 2>`
- `array<ll, 3>`
- `vector<vector<array<ll, 2>>>`
- `vector<vector<vector<ll>>>`

输入如何整理:

- 先写错误状态, 例如 $dp[i][j]$ 。
- 找反例: 同一个 (i, j) , 不同历史导致下一步能做的选择不同。
- 把这个关键历史压成小整数: `last/used/color/phase/taken`。

接口:

错误状态 -> 找出影响未来的关键历史 -> 加一维 -> 新状态转移

输出能力:

- 最大收益。
- 最小代价。
- 方案数。
- 是否可行。

下游可接:

- DP-03B 状态增维路由。
- DP-12 网格 DP。
- DP-04/05 线性 DP、选/不选 DP。
- DP-25 暴力 DFS 到记忆化搜索。

可拼接模块:

- GridDP: 位置 (i, j) 。
- ChooseOrSkip: 选择状态 0/1。
- MemoDFS: 先写 `dfs(..., last, used)`。

1. 核心原则

有后效性 = 同一个旧状态, 未来不唯一。

破局方法 = 把影响未来的那一点历史加进状态。

升维不是乱加维度, 而是只加“未来还需要知道”的最小历史摘要。

检查句式:

只知道我在 X , 下一步能做什么是否唯一?

如果不唯一, 还差哪一点历史?

把那一点历史变成状态打标。

2. 例题一: 网格最大收益, 不能连续向下走两步

题意:

给 $n*m$ 网格, 每格有收益。

从 $(1, 1)$ 走到 (n, m) , 每步只能向下或向右。

规则: 不能连续向下走两步。

求最大收益。

错误状态:

$dp[i][j]$ = 到达 (i, j) 的最大收益

为什么错:

同样到达 (i,j), 如果上一步是向下, 下一步不能继续向下。

如果上一步是向右, 下一步可以向下。

只知道 (i,j) 不够, 未来不唯一。

正确升维:

$dp[i][j][0]$ = 到达 (i,j), 且最后一步是向下的最大收益

$dp[i][j][1]$ = 到达 (i,j), 且最后一步是向右的最大收益

转移:

最后一步向下: 只能从上方来, 且上一个状态最后一步不能也是向下。

$dp[i][j][0] = dp[i-1][j][1] + a[i][j]$

最后一步向右: 从左方来, 上一步方向不限。

$dp[i][j][1] = \max(dp[i][j-1][0], dp[i][j-1][1]) + a[i][j]$

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;
const int MAXN = 1000 + 5;
const int MAXM = 1000 + 5;
static ll a[MAXN][MAXM];
static ll dp[MAXN][MAXM][2];

ll max_grid_no_two_down(int n, int m) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            dp[i][j][0] = dp[i][j][1] = NEG;
        }
    }

    dp[1][1][0] = a[1][1];
    dp[1][1][1] = a[1][1];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (i == 1 && j == 1) continue;

            if (i > 1 && dp[i-1][j][1] != NEG) {
                dp[i][j][0] = max(dp[i][j][0], dp[i-1][j][1] + a[i][j]);
            }
            if (j > 1) {
                ll best_left = max(dp[i][j-1][0], dp[i][j-1][1]);
                if (best_left != NEG) {
                    dp[i][j][1] = max(dp[i][j][1], best_left + a[i][j]);
                }
            }
        }
    }

    ll ans = max(dp[n][m][0], dp[n][m][1]);
    return ans == NEG ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
```

```

    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }

    cout << max_grid_no_two_down(n, m) << '\n';
    return 0;
}

```

最小测试:

```

3 2
1 1
100 1
100 1

```

输出: 103

说明: 不能走下、下、右; 可走下、右、下。

3. 例题二: 爬楼梯, 不能连续跳同样步数

题意:

从第 0 级爬到第 n 级, 每次跳 1 或 2 级。

不能连续两次跳同样步数。

问方案数。

错误状态:

$dp[i]$ = 到达第 i 级的方案数

为什么错:

同样到达第 i 级, 如果上一跳是 1, 下一跳不能跳 1。

如果上一跳是 2, 下一跳不能跳 2。

只知道 i 不够。

正确状态:

$dp[i][last]$ = 到达第 i 级, 上一跳是 last 的方案数

$last=0$ 表示起点还没有上一跳, $last=1/2$ 表示上一跳步数。

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1000000007;

int count_stairs_no_same_jump(int n) {
    if (n == 0) return 1;
    vector<array<int, 3>> dp(n + 1);
    dp[0][0] = 1;

    for (int i = 0; i <= n; i++) {
        for (int last = 0; last <= 2; last++) {
            if (dp[i][last] == 0) continue;
            for (int step = 1; step <= 2; step++) {
                if (step == last) continue;
                if (i + step <= n) {
                    dp[i + step][step] += dp[i][last];
                    if (dp[i + step][step] >= MOD) dp[i + step][step] -= MOD;
                }
            }
        }
    }
}

```

```

    return (dp[n][1] + dp[n][2]) % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    cout << count_stairs_no_same_jump(n) << '\n';
    return 0;
}

```

最小测试:

4

输出: 1

说明: 只能 1,2,1。

4. 例题三: 恰好选 K 个数, 且不能选相邻

题意:

给 n 个数, 恰好选 K 个, 不能选相邻位置, 求最大和。

错误状态:

$dp[i][cnt]$ = 处理完前 i 个, 已经选 cnt 个的最大和

为什么还不够:

处理第 i+1 个时, 需要知道第 i 个是否已经选了。

如果第 i 个选了, 第 i+1 个不能选; 如果没选, 就可以选。

正确状态:

$dp[i][cnt][0]$ = 处理完前 i 个, 选了 cnt 个, 且第 i 个没选的最大和

$dp[i][cnt][1]$ = 处理完前 i 个, 选了 cnt 个, 且第 i 个选了的和

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;

ll max_sum_choose_k_no_adjacent(const vector<ll>& a, int K) {
    int n = (int)a.size() - 1;
    vector<vector<array<ll, 2>>> dp(n + 1, vector<array<ll, 2>>(K + 1, {NEG, NEG}));

    dp[0][0][0] = 0;
    for (int i = 1; i <= n; i++) {
        for (int cnt = 0; cnt <= K; cnt++) {
            dp[i][cnt][0] = max(dp[i - 1][cnt][0], dp[i - 1][cnt][1]);
            if (cnt > 0 && dp[i - 1][cnt - 1][0] != NEG) {
                dp[i][cnt][1] = dp[i - 1][cnt - 1][0] + a[i];
            }
        }
    }

    ll ans = max(dp[n][K][0], dp[n][K][1]);
    return ans == NEG ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```

```

int n, K;
cin >> n >> K;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];

cout << max_sum_choose_k_no_adjacent(a, K) << '\n';
return 0;
}

```

最小测试:

```

5 2
2 7 9 3 1

```

输出: 11

说明: 选 2 和 9。

5. 例题四: 网格最大收益, 最多一次收益翻倍

题意:

从 (1,1) 到 (n,m), 只能向右或向下。

可以选择最多一个格子, 把该格收益翻倍。

求最大收益。

有后效性的地方:

同样走到 (i,j), 如果已经用过翻倍机会, 后面就不能再用。

如果还没用过, 后面仍然可以用。

正确状态:

dp[i][j][0] = 到达 (i,j), 还没用过翻倍机会的最大收益

dp[i][j][1] = 到达 (i,j), 已经用过翻倍机会的最大收益

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;

ll max_grid_one_double(int n, int m, vector<vector<ll>>& a) {
    vector<vector<array<ll, 2>>> dp(n + 1, vector<array<ll, 2>>(m + 1, {NEG, NEG}));

    dp[1][1][0] = a[1][1];
    dp[1][1][1] = a[1][1] * 2;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (i == 1 && j == 1) continue;

            array<ll, 2> best = {NEG, NEG};
            if (i > 1) {
                best[0] = max(best[0], dp[i - 1][j][0]);
                best[1] = max(best[1], dp[i - 1][j][1]);
            }
            if (j > 1) {
                best[0] = max(best[0], dp[i][j - 1][0]);
                best[1] = max(best[1], dp[i][j - 1][1]);
            }

            if (best[0] != NEG) {
                dp[i][j][0] = max(dp[i][j][0], best[0] + a[i][j]);
                dp[i][j][1] = max(dp[i][j][1], best[0] + 2 * a[i][j]);
            }
            if (best[1] != NEG) {

```

```

        dp[i][j][1] = max(dp[i][j][1], best[1] + a[i][j]);
    }
}

ll ans = max(dp[n][m][0], dp[n][m][1]);
return ans == NEG ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<ll>> a(n + 1, vector<ll>(m + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }

    cout << max_grid_one_double(n, m, a) << '\n';
    return 0;
}

```

最小测试:

```

2 2
1 10
2 3

```

输出: 24

说明: 路径 1->10->3, 并把 10 翻倍。

6. 例题五: 股票买卖, 冷冻期一天

题意:

给每天股价。每天可以不动、买入、卖出。
卖出后的下一天不能买入, 求最大收益。

错误状态:

$dp[i]$ = 第 i 天结束后的最大收益

为什么错:

同样第 i 天结束, 有可能手里持股、空闲、刚卖出处于冷冻。
这三种状态下一天能做的操作不同。

正确状态:

hold = 手里持股

free = 手里没股且不在冷冻, 可买

cool = 今天刚卖出, 下一天冷冻

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;

ll max_profit_with_cooldown(const vector<ll>& price) {
    int n = (int)price.size() - 1;
    vector<array<ll, 3>> dp(n + 1, {NEG, NEG, NEG});

    dp[0][0] = NEG; // hold

```

```

dp[0][1] = 0;    // free
dp[0][2] = NEG; // cool

for (int i = 1; i <= n; i++) {
    dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - price[i]);
    dp[i][1] = max(dp[i - 1][1], dp[i - 1][2]);
    dp[i][2] = dp[i - 1][0] + price[i];
}

return max(dp[n][1], dp[n][2]);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<ll> price(n + 1);
    for (int i = 1; i <= n; i++) cin >> price[i];

    cout << max_profit_with_cooldown(price) << '\n';
    return 0;
}

```

最小测试:

```

5
1 2 3 0 2
输出: 3

```

7. 升维清单

发现的问题	加什么维度	典型状态
上一步动作影响下一步	lastMove	dp[i][j][lastMove]
相邻不能相同	lastColor/lastValue	dp[i][last]
当前元素选没选影响下一个	take	dp[i][0/1]
机会是否用过	used	dp[i][j][used]
还剩几次机会	rest	dp[i][rest]
处于持有/冷却/空闲	phase	dp[i][phase]
已访问哪些点	mask	dp[mask][last]

8. 从 DFS 角度理解升维

如果你先写 DFS, 升维会更自然:

不能连续向下:

```
dfs(i, j, lastMove)
```

不能连续同样跳法:

```
dfs(pos, lastStep)
```

恰好选 K 且不能相邻:

```
dfs(i, cnt, prevTaken)
```

最多一次翻倍:

```
dfs(i, j, used)
```

股票冷冻期:

```
dfs(day, phase)
```

这时 DFS 的可变参数就是 memo 的 key, 也是表推 DP 的下标。

常见坑:

- 忘记把影响未来的变量放进 memo key，导致不同历史被错误合并。
- 加了太多无关历史，状态数爆炸。
- last 的初始值没有设计好；可以用特殊值，或像网格例题那样给起点两个虚拟方向。
- 最大值题初始不可达状态要用 NEG，不要默认 0。
- 允许负收益时，默认 0 会错误地制造一条不存在的路径。

暴力/部分替代：

- 先写 dfs(..., last/used/phase)。
- 小数据不加 memo 也能跑。
- 中数据加 memo。
- 状态小且无环后，再改成表推 DP。

升级方向：

错误 dp[i][j]

-> 找到导致未来不同的历史 last/used/phase

-> dp[i][j][last]

-> memo DFS 或表推

-> 如果维度太大，压缩/换模型

最小测试样例：

网格不能连续向下：

```
3 2
1 1
100 1
100 1
输出：103
```

爬楼梯不能连续同样跳法：

```
4
输出：1
```

恰好选 K 且不能相邻：

```
5 2
2 7 9 3 1
输出：11
```

DP-27：高阶 DP 优化与少见模型索引

模型编号：DP-27

模型名称：高阶 DP 优化与少见模型索引

标签：DP、优化、SOS、斜率、分治、Knuth、轮廓线、自动机、概率期望

一句话用途：遇到“朴素 DP 会写但明显超时”的题时，判断是否有高阶优化信号；若不能确认条件，先交朴素/记忆化版本，不要硬套。

题面触发词：

- $n \leq 1e5/2e5$ ，但自然 DP 是 $O(n^2)$ 。
- $dp[i] = \min/\max(dp[j] + cost(j, i))$ 。
- 子集的所有子集、超集统计。
- 网格逐格填充、插头、连通性轮廓。
- 多模式串匹配后再做 DP。
- 随机过程有自环、期望方程组。

什么时候用：

- 朴素状态和转移已经写清楚。
- 能说出优化利用的结构：单调性、凸性、区间包含、子集包含、自动机状态、方程线性。
- 数据范围迫使你从 $O(n^2)$ 降到 $O(n \log n)$ 、 $O(n)$ 或 $O(3^n)$ 降到 $O(n2^n)$ 。

不要什么时候用：

- 状态定义还没想清楚。
- 无法证明决策点单调、代价满足四边形不等式或斜率有序。

- 朴素版本已经能拿不少分时，不要把所有时间赌在高阶优化上。

复杂度：

- SOS DP: 常见 $O(n^2 \log n)$ 。
- 分治 DP 优化: 常见 $O(k n \log n)$ 或 $O(k n)$ 级别，依题而定。
- Knuth 优化: 典型区间 DP 从 $O(n^3)$ 到 $O(n^2)$ 。
- 斜率优化: 常见 $O(n \log n)$ 或 $O(n)$ 。
- 轮廓线/插头 DP: 常见 $O(nm \times \text{状态数})$ ，实现难度高。

依赖的标准容器：

- `vector<ll> dp, ndp`
- `deque<int>`: 单调队列/凸包队列
- `vector<int> f(1 << n)`: SOS DP
- `map/unordered_map`: 轮廓状态稀疏存储

输入如何整理：

- 先写朴素 DP 公式。
- 把“枚举前驱”的条件单独圈出来。
- 判断前驱集合是否有序、是否是前缀/区间、是否和子集包含有关。

接口：

朴素 DP 公式 -> 判断结构 -> 选择优化；不满足条件则回退朴素/记忆化。

1. 高阶 DP 路由表

看到的形状	可能模型	先问自己
$dp[i] = \min_j dp[j] + cost(j, i)$, 最优 j 随 i 单调	分治 DP / Knuth / 单调队列	决策点真的单调吗?
区间 DP 的最优断点可夹逼	Knuth / 四边形优化	是否有 $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$?
直线 $y=kx+b$, 查询某个 x 的最小/最大	斜率优化 / Convex Hull Trick	斜率和查询 x 是否单调?
对每个集合统计所有子集/超集贡献	SOS DP	$n \leq 20 \dots 22$ 且状态是 bitmask
网格按格子填, 状态只和上一行/当前边界有关	轮廓线 DP	n, m 较小, 状态数可控
字符串匹配状态 + 选择/计数	自动机 DP	能否先建 Trie/KMP/AC 自动机
随机状态会回到自己	期望方程	是否要移项或高斯消元

2. 可靠使用顺序

- 写出能过小数据的朴素 DP 或记忆化。
- 用小样例/随机小数据确认朴素答案。
- 只优化“最慢的一层枚举”。
- 优化后保留朴素版本作为对拍或部分分版本。

如果第 1 步都写不出来，不要直接翻高级优化；先回 DP-00/DP-03/DP-25。

3. SOS DP 最短模板

适用: $f[\text{mask}]$ 已知，想快速得到每个 mask 的所有子集和。

```
// sub_sum[mask] = sum of f[sub] for all sub subset of mask
vector<long long> sub_sum = f;
for (int bit = 0; bit < n; bit++) {
    for (int mask = 0; mask < (1 << n); mask++) {
        if (mask & (1 << bit)) {
            sub_sum[mask] += sub_sum[mask ^ (1 << bit)];
        }
    }
}
```

如果要统计所有超集，把判断改成 `if (!(mask & (1 << bit)))`，从 `mask | (1 << bit)` 转移。

4. 分治 DP 优化只做索引

常见形状:

$dp[g][i] = \min \text{ over } k < i \text{ of } dp[g-1][k] + \text{cost}(k+1, i)$

如果最优决策点 $opt[g][i]$ 随 i 单调不下降, 可以用分治优化。若题目没给性质, 自己又证明不了, 考场不要硬上。

部分分策略:

- $n \leq 5000$: 先写 $O(k \cdot n^2)$ 。
- $\text{cost}(l, r)$ 频繁出现: 先用前缀和把代价算到 $O(1)$ 。
- 优化不稳: 提交朴素版本, 避免满盘皆输。

5. Knuth 优化只做索引

常见区间 DP:

$dp[l][r] = \min \text{ over } k \text{ in } [l, r-1] \text{ of } dp[l][k] + dp[k+1][r] + \text{cost}(l, r)$

如果满足最优断点单调:

$opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$

可把枚举 k 的范围缩小, 常从 $O(n^3)$ 到 $O(n^2)$ 。典型是合并石子一类, 但不是所有区间 DP 都能用。

6. 斜率优化只做索引

常见形状:

$dp[i] = \min_j (dp[j] + A[j] * X[i] + B[j])$

把每个 j 看成一条直线, 查询 $x=X[i]$ 时的最小值。若斜率加入顺序和查询 x 都单调, 可以用 deque; 否则要 Li Chao Tree 或平衡结构, 考场实现风险较高。

部分分策略: 朴素 $O(n^2)$ + 剪枝/记忆化; 若只差一层, 可先看 DP-18 的 树状数组/SegmentTree 是否能替代。

7. 轮廓线/插头 DP 只放最后

触发词:

- 小网格内铺砖、连通性、回路数量。
- 逐格处理, 当前格子只和左边/上边边界有关。

建议:

- 基础弱时不要把它放前排。
- 小数据可用 DFS/状态 BFS。
- 如果题面真考这种, 先拿特判、暴力和小网格分。

8. 自动机 DP

触发词:

- 字符串生成/选择, 同时禁止或要求出现某些模式串。
- 统计长度为 n 的字符串数量。

建模口令:

状态 = 处理到第 i 位 + 当前自动机节点 + 是否已经满足条件

最后一步 = 追加一个字符, 沿自动机转移到新节点

可拼接模块: KMP/Trie/AC 自动机 + 计数 DP。若 AC 自动机来不及写, 小数据用字符串暴力生成或 KMP 检查拿部分分。

常见坑:

- 没写朴素转移就急着优化, 状态含义会漂。
- 决策单调性没有证明, 分治/Knuth 可能直接错。
- SOS DP 的“子集”和“超集”方向写反。
- 轮廓状态编码过复杂, 容易调不完。
- 期望有自环时忘记移项。

暴力/部分分替代:

- 先保留朴素 DP。

- 状态少时写 DFS+memo。
- 高阶优化不会证明时，用随机小数据对拍，至少保证提交版本不炸。

升级方向：

朴素 DP -> 前缀和/树状数组/单调队列 -> 高阶优化索引

DFS+memo -> 表推 -> 只优化瓶颈枚举

高级模型不会 -> 小数据暴力 + 特判 + 合法兜底

最小测试样例：

SOS：

n=2

f[0]=1 f[1]=2 f[2]=4 f[3]=8

sub_sum[3]=1+2+4+8=15